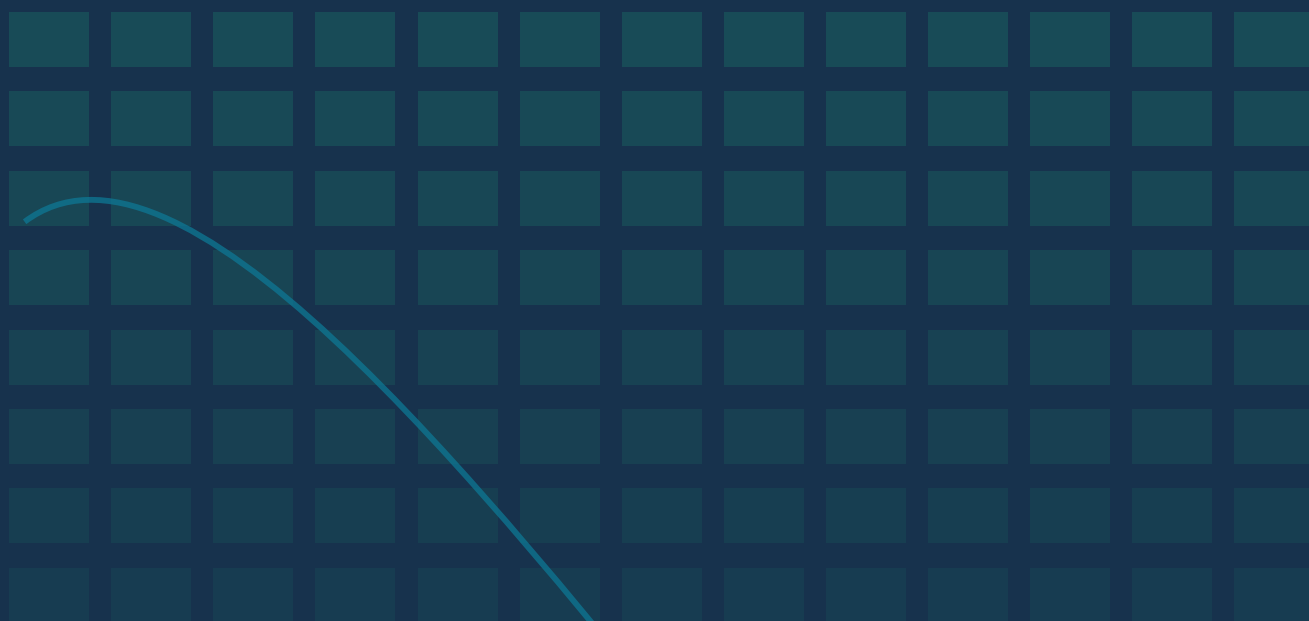


大规模并行处理器编程

中文精讲与实战：由浅入深学会 CUDA

基于 *Programming Massively Parallel Processors*
第五版的中文精讲教程



面向基础较弱学习者的分层教程

先修补课 · 逐行代码 · 定量推导 · 优化实验 · AI 与多 GPU

2026 年 8 月中文精讲修订版

大规模并行处理器编程

中文精讲与实战：由浅入深学会 CUDA

基于用户提供的下列教材整理

Programming Massively Parallel Processors: A Hands-on Approach

Fifth Edition

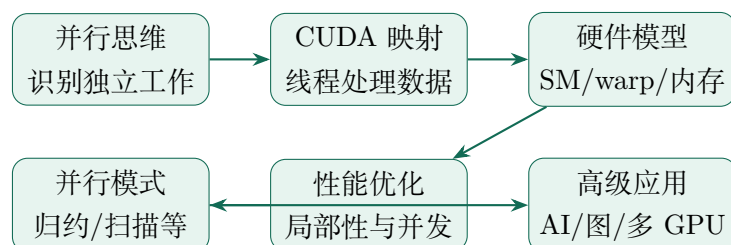
Wen-mei W. Hwu, David B. Kirk, Izzat El Hajj

本教程是面向学习的原创重组与讲解，不替代原书。章节后均标注原书对应范围；示意图与代码均重新绘制或重新编写。

使用说明：怎样真正学会这本书

这不是原书的逐章缩写，也不是概念名词表。本修订版把每个核心主题拆成三个层次：**第一层：先会用**，先通过一个可运行的小例子知道“代码怎样写”；**第二层：再理解**，再从线程映射、数据依赖和硬件执行解释“为什么正确”；**第三层：最后优化**，最后用访存量、同步次数、占用率和算术强度解释“为什么快或为什么慢”。

这本教材的知识跨度很大：它既讲 CUDA 语法，又讲 GPU 硬件，还讲内存优化、并行算法模式、深度学习与多 GPU。对基础较弱的学习者，最容易出现两种问题：一是逐页阅读但没有形成结构；二是代码能照抄，却不能解释为什么快或为什么错。本教程把原书重新组织成一条可执行的学习路径：



学习目标

完成本教程后，你应能做到：

- › 从顺序代码中识别数据并行性，把循环迭代映射为线程；
- › 独立编写、编译、检查一个完整 CUDA C++ 程序；
- › 解释 grid、block、thread、warp、SM、occupancy 的关系；
- › 判断程序是计算受限、带宽受限还是延迟受限；
- › 使用共享内存分块、合并访存、私有化、线程粗化、循环展开和双缓冲；
- › 掌握卷积、模板、直方图、归约、扫描、过滤、合并、排序等并行模式；
- › 理解稀疏矩阵、图遍历、CNN、注意力、KV cache、FlashAttention 与多 GPU 的并行化思路；
- › 用“问题表述 - 算法选择 - 问题分解 - 优化 - 验证”的流程处理新问题。

修订版的学习层次

从浅到深

第一层：先会用

给出最小背景、完整代码、具体输入和预期输出。此阶段不要求一次理解所有硬件细节，但必须能独立运行并检查结果。

第二层：再理解

对每一行关键代码回答：哪个处理器执行、创建了多少线程、每个线程访问什么地址、是否存在依赖、为什么需要边界检查或同步。

第三层：最后优化

先建立可测量的基线，再计算理论流量和运算量，使用 profiler 验证瓶颈，最后一次只改变一个优化因素。

全书的必学主线是第 1–13 章：它覆盖 C++ 基础、CUDA 编程模型、GPU 架构、内存体系和主要并行模式。第 14–18 章用于把这些能力迁移到动态规划、稀疏计算、图、深度学习、大语言模型和多 GPU。基础薄弱时，不要为了“读完”而跳过前六章；后面的每个优化都依赖前面的线程映射、warp 和内存局部性。

推荐的 12 周学习安排

周次	主题	必须完成的任务	验收标准
1	C++ 补课与并行思维	指针、数组、行主序；手算 Amdahl 定律	能解释指针和数组下标
2	第一个 CUDA 程序	向量加法；错误检查；改变 block 大小	可独立编译运行
3	多维网格与矩阵	灰度化、模糊、朴素矩阵乘	正确处理边界
4	GPU 执行模型	warp、分支发散、同步、occupancy	能画出层次图
5	内存与分块	共享内存矩阵乘；计算算术强度	能解释为什么减少 DRAM 流量
6	通用优化清单	合并访存、bank conflict、粗化、展开	能用 profiler 找瓶颈
7	卷积与 stencil	写基本核与 tiled 核	比较数据复用率

8	直方图与归约	原子、私有化、树形归约、warp shuffle	理解竞争与同步
9	扫描与过滤	Kogge-Stone; 稳定过滤	能解释 work efficiency
10	合并、排序与高级 GEMM	co-rank、radix、寄存器分块	能描述优化阶梯
11	稀疏、图、动态规划	SpMV、BFS、wavefront	会根据数据分布选策略
12	AI 与多 GPU + 项目	attention/KV cache; halo exchange	完成一个可复现实验报告

💡 直觉

学习 CUDA 不是记住 API，而是训练一种固定的提问顺序：

- 1) 工作能否拆成大量相互独立的小任务？
- 2) 每个线程负责什么数据？
- 3) 数据从哪里来、会被复用几次、要写到哪里？
- 4) 同一 warp 的线程是否走同一控制流、访问相邻地址？
- 5) 为了减少流量或同步，应该改变算法还是只改代码？

后续每章都围绕这五个问题展开。

每章的学习动作

不要只读。每章按下面四步执行：

1. **先画图**：画线程到数据的映射，标出输入、输出和共享数据。
2. **再手算**：至少手算一个小规模例子，包括索引、线程数、内存访问量。
3. **后编码**：先写正确的基线，再一次只加入一个优化。
4. **做解释**：记录“改了什么、理论上减少了什么、测量结果是否一致”。

内容与原书的关系

原书面向已有基本 C++ 经验的理工科学生和工程人员，以 CUDA 为具体载体，通过直觉和实践教授并行编程。本教程额外加入了 C++ 补课、逐步推导、中文注释、学习

检查点和项目模板，并按照“基础概念 - 并行模式 - 高级应用”的主线压缩重复内容。所有技术主题仍遵循原书第五版的章节范围。

 易错点

本教程使用的是稳定的概念模型，不把某一代 GPU 的具体数字当成永久规则。诸如“每个 block 的最大线程数”“每个 SM 的寄存器数”“集群支持”等都应在实际设备上查询，而不是硬编码。

目录

使用说明：怎样真正学会这本书	iii
第一部分 建立并行思维与 CUDA 基础	1
第 1 章 学习前准备：补齐最少的 C++、数学与实验能力	2
1.1 只学 CUDA 所需的 C++ 子集	2
1.1.1 变量、类型与字节数	2
1.1.2 指针：把它理解为地址，而不是神秘符号	3
1.1.3 动态内存与资源生命周期	3
1.2 多维数组为什么最终是一维地址	4
1.3 只需掌握的数学工具	4
1.3.1 求和、点积与矩阵乘	4
1.3.2 数量级与单位	5
1.4 实验环境与最小 workflow	5
1.5 正确性验证模板	6
1.6 由浅入深精讲：地址、指针与二维数据	6
1.6.1 变量名、值与地址是三个不同概念	6
1.6.2 数组名为什么能当指针使用	7
1.6.3 二维数组线性化：从具体数字开始	7
1.6.4 三维张量：先平面、再行、再列	8
1.7 编译、运行与错误定位的最小闭环	8
第 2 章 为什么需要 GPU：从顺序执行到吞吐导向	10
2.1 CPU 和 GPU 优化的是不同目标	10
2.2 加速比：先明确比较对象	11
2.3 Amdahl 定律：整体速度受不可加速部分限制	11
2.4 数据并行与任务并行	11
2.4.1 数据并行	11

2.4.2	任务并行	12
2.5	从顺序循环中找并行性	12
2.6	并行编程真正困难在哪里	13
2.7	并行化决策表	13
2.8	由浅入深精讲：怎样判断一段循环能否并行	14
2.8.1	用读写集合分析依赖	14
2.8.2	“可以并行”不等于“值得放到 GPU”	14
2.8.3	Amdahl 定律的逐步例题	15
2.9	从问题描述到线程工作的五步法	15
2.10	工作效率：线程数量不是唯一指标	16
第 3 章	第一个完整 CUDA 程序：向量加法	17
3.1	一段 CUDA 程序包含两个执行世界	17
3.2	函数限定符	18
3.3	完整的向量加法程序	18
3.4	逐行理解 kernel	20
3.4.1	线程的全局一维索引	20
3.4.2	为什么必须边界检查	20
3.5	设备内存 API 的参数方向	20
3.5.1	cudaMalloc	20
3.5.2	cudaMemcpy	21
3.5.3	cudaFree	21
3.6	kernel 启动语法	21
3.7	编译过程的概念模型	21
3.8	性能上为什么这个版本还不理想	22
3.9	由浅入深精讲：把完整程序拆成七个阶段	22
3.9.1	线程索引公式不是魔法	24
3.9.2	上取整分块的整数推导	24
3.9.3	kernel 启动为什么看起来“立即返回”	25
3.10	逐项计算数据量和理论特征	25
3.11	从一线程一元素到 grid-stride loop	25
3.12	把端到端时间拆开测量	26

第 4 章 多维网格与数据：把线程准确映射到像素和矩阵	27
4.1 grid 和 block 最多可表达三维结构	27
4.2 二维线程坐标	28
4.3 行主序线性化	28
4.4 案例一：RGB 图像转灰度	28
4.5 案例二：图像模糊与邻域访问	29
4.6 案例三：朴素矩阵乘	30
4.7 选择 block 形状的初步原则	30
4.8 线程到数据映射的通用模板	31
4.9 由浅入深精讲：二维线程怎样覆盖二维数据	32
4.9.1 先算一个完整覆盖例子	32
4.9.2 通用二维 kernel 骨架	32
4.10 灰度转换：从数学式到地址式	33
4.11 图像模糊：一个输出开始读取邻域	33
4.12 矩阵乘：一线程一输出的完整矩形版本	34
4.12.1 手算一个线程	34
4.13 block 形状不是只看线程总数	35
 第二部分 理解硬件并系统优化	 36
第 5 章 GPU 执行架构：SM、warp、同步与 occupancy	37
5.1 从软件层次映射到硬件层次	37
5.2 block 调度带来的透明可扩展性	38
5.3 barrier：只保证范围内的“阶段完成”	38
5.3.1 死锁的典型写法	38
5.4 warp 与 SIMT	39
5.5 控制发散为什么损失吞吐	39
5.5.1 不是所有 if 都同样糟	39
5.6 延迟隐藏：GPU 不必让一次内存访问变得很短	40
5.7 occupancy：可驻留 warp 比例，不等于性能比例	40
5.8 性能悬崖	41
5.9 设备属性查询	41

5.10 由浅入深精讲：从 grid 到 SM 的实际执行过程	42
5.10.1 一个 grid 不会一次性全部“铺”在 GPU 上	42
5.10.2 block 如何切成 warp	42
5.11 控制发散：损失来自“分路径执行”	43
5.11.1 边界分支通常只影响少数 warp	43
5.11.2 数据相关分支可能影响所有 warp	43
5.12 同步：barrier 同时约束控制与内存可见性	44
5.13 延迟隐藏：为什么需要很多驻留 warp	44
5.14 occupancy 的定量计算	45
5.15 设备属性与 kernel 资源都要查询	45
第 6 章 内存体系与数据局部性：性能的核心	47
6.1 为什么“算得快”常常输给“拿数据慢”	47
6.2 CUDA 中常见内存类型	48
6.3 tiling：把大问题切成能在片上复用的小块	48
6.4 共享内存矩阵乘	49
6.4.1 为什么加载阶段不能提前 return	49
6.5 流量与复用的粗略分析	50
6.6 constant memory 的适用场景	50
6.7 共享内存与 occupancy 的权衡	50
6.8 局部性有两种	50
6.9 由浅入深精讲：先算流量，再决定是否使用 shared memory	51
6.10 内存层次不是简单的“快慢表”	51
6.11 朴素矩阵乘的流量为什么高	52
6.12 shared-memory tiling 的三阶段	52
6.12.1 为什么无效线程也不能提前退出	53
6.13 定量比较复用	54
6.14 global memory 合并访问的地址视角	54
6.15 shared memory bank conflict 的具体例子	54
6.16 寄存器复用与 local memory spill	55
6.17 Unified Memory：简化可见性，不自动保证高性能	55
第 7 章 通用性能优化清单：从访存到双缓冲	56

7.1	先建立优化顺序	56
7.2	global memory 合并访问	56
7.2.1	连续访问	57
7.2.2	跨步访问	57
7.2.3	矩阵转置的经典问题	57
7.3	隐藏延迟的三个来源	57
7.4	向量加载与存储	57
7.5	shared memory bank conflict	58
7.6	线程粗化：一个线程计算多个输出	58
7.7	循环展开	59
7.8	双缓冲与软件流水	59
7.9	优化清单	59
7.10	测量策略	60
7.11	由浅入深精讲：性能优化先做定量诊断	61
7.12	第一步：定义你到底在测什么	61
7.12.1	GPU event 计时的完整注意事项	61
7.13	有效带宽：内存型 kernel 的第一指标	62
7.14	Roofline：把计算上限和带宽上限放在一起	62
7.15	合并访问的两个对照 kernel	63
7.16	延迟受限、带宽受限和计算受限的区别	64
7.17	线程粗化的定量权衡	64
7.18	循环展开和寄存器压力	64
7.19	双缓冲的时序模型	64
7.20	优化实验记录模板	65
 第三部分 掌握可迁移的并行计算模式		66
 第 8 章 卷积与 stencil：从邻域计算学习 halo tiling		67
8.1	模式本质：每个输出依赖一个局部邻域	67
8.2	基本卷积 kernel	67
8.3	输出 tile、输入 tile 和 halo	68
8.4	二维 tiled 卷积骨架	69

8.5	stencil 的迭代与 ping-pong buffer	70
8.6	线程粗化与寄存器 tiling	70
8.7	边界条件要先定义数学语义	70
8.8	性能分析	70
8.9	由浅入深精讲：邻域计算中的 tile 与 halo	71
8.10	先从一维卷积看清下标	71
8.11	输出 tile 与输入 tile 的尺寸关系	72
8.11.1	加载线程数不必等于输出线程数	72
8.12	二维卷积的输入 tile	73
8.13	constant memory 适合小而只读的卷积核	73
8.14	stencil 与卷积的相同和不同	73
8.14.1	为什么块内 barrier 不能完成所有迭代	73
8.15	线程粗化与寄存器滑窗	74
第 9 章	直方图：原子操作、竞争与私有化	75
9.1	为什么普通自增不正确	75
9.2	基本原子版本	75
9.3	作用域与内存顺序	76
9.4	私有化：先局部聚合，再全局合并	76
9.5	私有化层次的权衡	77
9.6	线程粗化与 grid-stride loop	77
9.7	热点分布与性能	77
9.8	正确性与溢出	78
9.9	由浅入深精讲：原子操作到底解决了什么	78
9.10	普通自增不是一个不可分割动作	78
9.11	基本版本的成本模型	79
9.12	block 私有化为什么有效	79
9.13	私有化层次怎样选	79
9.14	作用域与内存顺序只使用所需强度	80
9.15	grid-stride 与线程粗化	80
第 10 章	归约：树形并行、同步与 warp 原语	81
10.1	归约模式	81

10.2 树形归约	81
10.3 block 级 shared-memory 归约	82
10.4 减少控制和内存发散	82
10.5 warp shuffle: 最后 32 个值不必反复走 shared memory	83
10.6 任意长度与单位元	83
10.7 线程粗化	83
10.8 数值稳定性	84
10.9 由浅入深精讲: 从顺序求和到树形归约	85
10.10 为什么不能让所有线程直接写一个 sum	85
10.11 8 个元素的完整树	85
10.12 block 级 shared 归约的稳定骨架	85
10.12.1 为什么这种索引比交错索引好	86
10.13 最后一个 warp: 用 shuffle 交换寄存器值	86
10.14 多 block 归约的两阶段	86
10.15 单位元与任意长度	87
10.16 浮点加法顺序改变结果	87
10.17 归约的性能账本	87
第 11 章 扫描与过滤: 把顺序依赖改写为并行前缀	89
11.1 扫描是什么	89
11.2 为什么顺序写法不能直接线程化	89
11.3 Kogge-Stone 扫描	90
11.4 双缓冲与 warp 原语	90
11.5 工作效率	91
11.6 跨 block 的全局扫描	91
11.7 过滤: 保留满足谓词的元素	91
11.7.1 不稳定过滤: 原子分配位置	92
11.7.2 稳定过滤: predicate + exclusive scan + scatter	92
11.8 in-place 稳定过滤为何困难	92
11.9 过滤优化选择	93
11.10 由浅入深精讲: 扫描怎样重构顺序依赖	93
11.11 inclusive 与 exclusive 的具体区别	93
11.12 Kogge-Stone: 8 个元素逐阶段	94

11.13 工作效率：为什么 Kogge–Stone 不是唯一方案	94
11.14 跨 block 扫描的三步分解	94
11.15 稳定过滤：scan 把布尔谓词变成写位置	95
11.16 不稳定过滤为什么可以更简单	95
11.17 scan 的典型用途	96
第 12 章 合并与排序：数据相关分区和规则化重排	97
12.1 顺序 merge	97
12.2 co-rank：输出前缀对应多少 A 和 B	97
12.3 并行 merge 分区	98
12.4 tiled merge 与 circular buffer	98
12.5 排序方法概览	99
12.5.1 odd-even sort	99
12.5.2 merge sort	99
12.5.3 radix sort	99
12.6 radix 大小的权衡	99
12.7 稳定性与键值对	100
12.8 内存流量是排序的主要成本	100
12.9 由浅入深精讲：co-rank 如何把一个 merge 切成独立任务	101
12.10 输出前缀对应输入前缀	101
12.11 并行分区	101
12.12 为什么需要 tiled merge	101
12.13 radix sort 是多个模式的组合	102
12.14 稳定排序和键值对	102
第 13 章 高级矩阵乘优化：从 shared tile 到寄存器分块	103
13.1 优化阶梯	103
13.2 多级数据复用	103
13.3 每线程计算多个输出	104
13.4 较大 block tile 不等于更多线程	105
13.5 global load 的布局与向量化	105
13.6 合并写回	105
13.7 bank conflict 与 shared layout	105

13.8 软件流水与双缓冲	105
13.9 专用矩阵硬件	106
13.10 何时自己写，何时用库	106
13.11 性能建模检查	106
13.12 由浅入深精讲：高性能 GEMM 的多级复用	107
13.13 GEMM 为什么值得复杂优化	107
13.14 三级 tile 的概念	107
13.15 寄存器外积更新	108
13.16 global load 的向量化与布局	108
13.17 合并写回	108
13.18 bank conflict 的消除不是只加一列	109
13.19 软件流水：加载下一块时计算当前块	109
13.20 一个简化资源预算	109
13.21 与专用硬件的关系	109
13.22 逐级优化的实验顺序	109

第四部分 高级模式、AI 与多 GPU 111

第 14 章 动态规划与 wavefront：沿依赖前沿提取并行性	112
14.1 动态规划为何“不完全并行”	112
14.2 最直接方法：一条反对角线一个 kernel	113
14.3 Floyd-Warshall 的阶段依赖	113
14.4 Smith-Waterman 序列比对	113
14.5 block-level tiling	113
14.6 超平面/倾斜变换	114
14.7 同步层次设计	114
14.8 负载与边界	114
14.9 正确性验证	115
14.10 由浅入深精讲：依赖图怎样变成 wavefront	115
14.11 先从二维依赖图开始	116
14.12 单元格级 kernel 的局限	116
14.13 tile 级 wavefront	116

14.14 Floyd–Warshall 的阶段依赖	117
14.15 Smith–Waterman 的边界数据	117
14.16 超平面变换的直觉	117
第 15 章 稀疏矩阵与图遍历：数据结构决定并行效率	118
15.1 为什么稀疏问题难	118
15.2 常见稀疏格式	118
15.3 CSR SpMV 的映射选择	119
15.3.1 一线程一行	119
15.3.2 一 warp 一行	119
15.4 格式选择是算法的一部分	120
15.5 图与稀疏矩阵的联系	120
15.6 BFS 的层次语义	120
15.6.1 vertex-centric push	120
15.6.2 edge-centric	120
15.6.3 pull	120
15.7 frontier 压缩	121
15.8 降低竞争与负载不均	121
15.9 图结构决定性能	121
15.10 由浅入深精讲：不规则数据为什么先要选表示	122
15.11 用一个小矩阵理解 CSR	122
15.12 一线程一行的负载不均	123
15.13 格式是性能算法的一部分	123
15.14 BFS 与稀疏矩阵的联系	123
15.15 push、pull 与 edge-centric	124
15.15.1 push	124
15.15.2 pull	124
15.15.3 edge-centric	124
15.16 frontier 压缩是 filter/scan 的应用	124
15.17 局部性与编号重排	124
第 16 章 深度学习与大语言模型：卷积、注意力、KV cache 与 FlashAttention	125
16.1 深度学习层本质上是并行模式组合	125

16.2 CNN 卷积层	126
16.2.1 直接卷积	126
16.2.2 卷积转 GEMM	126
16.3 为什么使用深度学习库	126
16.4 Transformer 的核心数据流	127
16.4.1 形状检查	127
16.5 朴素 attention 的多 kernel 实现	127
16.6 稳定 softmax	127
16.7 自回归解码与 KV cache	128
16.8 KV cache 布局	128
16.9 FlashAttention 的核心：不物化完整 L^2 矩阵	128
16.10 causal mask 与不规则 tile	129
16.11 attention 的算术强度与序列长度	129
16.12 batching：吞吐与延迟的交换	130
16.13 融合与数值精度	130
16.14 由浅入深精讲：把 Transformer 拆回熟悉的并行模式	131
16.15 从一个具体形状走完整个 attention	131
16.15.1 计算量的数量级	131
16.16 softmax 为什么必须先减最大值	131
16.17 朴素实现的 IO 账本	132
16.18 在线 softmax 合并的详细推导	132
16.19 KV cache：按实际 KV 头数计算	133
16.20 prefill 与 decode 是两类不同 kernel 形态	133
16.20.1 prefill	133
16.20.2 decode	133
16.21 KV cache 布局的地址问题	133
16.22 causal mask 与 tile 跳过	134
16.23 batching 的定量视角	134
16.24 与前面并行模式的对应	135
第 17 章 问题表述与算法选择：从静电势到通用计算思维	136
17.1 案例：静电势图	136
17.1.1 input-centric scatter	136

17.1.2 output-centric gather	136
17.2 cutoff: 用可控误差换复杂度	137
17.3 线程粗化与合并访存	137
17.4 通用问题求解流程	137
17.4.1 第一步: 问题表述	137
17.4.2 第二步: 算法选择	138
17.4.3 第三步: 问题分解	138
17.4.4 第四步: 优化	138
17.5 将 Amdahl 定律放回应用层	138
17.6 batching: 不只是把维度前面加一个 B	138
17.7 性能目标要可验收	139
17.8 决策示例: 图像模糊	139
17.9 由浅入深精讲: 先改变问题, 再优化 kernel	140
17.10 scatter 与 gather 的正确性和性能差异	140
17.11 cutoff 与 binning 的推导思路	140
17.12 通用决策清单	141
第 18 章 多 GPU 编程: 域分解、halo exchange 与通信重叠	142
18.1 从单 GPU 到多 GPU 的第一步: 分数据	142
18.2 通信模型一: CUDA-aware MPI	143
18.3 通信模型二: NCCL	143
18.4 通信模型三: NVSHMEM	143
18.5 计算与通信重叠	144
18.6 多 GPU 正确性	144
18.7 扩展性指标	145
18.7.1 强扩展	145
18.7.2 弱扩展	145
18.7.3 表面积/体积比	145
18.8 拓扑与通信路径	145
18.9 MPI、NCCL、NVSHMEM 如何选	145
18.10 由浅入深精讲: 多 GPU 的核心是数据所有权与通信	146
18.11 先定义 ownership	146
18.12 通信量与计算量的表面积/体积比	147

18.13	CUDA-aware MPI 的语义	147
18.14	NCCL 更适合 collective	147
18.15	NVSHMEM 的 one-sided 视角	148
18.16	计算通信重叠的必要条件	148
18.17	强扩展与弱扩展公式	148
18.18	多 GPU 正确性检查	148
第 19 章	综合项目：把知识变成可复现的 GPU 优化能力	150
19.1	项目不是“写一个很长的 kernel”	150
19.2	推荐项目一：二维卷积优化阶梯	150
19.3	推荐项目二：归约/扫描原语库	151
19.4	推荐项目三：小型 attention kernel	151
19.5	推荐项目四：稀疏矩阵格式自适应	151
19.6	推荐项目五：多 GPU stencil	151
19.7	项目设计文档模板	151
19.7.1	1. 问题与动机	151
19.7.2	2. 数学与顺序基线	152
19.7.3	3. 并行分解	152
19.7.4	4. 优化假设	152
19.7.5	5. 验证计划	152
19.7.6	6. 性能计划	152
19.8	实验表格模板	153
19.9	常见无效实验	153
19.10	最终报告结构	153
19.11	毕业检查：能否回答这些问题	154
19.12	由浅入深的项目执行计划	154
19.13	六个里程碑	154
19.14	每次提交必须回答的五句话	155
19.15	建议的最小最终成果	155
第五部分	附录	157
附录 A	数值计算基础：浮点表示、误差与低精度	158

A.1	浮点数不是实数	158
A.2	机器精度与误差模型	158
A.3	特殊值	158
A.4	并行归约的非确定末位	159
A.5	低精度的收益与风险	159
A.6	稳定算法比更高精度更重要	159
A.7	误差报告模板	159
附录 B	CUDA 快速参考与性能检查表	161
B.1	最常用索引公式	161
B.2	核心 API	161
B.3	kernel 正确性检查	161
B.4	性能检查	162
B.5	模式选择速查	163
附录 C	部分练习答案与推导	164
C.1	第 1 章: 线性化	164
C.2	第 2 章: Amdahl 定律	164
C.3	第 3 章: 线程数量	164
C.4	第 4 章: 二维图像覆盖	164
C.5	第 5 章: warp 数	165
C.6	第 6 章: tiled GEMM 边界	165
C.7	第 8 章: halo 比例	165
C.8	第 10 章: 归约 partial 数量	165
C.9	第 11 章: 稳定过滤	165
C.10	第 12 章: co-rank 小例子	166
C.11	第 13 章: 寄存器 tile	166
C.12	第 16 章: attention 形状	166
C.13	第 16 章: KV cache 估算	166
C.14	第 18 章: 三 GPU 行分区	167
附录 D	术语表	168
	结语与参考教材	170

第一部分

建立并行思维与 CUDA 基础

第 1 章 学习前准备：补齐最少的 C++、数学与实验能力

📖 **原书对应：**教程补充；用于满足原书“具备基本 C++ 编程经验”的先修要求，并为第 2-3 章做准备。

🎯 本章学习目标

- 能读懂指针、动态数组、函数参数、结构体和基本模板代码；
- 能把二维或三维数据转换为一维地址；
- 能区分“程序正确”“程序能运行”“程序性能好”；
- 建立可重复的编译、测试、计时和记录习惯。

1.1 只学 CUDA 所需的 C++ 子集

CUDA C++ 是 C++ 的扩展。初学阶段不需要先掌握整个现代 C++，但下面的知识必须熟练。

1.1.1 变量、类型与字节数

变量有**值**、**类型**和**存储位置**。GPU 编程中，字节数直接决定数据传输量和内存带宽压力，因此不能只把类型看成语法。

常见类型	常见大小	典型用途	注意事项
<code>int</code>	4 B	索引、计数	规模很大时可能溢出
<code>size_t</code>	8 B (常见 64 位系统)	字节数、容器长度	无符号减法容易出错
<code>float</code>	4 B	通用 GPU 数值计算	精度约 7 位十进制有效数字
<code>double</code>	8 B	高精度计算	带宽与吞吐代价通常更高
<code>unsigned char</code>	1 B	图像通道、原始字节	算术时注意类型提升

数组含 n 个 `float` 元素时，占用字节数为

$$\text{bytes} = n \times \text{sizeof}(\text{float}).$$

这正是后续 `cudaMalloc` 和 `cudaMemcpy` 需要的第三类信息：内存 API 通常按**字节**工作，而算法习惯按**元素**工作。

1.1.2 指针：把它理解为地址，而不是神秘符号

Listing 1.1 指针、数组和函数参数的最小示例

```

1  #include <stdio>
2
3  void scale(float* x, int n, float alpha) {
4      for (int i = 0; i < n; ++i) {
5          x[i] *= alpha; // x[i] 等价于 *(x + i)
6      }
7  }
8
9  int main() {
10     float a[4] = {1.f, 2.f, 3.f, 4.f};
11     float* p = &a[0];
12     scale(p, 4, 2.f);
13     std::printf("%.1f\n", a[2]); // 6.0
14 }
```

关键关系只有三条：

$$p = \&a[0],$$

$p + i$ 指向第 i 个元素，

$$p[i] = *(p + i).$$

在 CUDA 中，主机指针和设备指针的变量写法相同，但它们指向不同的物理内存。**地址属于哪个地址空间**比“变量长得像什么”更重要。

⚠ 易错点

设备指针可以作为参数传给 CUDA API 或 kernel，但在普通 CPU 代码中不能随意解引用。写出 `A_d[0]` 并不代表 CPU 真能访问设备显存。统一内存等机制会改变可访问性，但不会消除同步和性能问题。

1.1.3 动态内存与资源生命周期

CPU 端最基本的资源生命周期是“申请 - 使用 - 释放”。

Listing 1.2 CPU 动态数组生命周期

```

1 int n = 1 << 20;
2 float* x = new float[n];
3 // ... initialize and use x ...
4 delete[] x;

```

CUDA 端是同样的思想，只是申请者、使用者和释放 API 不同。后续将看到：

cudaMalloc $\rightarrow$ kernel 使用 $\rightarrow$ cudaFree.

任何提前释放、漏释放、越界访问或错误的字节数都会造成错误；GPU 的异步执行还可能让错误在“较晚的 API 调用”才暴露。

1.2 多维数组为什么最终是一维地址

现代内存可以视为按地址编号的一维字节序列。C/C++ 的二维数组通常按**行主序**存放。一个高为 H 、宽为 W 的矩阵 A ，元素 $A_{r,c}$ 的一维下标是

$$\text{offset}(r, c) = rW + c.$$

三维张量若维度为 $D \times H \times W$ ，元素 (z, y, x) 的一维下标是

$$\text{offset}(z, y, x) = zHW + yW + x.$$

💡 直觉

二维下标转一维下标的口诀是：先跳过前面的完整行，再走到本行的目标列。三维则是：先跳过完整平面，再跳过完整行，最后走列。

📖 动手练习

一个 500×400 的矩阵采用行主序。请计算第 20 行、第 10 列的下标。再计算一个 $300 \times 500 \times 400$ 张量中 $(z, y, x) = (5, 20, 10)$ 的下标。答案见附录 C。

1.3 只需掌握的数学工具

1.3.1 求和、点积与矩阵乘

并行计算经常把一个大问题拆成大量相似的求和或点积：

$$\mathbf{x} \cdot \mathbf{y} = \sum_{k=0}^{K-1} x_k y_k, \quad C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} B_{k,j}.$$

GPU 并不是自动把公式变快，而是让许多输出元素或许多部分和同时计算。真正困难的是如何安排数据，使输入被复用、输出不冲突、线程少等待。

1.3.2 数量级与单位

性能分析必须携带单位：

- 延迟：ns、 μ s、ms；
- 带宽：GB/s，表示每秒可搬运多少字节；
- 计算吞吐：FLOP/s，表示每秒浮点运算数；
- 算术强度：FLOP/B，表示每搬运一字节做多少计算。

若一个 kernel 处理 N 个元素，每个元素读取两个 float、写一个 float，并做一次加法，则理想化的数据流量约为 $12N$ B，运算量为 N FLOP，算术强度约为 $1/12$ FLOP/B。这类 kernel 通常更容易受内存带宽限制。

1.4 实验环境与最小工作流

本教程不绑定某一代 GPU。需要一个可用的 NVIDIA GPU、驱动和 CUDA 编译工具链。先运行：

Listing 1.3 环境自检命令

```
1 nvidia-smi
2 nvcc --version
```

然后坚持下面的实验工作流：

1. 编译时打开警告，保留完整命令；
2. 先用很小输入验证结果，再扩大规模；
3. CPU 参考实现与 GPU 实现逐元素比较；
4. 每次只改变一个优化因素；
5. 预热若干次后计时，多次重复并报告中位数；
6. 记录 GPU 型号、编译选项、输入规模和 block 配置。

Listing 1.4 最小编译命令

```
1 nvcc -O3 -std=c++17 vector_add.cu -o vector_add
2 ./vector_add
```

1.5 正确性验证模板

浮点结果不应总用精确相等。常用判定是

$$|x_{\text{gpu}} - x_{\text{ref}}| \leq \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} |x_{\text{ref}}|.$$

其中绝对误差照顾接近零的数，相对误差照顾数值较大的结果。并行归约改变了加法顺序，因此即使算法正确，末位也可能与 CPU 顺序结果不同。

📌 掌握检查

完成本章后，应能不查资料回答：

- `float* p`、`&p`、`*p` 分别是什么；
- 为什么申请 n 个 `float` 要传 $n \times \text{sizeof}(\text{float})$ ；
- 二维和三维下标怎样线性化；
- 为什么性能实验必须同时做正确性验证和环境记录。

1.6 由浅入深精讲：地址、指针与二维数据

📖 从浅到深

第一层：先会用：能看懂 `float* p`，会申请和释放一维数组。 **第二层：再理解**：能画出变量、地址和指针之间的关系。 **第三层：最后优化**：能根据数据类型、维度和布局准确计算字节数与地址偏移。

1.6.1 变量名、值与地址是三个不同概念

假设程序中有：

Listing 1.5 值、地址和指针的关系

```
1 float x = 3.5f;
2 float* p = &x;
3 *p = 8.0f;
```

第一行创建一个 `float` 变量，变量名是 `x`，当前值是 3.5。第二行的 `&x` 取得 `x` 的地址，并把地址存进指针变量 `p`。第三行的 `*p` 表示“访问 `p` 所指向的对象”，所以它把 `x` 改成 8.0。可以把它画成：

`p: 0x1000` → `x: 8.0, 地址 0x1000`.

指针变量本身也占用内存，并且也有自己的地址。`p` 保存的是 `x` 的地址，而 `&p` 是指针变量 `p` 自身的地址。这正是 `cudaMalloc` 为什么接收“指针的地址”的基础。

✂ 完整例题

已知：

```
1 float* d_x = nullptr;
2 cudaMalloc((void**)&d_x, 1024 * sizeof(float));
```

调用前，`d_x` 没有指向有效设备内存；`&d_x` 告诉运行时“把新分配内存的首地址写到这个指针变量里”。因此：

- 第二个参数回答“申请多少字节”；
- 第一个参数回答“把申请结果写到哪个指针变量”。

若只传 `d_x`，运行时拿到的是空地址值，却不知道应修改哪个主机变量。

1.6.2 数组名为什么能当指针使用

对数组 `float a[4]`，表达式 `a` 在大多数场景中会转换为首元素地址 `&a[0]`。因此

$$a[i] = *(a + i).$$

这里的“+*i*”不是增加 *i* 个字节，而是跨过 *i* 个元素。若 `float` 占 4 B，则 `a+3` 的数值地址比 `a` 大 12 B。编译器根据指针类型自动完成缩放。

⚠ 易错点

`sizeof(a)` 与 `sizeof(p)` 不同。若 `a` 是本地数组，`sizeof(a)` 是整个数组字节数；若 `p` 是指针，`sizeof(p)` 只是一个地址所占字节数。数组传入函数后通常退化为指针，函数无法仅凭参数 `float* a` 得知数组长度，所以必须额外传 `n`。

1.6.3 二维数组线性化：从具体数字开始

设矩阵有 $H = 3$ 行、 $W = 4$ 列，按行主序排列：

$$\begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \end{bmatrix} \longrightarrow [a_{0,0}, a_{0,1}, a_{0,2}, a_{0,3}, a_{1,0}, \dots].$$

要访问 $a_{2,1}$ ，先跳过前面两整行，共 $2 \times 4 = 8$ 个元素，再在当前行走 1 个元素，所以一维下标是

$$2 \times 4 + 1 = 9.$$

若基地址为 B ，元素类型大小为 s 字节，则字节地址为

$$\text{addr}(r, c) = B + (rW + c)s.$$

这条公式同时决定正确性和访存性能。相邻线程若让列索引 c 连续变化，它们通常访问连续地址；若让行索引 r 连续变化，则地址间隔为 W 个元素，可能形成跨步访问。

1.6.4 三维张量：先平面、再行、再列

张量形状为 $D \times H \times W$ ，下标为 (z, y, x) 。在 z 之前有 z 个完整平面，每个平面有 HW 个元素；在第 z 个平面内， y 之前有 y 行，每行 W 个元素；最后再走 x 个元素。因此

$$\text{offset}(z, y, x) = zHW + yW + x.$$

例如 $D = 300, H = 500, W = 400$ ，访问 $(5, 20, 10)$ ：

$$5 \times 500 \times 400 + 20 \times 400 + 10 = 1,008,010.$$

若元素是 `float`，相对基地址的字节偏移为 $1,008,010 \times 4 \text{ B}$ 。

1.7 编译、运行与错误定位的最小闭环

CUDA 初学者常把“编译成功”误当成“程序正确”。实际上至少有四层：

1. **编译期**：语法、类型和可见性错误；
2. **kernel 启动期**：非法配置、函数属性不满足等；
3. **设备执行期**：越界、非法地址、同步错误；
4. **数值层**：程序不崩溃，但结果错误或误差超限。

设备执行通常是异步的，所以错误不一定在 kernel 启动语句处立即报告。调试阶段应在关键 kernel 后加入：

Listing 1.6 调试阶段的两步错误检查

```
1 kernel<<<grid, block>>>(...);
2 CUDA_CHECK(cudaGetLastError());           // 检查启动错误
3 CUDA_CHECK(cudaDeviceSynchronize());      // 等待执行并暴露运行错误
```

性能测试时不能在每个 kernel 后无条件同步，否则会破坏流水和重叠；调试正确后，再只保留必要同步。

分层实验

实验 A：指针与字节数。分别为 7、1000、 2^{20} 个 `float` 计算申请字节数，并打印 `sizeof(float)`、`sizeof(float*)`。

实验 B：线性化。用双重循环填写一个 5×7 矩阵，使元素值等于 $100r + c$ ，再用一

维指针验证 `a[r*7+c]`。

实验 C：错误定位。故意把申请大小写成 `n` 而不是 `n*sizeof(float)`，观察小输入与大输入的不同现象，并用同步定位错误。

📌 本章加深后的掌握标准

能在纸上画出 `p`、`&p`、`*p` 的关系；能解释 `cudaMalloc` 为什么接收二级指针形式；能从任意二维或三维下标推导一维偏移；能区分编译错误、启动错误、设备运行错误和数值错误。

第 2 章 为什么需要 GPU：从顺序执行到吞吐导向

📖 **原书对应：**原书第 1 章，重点对应 1.1–1.6：异构并行、加速比、并行编程挑战与总体目标。

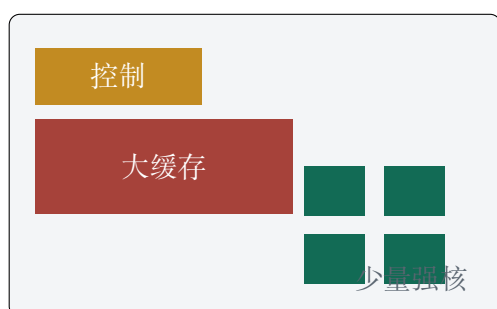
🎯 本章学习目标

- 区分 CPU 的低延迟设计和 GPU 的高吞吐设计；
- 用 Amdahl 定律判断“值得加速的部分”；
- 区分数据并行与任务并行；
- 识别并行化的四类主要障碍：额外工作、内存、负载不均和同步。

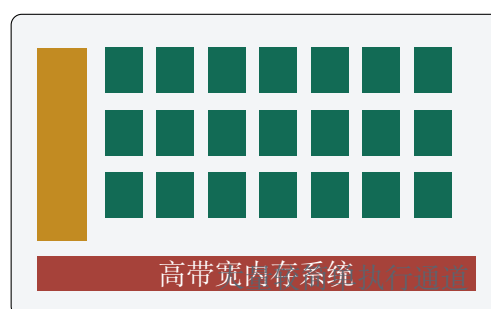
2.1 CPU 和 GPU 优化的是不同目标

CPU 通常把大量晶体管 and 功耗用于减少单个线程的延迟：复杂控制、分支预测、乱序执行和较大的缓存。GPU 则把更多资源用于大量算术单元和内存通道，允许单个操作延迟较长，但通过同时维护大量线程来提高总吞吐。

CPU：延迟导向



GPU：吞吐导向



这并不意味着“GPU 永远比 CPU 快”。当任务只有很少并行工作、控制流复杂、数据规模太小或频繁在主机与设备之间搬运时，CPU 可能更合适。异构计算的核心是：

CPU 负责控制与顺序部分，GPU 负责规则且计算密集的并行部分。

2.2 加速比：先明确比较对象

若基准系统用时 T_{base} ，优化系统用时 T_{new} ，则加速比为

$$S = \frac{T_{\text{base}}}{T_{\text{new}}}.$$

“GPU 加速 20 倍”必须说明：与单核 CPU、优化多核 CPU，还是旧 GPU 比？是否包含数据传输？输入和精度是否相同？没有这些条件，加速比没有可比性。

2.3 Amdahl 定律：整体速度受不可加速部分限制

设原程序中可并行部分占比为 p ，该部分被加速 s 倍。归一化原执行时间为 1：

推导

不可并行部分仍需 $1 - p$ ，可并行部分由 p 降为 p/s ，因此新时间为

$$T_{\text{new}} = (1 - p) + \frac{p}{s}.$$

整体加速比为

$$S_{\text{total}} = \frac{1}{(1 - p) + p/s}.$$

当 $s \rightarrow \infty$ 时，极限为

$$S_{\text{max}} = \frac{1}{1 - p}.$$

例如 $p = 0.30$ ，即使并行部分无限快，整体也最多约 $1.43\times$ ；若 $p = 0.99$ 且并行部分加速 $100\times$ ，整体约为

$$\frac{1}{0.01 + 0.99/100} \approx 50.25 \times .$$

💡 直觉

不要一开始就优化最“显眼”的函数，要优化占总时间最大的函数。性能工程的第一条规则是：**先测量，再决定**。

2.4 数据并行与任务并行

2.4.1 数据并行

同一种操作施加到大量不同数据元素，且各元素之间大体独立。例如：

- 每个像素独立从 RGB 转灰度；

- 每个向量元素执行 $C_i = A_i + B_i$;
- 每个输出矩阵元素计算一个点积;
- 每个网格点根据邻域更新。

GPU 最擅长的是可扩展的数据并行：数据越大，可创建的独立工作越多。

2.4.2 任务并行

不同任务同时执行，例如一边计算、一边传输数据，或同时执行预处理和后处理。任务并行可以与数据并行叠加，但通常数量比数据元素少，粒度也更大。

2.5 从顺序循环中找并行性

考虑：

Listing 2.1 典型顺序循环

```
1 for (int i = 0; i < n; ++i) {  
2     c[i] = a[i] + b[i];  
3 }
```

判断迭代是否可并行，依次问：

1. 第 i 次迭代读写哪些位置？
2. 不同 i 是否写同一个位置？
3. 第 i 次是否依赖第 $i - 1$ 次的结果？
4. 迭代之间是否必须保持顺序？

本例每次只写 $c[i]$ ，没有跨迭代依赖，因此可以令一个线程负责一个 i 。

反例：

Listing 2.2 具有循环携带依赖的前缀和

```
1 for (int i = 1; i < n; ++i) {  
2     x[i] += x[i - 1];  
3 }
```

每次迭代依赖前一次结果，不能直接把所有迭代同时执行。第 11 章将展示如何通过扫描模式重构这种计算。

2.6 并行编程真正困难在哪里

1. 工作效率

并行算法可能比顺序算法做更多总工作。线程多并不自动抵消算法复杂度恶化。

2. 内存瓶颈

大量线程可能同时等待 DRAM。若计算很少、搬运很多，增加算术单元无济于事。

3. 负载不均

数据分布不规则时，部分线程很快结束，部分线程工作很重，导致硬件空闲。

4. 同步与竞争

多线程更新共享数据需要 barrier 或 atomic，等待和串行化会吞噬并行收益。

2.7 并行化决策表

观察	更适合 GPU 的信号	风险信号
工作数量	数十万以上相似任务	只有少量任务
依赖	输出大多独立	强顺序依赖
控制流	线程路径相似	每个元素走完全不同路径
数据复用	可在块内共享	每次访问随机且只用一次
传输	多次 kernel 复用驻留数据	每次只算一点就传回 CPU
同步	块内局部协作	全局频繁同步/竞争

动手练习

对下面任务判断主要并行性，并说明潜在瓶颈：图像灰度化、对一亿个数排序、解析一个短 JSON、训练神经网络、图 BFS、计算一个 32 元素向量和。不要只回答“能/不能用 GPU”，要说明任务数量、依赖、内存和同步。

掌握检查

你应能解释：

- 为什么 GPU 接受较长单线程延迟仍能获得高吞吐；
- 为什么并行部分加速 100 倍不等于程序加速 100 倍；
- 数据并行与任务并行的差别；
- 一个顺序循环能否直接线程化，取决于哪些依赖。

2.8 由浅入深精讲：怎样判断一段循环能否并行

从浅到深

第一层：先会用：看不同迭代是否互不依赖。 **第二层：再理解：**列出每次迭代的读集合与写集合。 **第三层：最后优化：**在保证工作效率的前提下重构依赖，而不是盲目创建线程。

2.8.1 用读写集合分析依赖

对第 i 次迭代，记读取位置集合为 R_i ，写入位置集合为 W_i 。两次迭代 $i \neq j$ 若满足

$$W_i \cap W_j = \emptyset, \quad W_i \cap R_j = \emptyset, \quad R_i \cap W_j = \emptyset,$$

则它们没有写写冲突和跨迭代真依赖，通常可以同时执行。这个形式化方法比“看起来每次都差不多”更可靠。

完整例题

例 1：向量加法。

$$R_i = \{A_i, B_i\}, \quad W_i = \{C_i\}.$$

不同 i 的写集合互不相交，且没有读取其他迭代写出的值，可以直接一线程一元素。

例 2：顺序前缀和。

$$x_i \leftarrow x_i + x_{i-1}.$$

第 i 次读取 x_{i-1} ，而第 $i-1$ 次会写 x_{i-1} ，存在跨迭代依赖，不能直接并发。后续扫描算法通过改变计算树来保留数学结果。

例 3：直方图。每个输入元素决定一个 bin，多次迭代可能写同一个 bins[b]，存在写写冲突。算法仍可并行，但需要原子操作或私有化。

2.8.2 “可以并行”不等于“值得放到 GPU”

设有 n 个独立任务，每个任务只做一次加法。即使完全并行，GPU 方案还要承担 kernel 启动、数据传输和同步开销。可以建立粗略时间模型：

$$T_{GPU} = T_{H \rightarrow D} + T_{launch} + T_{kernel} + T_{D \rightarrow H}, \quad T_{CPU} = T_{cpu_loop}.$$

当 n 很小时，固定开销占主导；当数据已经驻留 GPU、后续还会执行多个 kernel 时，传输成本可以被摊薄。因此判断是否使用 GPU，要看整个应用的数据生命周期，而不只是某个循环。

2.8.3 Amdahl 定律的逐步例题

一个程序原来耗时 100 s，其中数据预处理 12 s、可由 GPU 加速的核心计算 80 s、结果写盘 8 s。若核心计算加速 40 倍：

$$T_{new} = 12 + 80/40 + 8 = 22 \text{ s},$$

$$S = 100/22 \approx 4.55.$$

虽然核心部分加速 40 倍，整体只有 4.55 倍。若再把预处理从 12 s 优化到 3 s，整体变为 13 s，加速约 7.69 倍。这说明性能优化会迁移瓶颈：核心变快后，原本不起眼的顺序部分会变得重要。

❓ 为什么

为什么书中反复强调“把绝大部分工作放到并行部分”？因为并行硬件只能缩短 p/s ，无法消除 $1 - p$ 。当目标是 $20\times$ 整体加速时，即使并行部分无限快，也必须满足 $1/(1 - p) \geq 20$ ，即 $p \geq 0.95$ 。

2.9 从问题描述到线程工作的五步法

面对一个新问题，先不要写 CUDA 语法，按下面顺序写在纸上：

1. **定义输出**。一共有多少个输出元素？每个输出的数学定义是什么？
2. **找独立单位**。不同输出是否可以独立计算？如果不能，依赖发生在何处？
3. **选择粒度**。一个线程算一个输出、多个输出，还是一个 warp/block 合作算一个输出？
4. **列出数据访问**。每个线程读几次、写几次，邻近线程是否访问邻近地址？
5. **列出协作**。是否需要 barrier、atomic、warp shuffle 或跨 kernel 的阶段边界？

✂ 完整例题

以矩阵乘 $C_{ij} = \sum_k A_{ik} B_{kj}$ 为例：

1. 输出有 $M \times N$ 个元素；
2. 不同 C_{ij} 可独立计算；
3. 初始方案是一线程一输出；
4. 每线程读取 K 个 A 和 K 个 B ，写一个 C ；邻近输出会重复读取相同数据；
5. 基础方案不需线程协作，高性能方案让 block 共享 tile，需要 barrier。

这五步自然引出后续的二维线程映射、共享内存分块和数据复用，而不是先背诵“矩阵乘要用 shared memory”。

2.10 工作效率：线程数量不是唯一指标

若顺序算法做 $O(n)$ 次工作，而并行算法为了并行化做 $O(n \log n)$ 次工作，那么当 n 足够大时，额外工作可能抵消并行收益。评估并行算法至少要同时看：

- **跨度 (span)**：关键依赖链有多长，理想并行时间下界；
- **总工作量 (work)**：所有线程操作数总和；
- **通信与同步**：数据搬运、barrier、atomic 的代价；
- **负载均衡**：最慢线程或最慢 block 是否决定总时间。

书中的 reduction、scan、histogram、graph 等章节，本质上都是在这四项之间做权衡。

分层实验

选择你熟悉的三个循环，分别写出 R_i 和 W_i 。至少包含：一个可直接并行的循环、一个有前缀依赖的循环、一个存在共享写冲突的循环。然后为每个循环提出最自然的线程粒度，并说明固定开销是否可能超过收益。

第 3 章 第一个完整 CUDA 程序：向量加法

📖 **原书对应：**原书第 2 章：数据并行、CUDA C++ 程序结构、设备全局内存、kernel、线程索引、kernel 调用与编译。

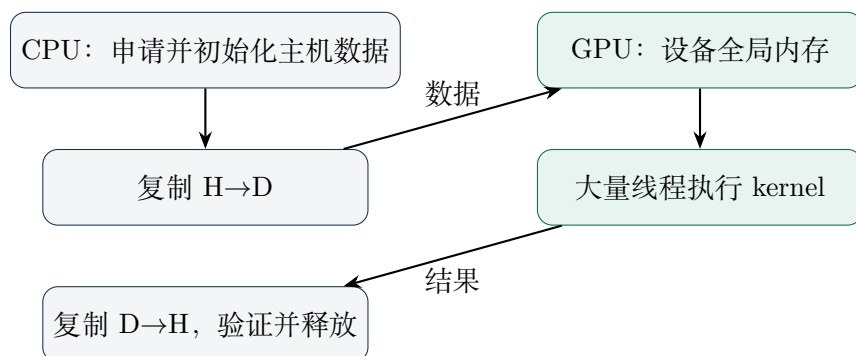
🎯 本章学习目标

- 理解 host、device、kernel、grid、block、thread 的基本关系；
- 掌握 `cudaMalloc`、`cudaMemcpy`、`cudaFree`；
- 写出全局线程索引和上取整分块公式；
- 建立 CUDA 错误检查、同步和结果验证的习惯。

3.1 一段 CUDA 程序包含两个执行世界

CUDA 源文件可以同时包含：

- **主机代码 (host code)**：由 CPU 执行，负责控制、内存管理、数据传输和启动 kernel；
- **设备代码 (device code)**：由 GPU 线程执行，核心入口称为 kernel。



3.2 函数限定符

限定符	调用位置	执行位置	是否启动新线程网格
无或 <code>__host__</code>	host	CPU	否
<code>__global__</code>	通常 host	GPU	是
<code>__device__</code>	device/kernel	GPU	否
<code>__host__</code> <code>__device__</code>	两侧各自调用	CPU/GPU 各编译一 版	否

kernel 的“单程序多数据 (SPMD)”含义是：所有线程执行同一份源代码，但内置索引不同，因此处理不同数据。

3.3 完整的向量加法程序

下面的代码包含资源管理、错误检查、kernel 启动、同步和验证。它比最短示例更长，但更接近真正可调试的程序。

Listing 3.1 可直接编译的向量加法程序

```

1  #include <cuda_runtime.h>
2  #include <cmath>
3  #include <cstdio>
4  #include <cstdlib>
5  #include <vector>
6
7  #define CUDA_CHECK(call) do {                                \
8      cudaError_t err__ = (call);                              \
9      if (err__ != cudaSuccess) {                                \
10         std::fprintf(stderr, "CUDA error %s:%d: %s\n",         \
11             __FILE__, __LINE__, cudaGetErrorString(err__)); \
12         std::exit(EXIT_FAILURE);                                \
13     }                                                            \
14 } while (0)
15
16 __global__ void vectorAddKernel(const float* a,
17                                 const float* b,
18                                 float* c,
19                                 int n) {
20     int i = blockIdx.x * blockDim.x + threadIdx.x;
21     if (i < n) {
22         c[i] = a[i] + b[i];
23     }

```

```

24 }
25
26 int main() {
27     const int n = 1 << 20;
28     const size_t bytes = static_cast<size_t>(n) * sizeof(float);
29
30     std::vector<float> a(n), b(n), c(n);
31     for (int i = 0; i < n; ++i) {
32         a[i] = 0.25f * i;
33         b[i] = 1.0f - 0.5f * i;
34     }
35
36     float *a_d = nullptr, *b_d = nullptr, *c_d = nullptr;
37     CUDA_CHECK(cudaMalloc(&a_d, bytes));
38     CUDA_CHECK(cudaMalloc(&b_d, bytes));
39     CUDA_CHECK(cudaMalloc(&c_d, bytes));
40
41     CUDA_CHECK(cudaMemcpy(a_d, a.data(), bytes,
42                           cudaMemcpyHostToDevice));
43     CUDA_CHECK(cudaMemcpy(b_d, b.data(), bytes,
44                           cudaMemcpyHostToDevice));
45
46     const int threads = 256;
47     const int blocks = (n + threads - 1) / threads;
48     vectorAddKernel<<<blocks, threads>>>(a_d, b_d, c_d, n);
49     CUDA_CHECK(cudaGetLastError());
50     CUDA_CHECK(cudaDeviceSynchronize());
51
52     CUDA_CHECK(cudaMemcpy(c.data(), c_d, bytes,
53                           cudaMemcpyDeviceToHost));
54
55     int bad = 0;
56     for (int i = 0; i < n; ++i) {
57         float ref = a[i] + b[i];
58         if (std::fabs(c[i] - ref) > 1e-5f) {
59             bad = i + 1;
60             break;
61         }
62     }
63     std::printf("verification: %s\n", bad ? "FAILED" : "PASSED");
64
65     CUDA_CHECK(cudaFree(c_d));
66     CUDA_CHECK(cudaFree(b_d));

```

```

67     CUDA_CHECK(cudaFree(a_d));
68     return bad ? EXIT_FAILURE : EXIT_SUCCESS;
69 }

```

3.4 逐行理解 kernel

3.4.1 线程的全局一维索引

一个 grid 由多个 block 构成，每个 block 有多个 thread。对于一维配置：

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}.$$

若每个 block 有 256 个线程：

- block 0 的 i 为 0–255；
- block 1 的 i 为 256–511；
- block b 的起点为 $256b$ 。

这相当于把原来循环的第 i 次迭代交给全局线程 i 。

3.4.2 为什么必须边界检查

数据长度通常不是 block 大小的整数倍。线程块数使用上取整：

$$B = \left\lceil \frac{n}{T} \right\rceil = \frac{n + T - 1}{T} \quad (\text{整数除法}).$$

实际启动 $BT \geq n$ 个线程，多出的线程若不执行 `if (i < n)`，会越界访问。

推导

以 $n = 1000, T = 256$ 为例： $B = (1000 + 255) / 256 = 4$ ，共 1024 个线程。前 1000 个线程处理数据，最后 24 个线程只做边界判断。

3.5 设备内存 API 的参数方向

3.5.1 cudaMalloc

```

1 cudaMalloc(&a_d, bytes);

```

第一个参数是“设备指针变量的地址”，因为 API 要把新申请到的设备地址写回 `a_d`。第二个参数是字节数。

3.5.2 cudaMemcpy

函数顺序始终是：

```
cudaMemcpy(destination, source, bytes, direction).
```

方向常量是额外的自检信息。若目标是 `a_d`、源是 `a.data()`，方向应为 `H→D`。

3.5.3 cudaFree

释放时传设备地址的值：

```
1 cudaFree(a_d);
```

释放后最好不再使用该指针；复杂程序中可将它设为 `nullptr`，减少误用。

3.6 kernel 启动语法

```
1 vectorAddKernel<<<blocks, threads>>>(a_d, b_d, c_d, n);
```

尖括号内是执行配置，圆括号内是普通函数参数。最常见的前两个执行配置参数是 `grid` 大小和 `block` 大小。`kernel` 启动通常相对 `host` 是异步的：调用返回不代表 `GPU` 已完成。

⚠ 易错点

只调用 `cudaGetLastError()` 主要能发现启动配置等立即错误；运行时越界等错误往往要在同步点才能报告。因此调试阶段应在关键 `kernel` 后调用 `cudaDeviceSynchronize()` 并检查返回值。性能版本再根据依赖关系减少不必要的全局同步。

3.7 编译过程的概念模型

`nvcc` 识别 `CUDA` 扩展，将 `host` 代码交给主机 `C++` 编译器，将 `device` 代码编译为 `GPU` 代码或中间表示，再链接 `CUDA runtime`。你只需要先记住：

- `CUDA` 源文件通常以 `.cu` 结尾；
- 普通 `C++` 编译器不理解 `<<< >>>` 和 `__global__`；
- 编译成功不保证 `kernel` 运行正确，必须检查运行时错误。

3.8 性能上为什么这个版本还不理想

它每次调用都申请显存、复制输入、启动 kernel、复制输出、释放显存。若向量很小，固定开销可能超过计算时间。实际应用应尽量让重要数据长期驻留设备端，并串联多个 kernel，减少主机-设备往返。

动手练习

完成四组实验：

1. 把 `threads` 依次设为 32、64、128、256、512，验证结果；
2. 把 n 设为 1000，打印最后 30 个线程的全局索引与是否工作；
3. 故意把复制方向写反，观察错误信息；
4. 删除边界判断并用非整数倍长度运行，使用内存检查工具定位问题。

掌握检查

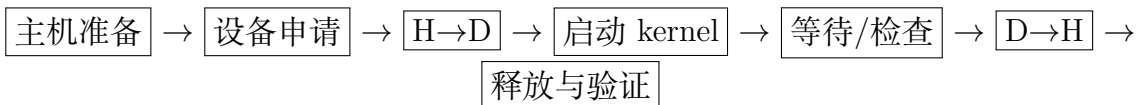
不看代码写出：设备内存申请、两次 $H \rightarrow D$ 复制、kernel 启动、 $D \rightarrow H$ 复制、释放。还应能解释 `&a_d` 为什么有两个星号语义，以及上取整公式为什么正确。

3.9 由浅入深精讲：把完整程序拆成七个阶段

从浅到深

第一层：先会用：照着模板写出能运行的向量加法。 **第二层：再理解：**能说明每个 API 在 CPU 还是 GPU 上发生。 **第三层：最后优化：**能区分传输、启动和 kernel 时间，并解释异步执行。

一个完整 CUDA 程序可按七个阶段阅读：



下面给出一份可以独立编译的版本。它比教学中的最短代码多了错误检查和 CPU 对照，但仍保持结构清晰。

Listing 3.2 可直接编译的完整向量加法程序

```

1 #include <cuda_runtime.h>
2 #include <cmath>
3 #include <stdio>
4 #include <stdlib>

```

```

5  #include <vector>
6
7  #define CUDA_CHECK(call) do {                                     \
8      cudaError_t err__ = (call);                                   \
9      if (err__ != cudaSuccess) {                                   \
10         std::fprintf(stderr, "CUDA error %s:%d: %s\n",            \
11             __FILE__, __LINE__, cudaGetErrorString(err__));\
12         std::exit(EXIT_FAILURE);                                   \
13     }                                                             \
14 } while (0)
15
16 __global__ void vector_add(const float* a, const float* b,
17     float* c, int n) {
18     int i = blockIdx.x * blockDim.x + threadIdx.x;
19     if (i < n) {
20         c[i] = a[i] + b[i];
21     }
22 }
23
24 int main() {
25     const int n = 1 << 20;
26     const size_t bytes = static_cast<size_t>(n) * sizeof(float);
27     std::vector<float> h_a(n), h_b(n), h_c(n);
28     for (int i = 0; i < n; ++i) {
29         h_a[i] = 0.25f * i;
30         h_b[i] = 1.0f - 0.5f * i;
31     }
32
33     float *d_a = nullptr, *d_b = nullptr, *d_c = nullptr;
34     CUDA_CHECK(cudaMalloc((void**)&d_a, bytes));
35     CUDA_CHECK(cudaMalloc((void**)&d_b, bytes));
36     CUDA_CHECK(cudaMalloc((void**)&d_c, bytes));
37     CUDA_CHECK(cudaMemcpy(d_a, h_a.data(), bytes,
38         cudaMemcpyHostToDevice));
39     CUDA_CHECK(cudaMemcpy(d_b, h_b.data(), bytes,
40         cudaMemcpyHostToDevice));
41
42     const int block = 256;
43     const int grid = (n + block - 1) / block;
44     vector_add<<<grid, block>>>>(d_a, d_b, d_c, n);
45     CUDA_CHECK(cudaGetLastError());
46     CUDA_CHECK(cudaDeviceSynchronize());
47

```

```

48     CUDA_CHECK(cudaMemcpy(h_c.data(), d_c, bytes,
49                           cudaMemcpyDeviceToHost));
50     float max_error = 0.0f;
51     for (int i = 0; i < n; ++i) {
52         max_error = fmaxf(max_error,
53                           fabsf(h_c[i] - (h_a[i] + h_b[i])));
54     }
55     std::printf("n=%d, grid=%d, block=%d, max_error=%g\n",
56               n, grid, block, max_error);
57
58     CUDA_CHECK(cudaFree(d_a));
59     CUDA_CHECK(cudaFree(d_b));
60     CUDA_CHECK(cudaFree(d_c));
61     return max_error < 1e-6f ? 0 : 1;
62 }

```

3.9.1 线程索引公式不是魔法

假设每个 block 有 $T = 256$ 个线程。block 0 负责全局下标 0 到 255；block 1 应从 256 开始；block b 应从 bT 开始。block 内线程 t 再向后偏移 t ，因此

$$i = bT + t = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}.$$

例如 block 3 中 thread 17 的全局下标是 $3 \times 256 + 17 = 785$ 。一旦能用“block 起点 + block 内偏移”解释公式，就不必死记。

3.9.2 上取整分块的整数推导

要覆盖 n 个元素，每块 T 个线程，需要

$$G = \left\lceil \frac{n}{T} \right\rceil.$$

对非负整数，可用

$$G = \frac{n + T - 1}{T}$$

的整数除法实现。以 $n = 1000, T = 256$ 为例： $(1000 + 255)/256 = 4$ ，共启动 1024 个线程，最后 24 个由 `if(i<n)` 屏蔽。

⚠ 易错点

表达式 `n / block + 1` 在 n 恰好整除 `block` 时会多启动一整块。虽然边界检查可保证正确，但会浪费资源。统一使用 `(n + block - 1) / block`，并注意大规模时采用足够宽的整数类型。

3.9.3 kernel 启动为什么看起来“立即返回”

主机调用

```
1 vector_add<<<grid, block>>>>(...);
```

通常只是把工作提交到默认 stream, CPU 随后可以继续执行。真正的设备错误可能在后续同步或阻塞式拷贝处才被观察到。因此:

- `cudaGetLastError()` 主要检查启动配置等立即错误;
- `cudaDeviceSynchronize()` 等待设备完成并返回执行错误;
- 从设备到主机的同步拷贝通常也会等待相关工作完成。

异步性是以后实现计算与通信重叠的基础, 但也是初学调试的主要难点。

3.10 逐项计算数据量和理论特征

每个元素读取 a_i, b_i 两个 float, 写一个 float, 理想数据流量为 12 B; 计算只有一次浮点加法。总运算量和流量近似为

$$F = n \text{ FLOP}, \quad Q = 12n \text{ B}, \quad I = F/Q = 1/12 \text{ FLOP/B}.$$

算术强度极低, 所以大规模向量加法通常受内存带宽限制。若设备有效带宽是 B_{eff} , kernel 时间的理想下界近似

$$T \gtrsim \frac{12n}{B_{eff}}.$$

这也解释了为什么“减少一条整数索引指令”往往不如改善内存访问重要。

3.11 从一线程一元素到 grid-stride loop

当数据极大, 或者希望固定一个足够大的 grid 让每个线程处理多个元素, 可使用:

Listing 3.3 grid-stride loop

```
1 __global__ void vector_add_stride(const float* a, const float* b,
2                                   float* c, int n) {
3     int i = blockIdx.x * blockDim.x + threadIdx.x;
4     int stride = blockDim.x * gridDim.x;
5     for (; i < n; i += stride) {
6         c[i] = a[i] + b[i];
7     }
8 }
```

同一线程处理 $i, i + \text{stride}, i + 2\text{stride}, \dots$ 。在每一轮中，相邻线程仍访问相邻元素，保持合并访问。这个模式也方便限制 block 数并复用线程，后续直方图、归约和多元素粗化会反复使用。

3.12 把端到端时间拆开测量

只报告 kernel 时间可能误导。初次实验至少记录：

$$T_{total} = T_{alloc} + T_{H \rightarrow D} + T_{kernel} + T_{D \rightarrow H} + T_{free},$$

$$T_{steady} = T_{kernel} \quad (\text{数据已驻留设备时}).$$

若单次向量加法，传输量与 kernel 量同阶，端到端 GPU 方案可能不占优势；若同一数据在 GPU 上连续执行几十个算子，传输被摊薄，kernel 性能才更有意义。

分层实验

层 1：正确性。分别用 $n = 1, 31, 32, 33, 1000$ 验证边界。

层 2：映射。在小规模 kernel 中打印 block、thread 和全局下标，只启动两个小 block。

层 3：性能。比较 block 大小 32、64、128、256、512；分别报告 kernel 时间和端到端时间。

层 4：模型。根据实际时间计算有效带宽 $12n/T$ ，与设备理论带宽比较，说明差距可能来自哪些因素。

第 4 章 多维网格与数据：把线程准确映射到像素和矩阵

📖 原书对应：原书第 3 章：多维 grid/block、线程到多维数据的映射、图像模糊与矩阵乘。

🎯 本章学习目标

- 使用 dim3 配置一维、二维和三维 grid/block；
- 把线程坐标映射为行、列、平面坐标；
- 正确线性化动态多维数组；
- 写出灰度化、模糊和朴素矩阵乘 kernel。

4.1 grid 和 block 最多可表达三维结构

dim3 有 x、y、z 三个字段，未指定的维度默认是 1。

Listing 4.1 二维 block 与二维 grid

```
1 dim3 block(16, 16, 1);
2 dim3 grid((width + block.x - 1) / block.x,
3           (height + block.y - 1) / block.y,
4           1);
5 kernel<<<grid, block>>>(...);
```

每个 block 的线程数是

$$T_{\text{block}} = \text{blockDim.x} \text{blockDim.y} \text{blockDim.z}.$$

必须满足设备对每个 block 的线程数和各维度大小限制。不要默认所有设备都与某个示例完全相同，应查询设备属性。

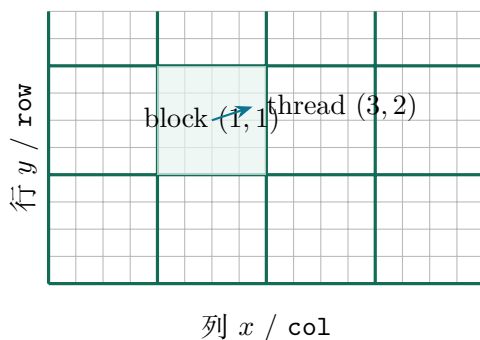
4.2 二维线程坐标

图像或矩阵的行、列映射通常写为

$$\begin{aligned}\text{row} &= \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}, \\ \text{col} &= \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}.\end{aligned}$$

然后进行双边界判断：

```
1 if (row < height && col < width) { ... }
```



4.3 行主序线性化

对宽度为 W 的二维数组：

$$A_{r,c} \longleftrightarrow A[rW + c].$$

对尺寸 $D \times H \times W$ 的三维数组：

$$A_{z,y,x} \longleftrightarrow A[zHW + yW + x].$$

⚠ 易错点

配置 `dim3(widthBlocks, heightBlocks)` 时，代码顺序是 x 再 y ；叙述矩阵尺寸时通常说“高 $\times$ 宽”，即 y 再 x 。混淆这两种顺序是初学者最常见错误之一。

4.4 案例一：RGB 图像转灰度

每个输出像素只依赖同位置的三个颜色通道，因此是近乎理想的数据并行。

$$L = 0.299R + 0.587G + 0.114B.$$

下面假设输入按 `RGBRGB...` 连续存储，输出每像素一个字节。

Listing 4.2 二维灰度化 kernel

```

1  __global__ void rgbToGray(const unsigned char* rgb,
2                          unsigned char* gray,
3                          int width, int height) {
4      int col = blockIdx.x * blockDim.x + threadIdx.x;
5      int row = blockIdx.y * blockDim.y + threadIdx.y;
6      if (row >= height || col >= width) return;
7
8      int pixel = row * width + col;
9      int base = 3 * pixel;
10     float r = rgb[base + 0];
11     float g = rgb[base + 1];
12     float b = rgb[base + 2];
13     gray[pixel] = static_cast<unsigned char>(
14         0.299f * r + 0.587f * g + 0.114f * b);
15 }

```

线程与输出一一对应。输入偏移多乘一个通道数 3。若图像存储格式是 BGR、RGBA 或有行对齐 padding，索引必须相应改变。

4.5 案例二：图像模糊与邻域访问

设模糊半径为 R ，每个输出像素取 $(2R + 1) \times (2R + 1)$ 邻域平均：

$$O_{r,c} = \frac{1}{|\Omega_{r,c}|} \sum_{(u,v) \in \Omega_{r,c}} I_{u,v}.$$

边缘像素的邻域会越界，可采用只统计有效像素、补零、复制边界或镜像等策略。策略不同，数学结果也不同。

Listing 4.3 只统计有效邻居的基本模糊 kernel

```

1  __global__ void blur(const unsigned char* in,
2                      unsigned char* out,
3                      int width, int height, int radius) {
4      int col = blockIdx.x * blockDim.x + threadIdx.x;
5      int row = blockIdx.y * blockDim.y + threadIdx.y;
6      if (row >= height || col >= width) return;
7
8      int sum = 0, count = 0;
9      for (int dy = -radius; dy <= radius; ++dy) {
10         for (int dx = -radius; dx <= radius; ++dx) {
11             int y = row + dy;
12             int x = col + dx;

```

```

13         if (0 <= y && y < height && 0 <= x && x < width) {
14             sum += in[y * width + x];
15             ++count;
16         }
17     }
18 }
19 out[row * width + col] = static_cast<unsigned char>(sum / count);
20 }

```

这个 kernel 正确，但相邻线程反复读取大量重叠像素，造成冗余全局内存访问。第 5、7 章会用共享内存和 halo tile 优化。

4.6 案例三：朴素矩阵乘

设 A 为 $M \times K$ ， B 为 $K \times N$ ， $C = AB$ 。令一个线程计算一个 $C_{r,c}$ ：

$$C_{r,c} = \sum_{k=0}^{K-1} A_{r,k} B_{k,c}.$$

Listing 4.4 支持非方阵的朴素矩阵乘 kernel

```

1  __global__ void matmulNaive(const float* A, const float* B, float* C,
2                               int M, int N, int K) {
3      int col = blockIdx.x * blockDim.x + threadIdx.x;
4      int row = blockIdx.y * blockDim.y + threadIdx.y;
5      if (row >= M || col >= N) return;
6
7      float acc = 0.0f;
8      for (int k = 0; k < K; ++k) {
9          acc += A[row * K + k] * B[k * N + col];
10     }
11     C[row * N + col] = acc;
12 }

```

每个线程做 K 次乘加。不同线程会重复读取同一行的 A 或同一列的 B 。算法正确但数据复用没有显式利用，是后续分块优化的基线。

4.7 选择 block 形状的初步原则

二维问题常见 $16 \times 16 = 256$ 或 $32 \times 8 = 256$ 等配置。不能只看线程总数，还要看：

- warp 线性化时，threadIdx.x 变化最快；

- 连续线程最好访问连续列，以形成合并访存；
- block 形状影响边界浪费、共享内存 tile 形状和 occupancy；
- 不同 kernel 的最佳形状可能不同，必须测量。

4.8 线程到数据映射的通用模板

Listing 4.5 二维 kernel 的标准骨架

```

1  __global__ void kernel2D(/* pointers */, int width, int height) {
2      int x = blockIdx.x * blockDim.x + threadIdx.x;
3      int y = blockIdx.y * blockDim.y + threadIdx.y;
4      if (x >= width || y >= height) return;
5
6      int idx = y * width + x;
7      // read inputs associated with (y, x)
8      // compute
9      // write output[idx]
10 }

```

这个模板只解决“谁负责哪个输出”。复杂 kernel 还要回答输入复用、线程协作、同步和布局问题。

动手练习

1. 对 62×76 图像和 16×16 block，计算 grid 大小、总线程数和多余线程数；
2. 将朴素矩阵乘改为每个线程计算一整行，再分析并行度和访存；
3. 写一个三维 kernel，把体数据中每个元素乘以常数，并正确线性化；
4. 对模糊 kernel 分别实现补零和复制边界，比较四个角的输出。

掌握检查

你应能从数据维度反推出 grid/block 维度，并对任意线程手算其输出坐标和一维地址。看到二维 kernel 时，应第一时间检查 x/y 顺序、边界判断和行主序公式。

4.9 由浅入深精讲：二维线程怎样覆盖二维数据

从浅到深

第一层：先会用：会写 row、col 和边界判断。 **第二层：再理解：**能手算任意线程对应的像素或矩阵元素。 **第三层：最后优化：**能根据行主序布局选择 block 形状和线程映射。

4.9.1 先算一个完整覆盖例子

图像大小为 $H = 62, W = 76$, block 取 16×16 。grid 维度是

$$G_x = \left\lceil \frac{76}{16} \right\rceil = 5, \quad G_y = \left\lceil \frac{62}{16} \right\rceil = 4.$$

共启动 $5 \times 4 \times 16 \times 16 = 5120$ 个线程，而像素只有 $62 \times 76 = 4712$ 个，因此 408 个线程被边界判断屏蔽。block $(x = 3, y = 2)$ 中 thread $(x = 7, y = 5)$ 对应

$$\begin{aligned} col &= 3 \times 16 + 7 = 55, \\ row &= 2 \times 16 + 5 = 37, \\ offset &= 37 \times 76 + 55 = 2867. \end{aligned}$$

请注意 CUDA 构造 `dim3(x,y,z)` 时先写 x ，矩阵叙述通常先说“行 $\times$ 列”，这两个顺序很容易混淆。

4.9.2 通用二维 kernel 骨架

Listing 4.6 二维输出的一般映射模板

```
1 __global__ void kernel2d(const float* in, float* out,
2                          int height, int width) {
3     int col = blockIdx.x * blockDim.x + threadIdx.x;
4     int row = blockIdx.y * blockDim.y + threadIdx.y;
5     if (row >= height || col >= width) return;
6     int idx = row * width + col;
7     out[idx] = f(in, row, col, height, width);
8 }
```

这个模板只适合每个输出可以独立计算的阶段。若 `f` 需要 block 内线程先共同加载数据，则不能让部分线程在 `barrier` 前直接 `return`；应让所有线程参与加载和同步，仅在最终写回时判断输出是否有效。

4.10 灰度转换：从数学式到地址式

每个彩色像素由连续的 R、G、B 三个字节组成，输出灰度像素一个字节。对 (row, col) ：

$$gidx = row \times width + col,$$

$$ridx = 3 \times gidx,$$

$$L = 0.299R + 0.587G + 0.114B.$$

Listing 4.7 二维灰度转换核心代码

```

1  __global__ void rgb_to_gray(const unsigned char* rgb,
2                               unsigned char* gray,
3                               int height, int width) {
4      int col = blockIdx.x * blockDim.x + threadIdx.x;
5      int row = blockIdx.y * blockDim.y + threadIdx.y;
6      if (row < height && col < width) {
7          int g = row * width + col;
8          int p = 3 * g;
9          float y = 0.299f * rgb[p] +
10                  0.587f * rgb[p + 1] +
11                  0.114f * rgb[p + 2];
12          gray[g] = static_cast<unsigned char>(y);
13      }
14 }
```

这里“一线程一输出”非常自然，因为每个输出只依赖一个输入像素。性能主要取决于内存布局、数据类型转换和图像规模，而不需要线程协作。

4.11 图像模糊：一个输出开始读取邻域

设半径为 r ，一个输出像素最多读取 $(2r + 1)^2$ 个输入。最直接的线程代码会对每个输出重复读取邻域。对于内部区域，一个输入像素会被多个相邻输出重复使用。这提示后续的 tiling：让一个 block 先把包含 halo 的输入区域加载到 shared memory，再从片上数据计算多个输出。

边界必须先规定数学语义，常见选择包括：

- 忽略越界点，并用实际有效点数做平均；
- 越界补零；
- clamp 到最近边界；
- 镜像或周期边界。

不同语义产生不同结果，不能把边界处理仅视为“防止崩溃的 if”。

✂ 完整例题

半径 $r = 1$ ，计算左上角 $(0,0)$ 。若采用“只平均有效点”，有效坐标是 $(0,0), (0,1), (1,0), (1,1)$ ，分母为 4；若采用零填充，仍除以 9。两种实现都可能不越界，但输出不同，所以正确性测试必须明确参考语义。

4.12 矩阵乘：一线程一输出的完整矩形版本

设 A 为 $M \times K$ ， B 为 $K \times N$ ， C 为 $M \times N$ ：

Listing 4.8 支持矩形矩阵的朴素矩阵乘

```
1 __global__ void matmul_naive(const float* A, const float* B,
2                             float* C, int M, int K, int N) {
3     int col = blockIdx.x * blockDim.x + threadIdx.x;
4     int row = blockIdx.y * blockDim.y + threadIdx.y;
5     if (row < M && col < N) {
6         float sum = 0.0f;
7         for (int k = 0; k < K; ++k) {
8             sum += A[row * K + k] * B[k * N + col];
9         }
10        C[row * N + col] = sum;
11    }
12 }
```

对固定 warp 中相邻的 col ：读取 $B[kN + col]$ 是连续的，通常容易合并；所有线程读取同一行 A 的相同 k 元素，存在广播/缓存复用机会。但跨 k 循环，每个输出都重复读取大量输入，global memory 流量仍很高。第 6 章会用 shared tile 把相同数据在 block 内复用。

4.12.1 手算一个线程

若 $M = 5, K = 3, N = 4$ ，block 为 2×2 ，block $(1,1)$ 的 thread $(0,1)$ ：

$$col = 1 \times 2 + 0 = 2, \quad row = 1 \times 2 + 1 = 3.$$

它计算

$$C_{3,2} = A_{3,0}B_{0,2} + A_{3,1}B_{1,2} + A_{3,2}B_{2,2},$$

对应地址分别是 $A[9]$ ， $A[10]$ ， $A[11]$ 和 $B[2]$ ， $B[6]$ ， $B[10]$ ，最后写 $C[14]$ 。

4.13 block 形状不是只看线程总数

32×8 和 16×16 都有 256 个线程，但 warp 的二维排列不同。CUDA 先让 x 维连续线性化，所以应优先让 `threadIdx.x` 对应内存中连续的列方向。选择 block 形状时考虑：

- 每块总线程数是否是 warp 大小的整数倍；
- x 维是否让 warp 访问连续地址；
- 邻域算法的 halo 开销；
- shared memory tile 大小；
- 数据长宽比例和边界浪费。

不存在对所有二维算法都最优的固定形状，必须结合访问模式和资源使用测试。

分层实验

实验 1：索引验证。对 7×10 矩阵，用 4×3 block，在输出中写入每个线程的全局线性下标。

实验 2：布局对比。分别让 x 维映射列和映射行，比较矩阵逐元素操作性能。

实验 3：边界语义。实现三种模糊边界策略，使用同一小图逐元素比较。

实验 4：矩阵乘基线。支持非方阵，先与 CPU 三重循环比对，再记录不同 block 形状的时间。

第二部分

理解硬件并系统优化

第 5 章 GPU 执行架构：SM、warp、同步与 occupancy

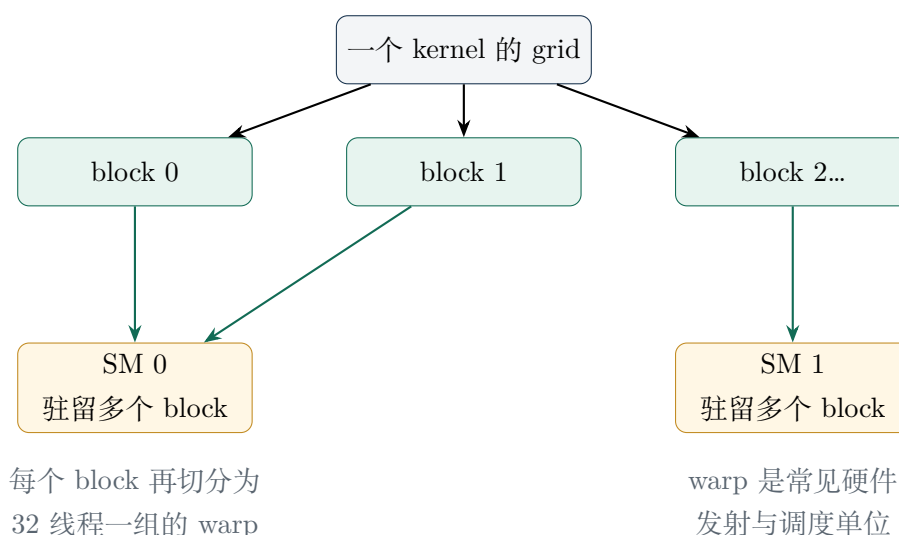
📖 **原书对应：**原书第 4 章：现代 GPU 计算架构、线程块调度、同步、warp/SIMD、控制发散、延迟隐藏、资源划分与 occupancy。

🎯 本章学习目标

- 理解 block 如何被调度到 SM，warp 如何成为硬件调度单位；
- 正确使用 block 级 barrier，并识别死锁风险；
- 解释控制发散、延迟隐藏、wave、tail effect 和 occupancy；
- 从寄存器、共享内存、线程槽位等资源约束推导并发量。

5.1 从软件层次映射到硬件层次

CUDA 软件层次是 $\text{grid} \rightarrow \text{block} \rightarrow \text{thread}$ 。硬件执行的核心层次是 $\text{GPU} \rightarrow \text{SM} \rightarrow \text{warp}$ 执行资源。



关键保证是：同一 block 的所有线程会被放在同一个 SM 上，并在 block 生命周期内占有其所需资源。这使 block 内线程可以使用共享内存和 block 级同步。不同 block 一般不能假定同时执行或按某个顺序执行。

5.2 block 调度带来的透明可扩展性

一个 grid 往往包含远多于 GPU 同时容纳能力的 block。运行时维护待执行 block，SM 有资源时就取新 block。小 GPU 每次运行较少 block，大 GPU 每次运行较多 block，同一 kernel 无需改变正确性逻辑。

wave 与 tail effect

若 GPU 同时最多驻留 R 个 block，而 grid 有 G 个 block，则需要约 G/R 个 wave。最后一个 wave 若只有少量 block，会让部分 SM 空闲，称为**尾部效应 (tail effect)**。

例如 $G = 660, R = 264$ ，需要 2.5 waves。最后 132 个 block 只能占用约一半的 block 并发容量。它是否严重还取决于单 block 时间、负载均衡和整个应用中该 kernel 的占比。

5.3 barrier：只保证范围内的“阶段完成”

最常见的 block 级 barrier 是 `__syncthreads()`。当线程到达它时，必须等待同一 block 的所有线程也到达，随后才能继续。

Listing 5.1 共享内存协作的标准两阶段结构

```
1 __shared__ float tile[256];
2 int t = threadIdx.x;
3 tile[t] = load_input(t);    // phase 1: cooperative load
4 __syncthreads();           // all tile elements are ready
5 use_tile(tile, t);         // phase 2: consume shared data
```

5.3.1 死锁的典型写法

Listing 5.2 错误：同一 block 的线程到达不同 barrier

```
1 if ((threadIdx.x & 1) == 0) {
2     __syncthreads();
3 } else {
4     __syncthreads();
5 }
```

两个源码位置是两个不同 barrier。偶数线程等待奇数线程到第一个 barrier，奇数线程等待偶数线程到第二个 barrier，行为未定义，可能永久等待。

正确原则是：一个 block 内，要么所有线程执行同一个 `__syncthreads()`，要么所有线程都不执行。条件可以存在，但必须对整个 block 一致。

⚠ 易错点

“同一个 warp 的线程通常一起执行”不能替代同步语义。现代架构支持更独立的线程调度；如果算法需要 warp 内线程在某阶段全部完成，应使用合适的 warp 级同步原语，例如 `__syncwarp()`，而不是依赖隐含同步。

5.4 warp 与 SIMT

block 被按线性线程编号切分为 warp。当前主流 CUDA GPU 的 warp 大小是 32，但代码可以通过 `warpSize` 获取，不应把所有推理写死为不可变的硬件公理。

对二维 block，线性线程编号通常可理解为

$$t_{\text{linear}} = \text{extttthreadIdx.y} \times \text{blockDim.x} + \text{threadIdx.x},$$

x 维变化最快。编号 0–31 构成第一个 warp，32–63 构成第二个 warp，以此类推。

SIMT 表示程序员写的是独立标量线程，硬件把同一 warp 的线程放到 SIMD 风格执行资源上。一个 warp 在某时刻通常发射同一条指令，各线程使用自己的寄存器数据。

5.5 控制发散为什么损失吞吐

当同一 warp 的线程走不同分支时，硬件需要分别执行各路径，并屏蔽当前不属于该路径的线程。

Listing 5.3 会发生 warp 发散的分支

```
1 if (threadIdx.x < 24) {
2     pathA();
3 } else {
4     pathB();
5 }
```

第一个 warp 中线程 0–23 执行 A 时，24–31 不活跃；执行 B 时反过来。若两条路径开销相近，warp 有效利用率会明显下降。

5.5.1 不是所有 if 都同样糟

- 若条件对整个 warp 一致，不发散；
- 只在数据边界的最后一个 warp 发散，规模大时影响可能很小；
- 数据相关的随机分支遍布所有 warp，影响通常更大；
- 编译器可能把很短的分支转为谓词执行，实际情况应由 profiler 验证。

5.6 延迟隐藏：GPU 不必让一次内存访问变得很短

全局内存访问可能需要较长时间。SM 同时保留许多驻留 warp 的执行状态；当一个 warp 等待数据时，调度器选择另一个 ready warp 发射指令。由于各 warp 的寄存器状态已驻留硬件，切换不需要像 CPU 操作系统线程那样保存和恢复完整上下文。

💡 直觉

把 SM 想成有多个窗口的服务台。某位顾客（warp）等待材料时，窗口立即服务另一位材料齐全的顾客。GPU 的目标不是让每位顾客的单次等待最短，而是让服务台尽量不空闲。

延迟隐藏需要：

- 足够多的驻留 warp；
- 足够的独立指令或内存请求；
- 不是所有 warp 同时等待同一瓶颈；
- 算法有足够并行工作。

5.7 occupancy：可驻留 warp 比例，不等于性能比例

定义为

$$\text{occupancy} = \frac{\text{每个 SM 实际驻留 warp 数}}{\text{每个 SM 硬件允许的最大 warp 数}}.$$

实际驻留量同时受以下资源限制：

- 每 SM 最大线程/warp 数；
- 每 SM 最大 block 槽位；
- 每个 block 的线程数；
- 每线程寄存器数；
- 每 block 动态或静态共享内存；
- 特定架构的其他约束。

可把每种资源能支持的 block 数分别计算，再取最小值：

$$B_{\text{resident}} = \min(B_{\text{thread}}, B_{\text{register}}, B_{\text{shared}}, B_{\text{slot}}, \dots).$$

例如每 SM 有 R_{SM} 个寄存器，kernel 每线程用 r 个寄存器，每 block 有 T 个线程，则寄存器约束给出

$$B_{\text{register}} \leq \left\lfloor \frac{R_{SM}}{rT} \right\rfloor,$$

实际还要考虑寄存器分配粒度。

⚠ 易错点

更高 occupancy 不保证更快。一个 kernel 可能只需中等 occupancy 就能隐藏延迟；为追求 100% occupancy 而减少寄存器，可能触发寄存器溢出到较慢内存，反而变慢。正确目标是**足够的并发 + 更少的瓶颈**，不是最大化单一指标。

5.8 性能悬崖

资源按离散单位分配。每线程寄存器数只增加一点，可能使每 SM 从 4 个 block 降为 3 个 block，occupancy 发生跳变。这类非线性变化称为性能悬崖。优化时应记录编译器报告的寄存器数和共享内存用量，并观察是否跨过资源阈值。

5.9 设备属性查询

程序可通过设备属性 API 查询 SM 数、最大线程数、共享内存等。教学代码可先用命令行工具查看；可移植程序应在运行时查询并据此选择配置，而不是假定固定硬件。

Listing 5.4 查询部分设备属性的示意代码

```
1 cudaDeviceProp prop{};
2 CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
3 std::printf("name: %s\n", prop.name);
4 std::printf("SM count: %d\n", prop.multiProcessorCount);
5 std::printf("max threads/block: %d\n", prop.maxThreadsPerBlock);
6 std::printf("shared memory/block: %zu\n", prop.sharedMemPerBlock);
```

📖 动手练习

1. 对 block 大小 48、128、256、768，计算每个 block 的 warp 数以及最后一个 warp 是否无效槽位；
2. 写出一个会因条件 barrier 死锁的例子，再改成所有线程都经过同一 barrier；
3. 用编译器资源报告比较“增加几个局部变量”前后的寄存器数；
4. 对同一 kernel 绘制 block 大小、occupancy 和运行时间三者的曲线，说明为什么它们不必单调一致。

📌 掌握检查

看到一个 kernel 时，你应能回答：一个 block 在哪里执行？谁是硬件调度单位？何时需要 barrier？分支是否在 warp 内发散？驻留 warp 数受哪些资源限制？

5.10 由浅入深精讲：从 grid 到 SM 的实际执行过程

📖 从浅到深

第一层：先会用：知道 grid 由 block 组成，block 由 thread 组成。 **第二层：再理解：**知道 block 整体分配到 SM，thread 再按 32 个组成 warp。 **第三层：最后优化：**能定量分析发散、同步和资源限制对驻留 warp 的影响。

5.10.1 一个 grid 不会一次性全部“铺”在 GPU 上

假设 kernel 启动 1000 个 block，而 GPU 在该 kernel 的资源使用下只能同时驻留 120 个 block。运行时先把一批 block 分配到各 SM；某个 block 完成后，空出的资源再接纳后续 block。因此 1000 个 block 会分成若干 **wave**。block 之间没有默认执行顺序：block 17 可能早于 block 2 完成，也不能假设 block 0 一定先运行。

这条规则带来两个结论：

1. kernel 正确性不能依赖不同 block 的先后顺序；
2. block 内同步容易实现，因为同一 block 的线程同时驻留在一个 SM 上，而任意 grid 规模仍可在不同大小 GPU 上分批执行。

🔗 完整例题

一个 grid 有 660 个 block，硬件在当前配置下最多同时执行 264 个 block。前两 wave 各 264 个，最后只剩 132 个。若 block 工作量接近，最后一 wave 只使用约一半可并发槽位，这就是尾部效应。总时间近似为三轮 block 时间，而不是 $660/264 = 2.5$ 轮的连续时间。

5.10.2 block 如何切成 warp

block 中线程先按行主序线性化：

$$t_{linear} = threadIdx.x + blockDim.x(threadIdx.y + blockDim.y threadIdx.z).$$

线性编号 0–31 是 warp 0，32–63 是 warp 1。若 block 有 48 个线程，仍占两个 warp，第二个 warp 只有 16 个有效线程，另外 16 个 lane 不做用户工作。

完整例题

block 为 8×8 。因为 x 维先连续，warp 0 包含 $(y = 0, x = 0 \dots 7)$ 、 $(y = 1, x = 0 \dots 7)$ 、 $(y = 2, x = 0 \dots 7)$ 、 $(y = 3, x = 0 \dots 7)$ ；warp 1 包含后四行。若分支条件只依赖 `threadIdx.y < 4`，两个 warp 各自完全一致，不发散；若条件是 `threadIdx.x < 4`，每个 warp 中一半线程走不同路径，会发散。

5.11 控制发散：损失来自“分路径执行”

考虑同一 warp：

```

1 if (threadIdx.x < 24) {
2     A();
3 } else {
4     B();
5 }
6 C();

```

硬件需要让前 24 个 lane 执行 A，同时屏蔽后 8 个；再让后 8 个执行 B，同时屏蔽前 24 个；最后重新执行 C。若 A、B 成本分别为 t_A, t_B ，该 warp 在分支区的时间更接近 $t_A + t_B$ ，而不是 $\max(t_A, t_B)$ 。

5.11.1 边界分支通常只影响少数 warp

向量长度 1003，block 64，共 16 block、32 warp。只有最后一个 warp 含有越界线程，因此边界判断对大规模数据的整体影响通常很小。不要为了消除一个尾部 warp 的分支而引入复杂且更昂贵的处理。

5.11.2 数据相关分支可能影响所有 warp

若每个元素根据随机标志选择不同路径，几乎所有 warp 都可能发散。可考虑：

- 先按类别重排数据，使同类元素相邻；
- 将两类工作拆为不同 kernel；
- 用无分支表达式替代很短的分支；
- 改变线程粒度，让一个 warp 共同处理同一任务。

这些方法会增加重排、额外 kernel 或计算成本，是否值得必须测量。

5.12 同步：barrier 同时约束控制与内存可见性

`__syncthreads()` 的含义不是“让 GPU 全局暂停”，而是：同一 block 的所有线程到达同一个 barrier 后才能继续，并保证 barrier 前的 shared memory 操作对 block 内线程可见。

Listing 5.5 正确的阶段式协作

```
1 __shared__ float tile[256];
2 int t = threadIdx.x;
3 tile[t] = input[blockIdx.x * blockDim.x + t];
4 __syncthreads();
5 float left = tile[(t + 255) % 256];
6 output[...] = tile[t] + left;
```

若没有 barrier，某线程可能读取 tile 时，相邻线程尚未完成写入。

⚠ 易错点

下面代码可能死锁：

```
1 if (threadIdx.x < n) {
2     __syncthreads();
3 }
```

同一 block 中只有部分线程到达 barrier，其余线程永远不会到达。正确做法通常是让所有线程都经过 barrier，把有效性判断放到加载值或最终写回上。

5.13 延迟隐藏：为什么需要很多驻留 warp

一次 global memory 访问可能等待许多时钟周期。GPU 不一定让这次访问本身变快，而是在 warp A 等待时调度已就绪的 warp B、C、D。要有效隐藏延迟，需要：

- SM 上有足够多的驻留 warp；
- 这些 warp 不是同时等待同一类瓶颈；
- 每个 warp 内有可发出的独立指令或内存请求。

如果所有 warp 都在等待已经饱和的 DRAM 带宽，增加 occupancy 不会增加带宽；如果 kernel 依赖链很长且并行度低，增加 warp 可能显著有帮助。

5.14 occupancy 的定量计算

occupancy 定义为

$$\text{occupancy} = \frac{\text{实际驻留 warp 数}}{\text{硬件允许最大驻留 warp 数}}.$$

实际驻留 block 数受多项资源的最小值限制。设每 SM 上限为：最大 block 数 B_{max} 、最大线程数 T_{max} 、寄存器总数 R_{SM} 、shared memory 总量 S_{SM} 。一个 block 使用 T_b 个线程、每线程 R_t 个寄存器、每 block 使用 S_b shared memory，则可驻留 block 数近似

$$B_{res} = \min\left(B_{max}, \left\lfloor \frac{T_{max}}{T_b} \right\rfloor, \left\lfloor \frac{R_{SM}}{R_t T_b} \right\rfloor, \left\lfloor \frac{S_{SM}}{S_b} \right\rfloor\right),$$

还要考虑寄存器和 shared memory 的分配粒度等硬件细节。

✂ 完整例题

假设每 SM 最多 2048 线程、65536 个寄存器、64 个 warp。block 为 256 线程，每线程使用 40 个寄存器，不考虑其他限制：

$$\left\lfloor \frac{65536}{40 \times 256} \right\rfloor = 6 \text{ blocks}.$$

驻留线程为 $6 \times 256 = 1536$ ，驻留 warp 为 48，occupancy 为 $48/64 = 75\%$ 。若每线程寄存器降到 32，则可由线程上限驻留 8 block，达到 100%。但若降寄存器导致 spilling，实际性能可能反而下降。

❓ 为什么

为什么 occupancy 不是越高越好？因为它只是“可供调度的 warp 数量比例”。当 kernel 已经达到计算吞吐或内存带宽上限，更多 warp 不再提高吞吐；为了追求 occupancy 强行减少寄存器，还可能让中间值溢出到慢速 local memory。

5.15 设备属性与 kernel 资源都要查询

硬件上限可用 `cudaGetDeviceProperties` 查询；kernel 的寄存器、静态 shared memory 等可由编译器报告或 profiler 查看。编译时常用：

```
1 nvcc -O3 -Xptxas=-v kernel.cu -o kernel
```

输出中的 registers、smem、spill load/store 是分析 occupancy 与性能悬崖的重要依据。

分层实验

实验 1：warp 线性化。对 8×8 block，打印每个线程的线性编号和 warp 编号。

实验 2：发散。比较条件 `threadIdx.x < 16` 与 `(threadIdx.x/32) < 1` 的执行差异。

实验 3：资源悬崖。在 kernel 中逐步增加独立局部变量，记录寄存器数、occupancy 和时间，观察性能是否在某个点突变。

实验 4：同步错误。在安全的小程序中构造部分线程进入 barrier 的错误，用同步和检查工具定位；不要在生产代码中保留该版本。

第 6 章 内存体系与数据局部性：性能的核心

📖 **原书对应：**原书第 5 章：内存带宽、CUDA 内存类型、tiling、共享内存矩阵乘、边界检查与内存使用对 occupancy 的影响。

🎯 本章学习目标

- 区分寄存器、local、shared、constant、global memory 的作用；
- 使用算术强度判断计算受限或内存受限；
- 理解 tiling 如何以线程协作换取数据复用；
- 写出正确的共享内存矩阵乘，并处理非整块边界。

6.1 为什么“算得快”常常输给“拿数据慢”

现代 GPU 有很高的计算吞吐，但所有算术操作都需要操作数。若数据不断从片外 DRAM 读取，kernel 的上限可能由内存带宽而不是算术单元决定。

定义算术强度

$$I = \frac{\text{浮点运算次数 (FLOP)}}{\text{从目标内存层次搬运的字节数 (B)}}.$$

给定峰值计算吞吐 $P_{\max}$ 和内存带宽 $B_{\max}$ ，roofline 的简化上界是

$$\mathcal{P} \leq \min(P_{\max}, I B_{\max}).$$

当 $I B_{\max} < P_{\max}$ 时，更像内存受限；反之更可能计算受限。这个模型不是精确预测器，但能判断优化方向。

💡 直觉

若每个数据只做一次加法就丢掉，最需要的是搬运能力；若一个数据加载后参与数百次乘加，算术单元更可能成为瓶颈。优化的本质常常是让一次昂贵加载产生更多有用计算。

6.2 CUDA 中常见内存类型

内存/存储	可见范围	典型生命周期	主要用途与风险
寄存器	单线程私有	线程	最快的局部标量；过多会降低 occupancy
local memory	单线程私有	线程	名称“local”但通常位于片外/缓存层次；常由寄存器溢出或大局部数组产生
shared memory	同一 block	block	低延迟、软件管理；需同步，容量有限，存在 bank conflict
constant memory	全 grid 只读	应用/模块	小型只读数据；warp 广播访问特别合适
global memory	全 grid/host API	显式申请期	容量大、带宽高但延迟长；访问模式决定有效带宽

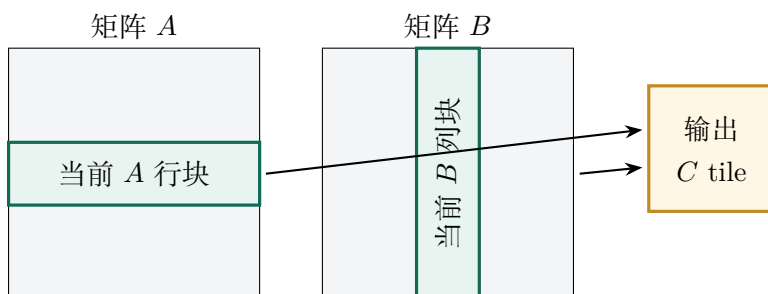
GPU 还有多级 cache。程序员通常不直接控制每条 cache 行，但数据布局、访问顺序和复用距离会影响命中率。

6.3 tiling：把大问题切成能在片上复用的小块

矩阵乘中，每个 $A_{r,k}$ 会被同一输出行的多个线程使用，每个 $B_{k,c}$ 会被同一输出列的多个线程使用。朴素实现让每个线程独立从 global memory 读取，重复流量很大。

设 tile 宽度为 T 。一个 $T \times T$ block 负责一个 C tile。对 K 维分阶段：

1. 每个线程加载一个 A tile 元素和一个 B tile 元素到 shared memory；
2. barrier，确保 tile 完整；
3. 每个线程使用 shared tile 做 T 次乘加；
4. barrier，确保所有线程用完旧 tile；
5. 前进到下一个 K tile。



6.4 共享内存矩阵乘

Listing 6.1 带边界处理的 tiled matrix multiplication

```

1  template<int TILE>
2  __global__ void matmulTiled(const float* A, const float* B, float* C,
3                             int M, int N, int K) {
4      __shared__ float As[TILE][TILE];
5      __shared__ float Bs[TILE][TILE];
6
7      int row = blockIdx.y * TILE + threadIdx.y;
8      int col = blockIdx.x * TILE + threadIdx.x;
9      float acc = 0.0f;
10
11     int phases = (K + TILE - 1) / TILE;
12     for (int p = 0; p < phases; ++p) {
13         int aCol = p * TILE + threadIdx.x;
14         int bRow = p * TILE + threadIdx.y;
15
16         As[threadIdx.y][threadIdx.x] =
17             (row < M && aCol < K) ? A[row * K + aCol] : 0.0f;
18         Bs[threadIdx.y][threadIdx.x] =
19             (bRow < K && col < N) ? B[bRow * N + col] : 0.0f;
20
21         __syncthreads();
22         #pragma unroll
23         for (int k = 0; k < TILE; ++k) {
24             acc += As[threadIdx.y][k] * Bs[k][threadIdx.x];
25         }
26         __syncthreads();
27     }
28
29     if (row < M && col < N) {
30         C[row * N + col] = acc;
31     }
32 }

```

6.4.1 为什么加载阶段不能提前 return

若边界线程在第一个 barrier 前 return，同 block 的有效线程会等待永不到达的线程。因此所有线程都必须参与 shared tile 的加载和 barrier；越界线程加载 0，最后只有有效输出线程写回。

⚠ 易错点

在包含 `__syncthreads()` 的 kernel 中，不能简单把通用的 `if (row>=M || col>=N) return;` 放在开头。先判断 barrier 是否仍能被整个 block 一致执行。

6.5 流量与复用的粗略分析

朴素矩阵乘中，每个输出做 K 次乘加，同时读取约 $2K$ 个 float。一个 $T \times T$ tiled block 每阶段加载 $2T^2$ 个 float，却完成 T^3 次乘加。每个加载元素被约 T 个线程复用，global memory 流量相对减少约 T 倍（忽略缓存与边界）。

如果把一次乘加记作 2 FLOP，则 tile 内计算量约 $2T^3$ FLOP，加载量约 $2T^2 \times 4 = 8T^2$ B，阶段算术强度约

$$I \approx \frac{2T^3}{8T^2} = \frac{T}{4} \text{ FLOP/B.}$$

增大 T 提高复用，但会增加每 block 线程数、共享内存和寄存器压力，存在权衡。

6.6 constant memory 的适用场景

当所有或许多 warp 线程读取同一个小只读值，如卷积核系数，constant memory 的广播和缓存机制可能有效。若每个线程读取完全不同的常量地址，则优势减弱。它容量有限，不适合任意大输入。

6.7 共享内存与 occupancy 的权衡

若每 SM 可用共享内存为 S_{SM} ，每 block 使用 S_B ，则共享内存最多允许

$$B_{\text{shared}} \leq \left\lfloor \frac{S_{SM}}{S_B} \right\rfloor$$

个 block 驻留。tile 增大可能减少 global memory 流量，却降低并发 block 数。应结合寄存器、线程槽位和实际运行时间比较，而不是只看单一理论复用率。

6.8 局部性有两种

时间局部性

同一数据在短时间内被重复使用。shared tile 就是显式保留数据以供多次使用。

空间局部性

相邻线程或相邻时间访问相邻地址，便于 cache 和内存事务高效服务。下一章的合并访存重点处理它。

 动手练习

1. 手算 $M = 5, N = 7, K = 6, T = 4$ 时 grid、phase 数和最后一个 phase 的补零位置；
2. 比较朴素与 tiled kernel 的全局内存读取次数上界；
3. 分别使用 tile 8、16、32，记录共享内存、occupancy 和时间；
4. 故意在边界线程开头 return，解释为何可能挂起，再修复。

 掌握检查

你应能解释每种内存的范围与代价，独立画出 tiled matrix multiplication 的“加载 - 同步 - 计算 - 同步”阶段，并用算术强度说明它为何可能更快。

6.9 由浅入深精讲：先算流量，再决定是否使用 shared memory

 从浅到深

第一层：先会用：知道 global memory 大而慢，shared memory 小而快。 **第二层：再理解：**知道 tiling 是为了让一次 global load 被多个计算复用。 **第三层：最后优化：**能定量比较优化前后的字节流量、算术强度和资源代价。

6.10 内存层次不是简单的“快慢表”

不同内存类型需要从四个维度理解：

位置/类型	典型作用域	生命周期	编程时最重要的问题
寄存器	单线程	thread 执行期间	数量有限；过多会降低 occupancy 或 spill
shared memory	单 block/-cluster	block 执行期间	程序显式管理；需同步；存在 bank conflict
global memory	整个 device	显式申请到释放	容量大、带宽高但延迟长；访问应合并并减少重复
constant memory	device, 全线程只读	模块/程序期间	warp 读取同一地址时广播高效
local memory	逻辑上单线程, 物理常在显存	thread 执行期间	大数组或寄存器溢出可能落入 local memory
缓存	硬件管理	硬件决定	不能把命中当保证；布局 and 复用仍应显式设计

“shared 一定比 global 快”并不足以指导优化。若数据只用一次，把它先复制到 shared 再读一次只会增加指令与同步。shared 的价值来自**跨线程或跨计算步骤复用**、重排访问或减少原子竞争。

6.11 朴素矩阵乘的流量为什么高

$C = AB$ ，假设方阵宽度为 N ，一线程计算一个 C_{ij} 。每个输出做 N 次乘加，读取 N 个 A 和 N 个 B ，写一个 C 。忽略缓存，单输出约：

$$F \approx 2N \text{ FLOP}, \quad Q \approx (2N + 1) \times 4 \text{ B.}$$

算术强度约为

$$I \approx \frac{2N}{8N + 4} \rightarrow 0.25 \text{ FLOP/B.}$$

但相邻输出高度复用输入：同一行 A 被一整行输出使用，同一列 B 被一整列输出使用。朴素 kernel 没有显式组织这种复用。

6.12 shared-memory tiling 的三阶段

设 tile 宽度为 T ，一个 $T \times T$ block 计算 C 的一个 $T \times T$ 输出块。对 K 维每个阶段：

1. 每线程从 global memory 加载一个 A 元素和一个 B 元素到 shared；
2. barrier，确保 tile 完整；

3. 每线程用 shared 中 T 对数据更新自己的累加器；
4. barrier，确保所有线程用完当前 tile，再覆盖 shared 缓冲。

Listing 6.2 带边界处理的 tiled GEMM 核心

```

1 constexpr int T = 16;
2 __global__ void matmul_tiled(const float* A, const float* B,
3                             float* C, int M, int K, int N) {
4     __shared__ float As[T][T];
5     __shared__ float Bs[T][T];
6     int row = blockIdx.y * T + threadIdx.y;
7     int col = blockIdx.x * T + threadIdx.x;
8     float sum = 0.0f;
9
10    for (int base = 0; base < K; base += T) {
11        int a_col = base + threadIdx.x;
12        int b_row = base + threadIdx.y;
13        As[threadIdx.y][threadIdx.x] =
14            (row < M && a_col < K) ? A[row * K + a_col] : 0.0f;
15        Bs[threadIdx.y][threadIdx.x] =
16            (b_row < K && col < N) ? B[b_row * N + col] : 0.0f;
17        __syncthreads();
18
19        #pragma unroll
20        for (int k = 0; k < T; ++k) {
21            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
22        }
23        __syncthreads();
24    }
25    if (row < M && col < N) C[row * N + col] = sum;
26 }

```

6.12.1 为什么无效线程也不能提前退出

边缘 block 中有些线程没有对应有效输出，但它们仍可能负责加载其他有效线程需要的 tile 元素，并且所有线程必须共同经过 barrier。安全做法是对越界加载写入零，让所有线程参与同步，最后只由有效线程写输出。

6.13 定量比较复用

一个 $T \times T$ block 在每个 K 阶段从 global memory 加载 $2T^2$ 个元素，完成 T^2 个输出的 T 次乘加，即约 $2T^3$ FLOP。阶段算术强度粗略为

$$I_{tile} \approx \frac{2T^3}{2T^2 \times 4} = \frac{T}{4} \text{ FLOP/B},$$

未计输出写回。 $T = 16$ 时约 4 FLOP/B， $T = 32$ 时约 8 FLOP/B，远高于朴素约 0.25 FLOP/B。其本质是每个加载到 shared 的元素被 T 个输出复用。

❓ 为什么

为什么不能无限增大 T ？因为 block 线程数为 T^2 ，shared memory 为 $2T^2 \times 4$ B，寄存器与同步代价也增长。 $T = 32$ 已有 1024 线程，达到常见 block 上限；更大的输出 tile 必须通过“每线程计算多个输出”而不是继续增加线程。

6.14 global memory 合并访问的地址视角

同一 warp 的 lane 若访问连续且对齐的数据，硬件可用较少内存事务服务。以 32 个线程读取 float 为例，地址理想序列是

$$B, B + 4, B + 8, \dots, B + 124.$$

若读取矩阵列，地址可能是

$$B, B + 4W, B + 8W, \dots, B + 12W, \dots$$

每个线程落在相隔很远的位置，事务利用率低。shared memory 常用于把“global 连续加载”转换成“shared 中按另一方向读取”，矩阵转置是典型案例。

6.15 shared memory bank conflict 的具体例子

可以把 shared memory 想象为多个可并行服务的 bank。若 32 个 lane 访问 32 个不同 bank，可同时进行；若访问同一 bank 的不同地址，则需分多次。二维数组 `tile[32][32]` 按行主序存储，当 warp 用 `tile[threadIdx.x][fixed]` 读取一列时，每行跨度 32 个元素，可能使所有 lane 映射到同一 bank。改成 `tile[32][33]` 后，每行起点相对 bank 偏移 1，列访问被分散。

⚠ 易错点

添加 padding 不是通用仪式。只有当访问模式确实产生 bank conflict 时才有收益；padding 会增加 shared memory 占用，可能降低驻留 block 数。应结合 profiler 的 bank conflict 指标验证。

6.16 寄存器复用与 local memory spill

累加器放在寄存器中避免循环内反复访问 shared/global memory。高级 GEMM 让每线程维护多个输出累加器，提升数据复用，但每个累加器都占寄存器。寄存器过多可能：

- 限制每 SM 驻留线程数；
- 使编译器把值 spill 到 local memory；
- 增加指令调度压力。

所以寄存器 tiling 的选择必须与 occupancy 和实际时间共同评估。

6.17 Unified Memory: 简化可见性，不自动保证高性能

cudaMallocManaged 返回 CPU 和 GPU 都可使用的统一指针，运行时通过页迁移或映射提供访问。它能降低显式拷贝代码复杂度，但首次触页、迁移、抖动和多处理器并发访问仍会产生代价。理解显式 host/device 模型后再使用 Unified Memory，才能判断何时预取、何时让数据驻留、何时仍应手动管理。

🔧 分层实验

实验 1：流量估算。对 $N = 1024$ 的方阵，估算朴素和 $T = 16$ tiled GEMM 的理想 global load 数量。

实验 2：复用验证。实现朴素与 tiled 版本，比较有效 GFLOP/s 和 global memory 吞吐。

实验 3：tile 扫描。比较 $T = 8, 16, 32$ ，记录 shared memory、寄存器、occupancy 和时间。

实验 4：转置 padding。实现无 padding 与 +1 padding 的 tiled transpose，验证 bank conflict 指标变化。

第 7 章 通用性能优化清单：从访存到双缓冲

📖 **原书对应：**原书第 6 章：global memory 合并访问、隐藏内存延迟、向量化加载、shared memory bank conflict、线程粗化、循环展开、双缓冲与系统优化策略。

🎯 本章学习目标

- 判断一个 warp 的 global memory 访问是否合并；
- 区分 occupancy、内存级并行和指令级并行；
- 识别 bank conflict，并使用 padding 或布局变换；
- 理解线程粗化、循环展开和双缓冲的收益与代价；
- 按测量驱动的顺序优化，而不是同时堆叠技巧。

7.1 先建立优化顺序

推荐的顺序是：

1. 确认算法复杂度与结果正确；
2. 测量 kernel 时间和端到端时间；
3. 判断瓶颈属于计算、内存带宽、延迟、同步还是负载不均；
4. 选择与瓶颈匹配的一项优化；
5. 验证资源用量、正确性和真实性能；
6. 重复，直到接近硬件上限或收益不再值得复杂度。

7.2 global memory 合并访问

一个 warp 的线程若访问连续且对齐的地址，硬件可用较少内存事务服务；若跨步或随机访问，可能需要更多事务，浪费带宽。

7.2.1 连续访问

```
1 int i = blockIdx.x * blockDim.x + threadIdx.x;
2 y[i] = x[i];
```

相邻线程访问相邻 float，通常是理想模式。

7.2.2 跨步访问

```
1 y[i] = x[i * stride];
```

当 stride 很大，warp 的 32 个地址分散到许多内存段，有效带宽下降。

7.2.3 矩阵转置的经典问题

读取按行连续、写入按列跨步，或反之。常用方法是先把 tile 连续读入 shared memory，再以转置后的索引连续写出。shared memory 在这里不仅复用数据，也用于重新排列访问顺序。

7.3 隐藏延迟的三个来源

线程级并行 (TLP)

更多驻留 warp，某些等待时调度其他 warp。

内存级并行 (MLP)

同时发出多个独立内存请求，避免串行等待。

指令级并行 (ILP)

单线程中交错独立运算，让流水线和加载重叠。

高 occupancy 只增强 TLP 的机会，不保证请求独立，也不解决带宽已经饱和的问题。

7.4 向量加载与存储

在对齐和连续条件满足时，可用 float2、float4 等一次表达多个元素：

Listing 7.1 向量化复制的核心写法

```
1 int i4 = blockIdx.x * blockDim.x + threadIdx.x;
2 const float4* src4 = reinterpret_cast<const float4*>(src);
3 float4* dst4 = reinterpret_cast<float4*>(dst);
```

```
4 dst4[i4] = src4[i4];
```

潜在收益包括减少指令数、增加每线程工作和内存请求并行性。风险包括地址对齐、尾部元素处理、寄存器压力和并不一定更好的生成代码。必须检查数据指针对齐和 $n \bmod 4$ 的尾部。

7.5 shared memory bank conflict

shared memory 被划分为多个 bank。warp 中不同线程访问不同 bank 时可并行；多个线程访问同一 bank 的不同地址时通常需要串行化。所有线程读取同一地址可由广播机制高效处理。

矩阵转置常见写法：

```
1 __shared__ float tile[TILE][TILE];
2 // later read tile[threadIdx.x][threadIdx.y]
```

当列访问使多个线程落到同一 bank，可添加一列 padding：

```
1 __shared__ float tile[TILE][TILE + 1];
```

这改变每行起始地址与 bank 的对应关系，常可消除规则冲突。padding 增加少量共享内存，需要检查 occupancy 影响。

7.6 线程粗化：一个线程计算多个输出

基础映射通常是一线程一输出。粗化让每个线程负责 C 个输出：

Listing 7.2 连续粗化的示意

```
1 int base = (blockIdx.x * blockDim.x + threadIdx.x) * C;
2 #pragma unroll
3 for (int j = 0; j < C; ++j) {
4     int i = base + j;
5     if (i < n) out[i] = f(in[i]);
6 }
```

收益可能来自：

- 减少线程和索引计算开销；
- 增加单线程 ILP；
- 在某些模式中复用已加载数据；

- 减少 barrier 或原子操作参与者数量。

代价可能是并行度下降、寄存器增加、负载不均、访存不再合并。连续粗化与交错粗化对访存模式影响不同。

7.7 循环展开

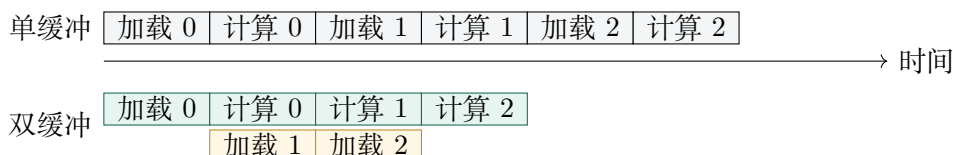
若循环次数小且编译期已知，展开可以减少分支与计数开销，并暴露更多 ILP：

```
1 #pragma unroll
2 for (int k = 0; k < TILE; ++k) {
3     acc += A[k] * B[k];
4 }
```

但过度展开会增大代码体积和寄存器活跃范围。不要把 `#pragma unroll` 当成无条件加速开关。

7.8 双缓冲与软件流水

分块算法每阶段通常是“加载 tile，再计算 tile”。若硬件和 API 支持异步复制或可通过预取实现，可准备两个缓冲：计算当前 tile 时加载下一 tile。



理想情况下阶段时间由“加载 + 计算”变为接近 $\max(\text{加载}, \text{计算})$ 。实际需要正确处理依赖、同步、缓冲切换和尾部阶段。

7.9 优化清单

问题	观察指标/现象	可能措施
全局内存事务浪费	有效带宽低，访问跨步	改布局、交换映射、shared tile、向量加载
数据重复从 DRAM 读取	cache 命中不佳、算术强度低	tiling、寄存器/共享内存复用、算法重构

延迟未隐藏	warp 等待多、吞吐未饱和	增加独立工作、调整 block、粗化、预取
控制发散	分支效率低	数据重排、warp 一致分支、分离 kernel
原子竞争	原子吞吐低	私有化、分层聚合、warp 聚合
barrier 过多	同步等待高	warp 原语、粗化、减少阶段数
共享内存冲突	bank conflict 指标高	padding、索引重排、改变 tile 形状
occupancy 过低	寄存器/共享内存限制	调 block、减少资源；但先确认是否真是瓶颈
尾部效应	最后一 wave 利用率低	增加 block 数、改变分解、合并小任务

7.10 测量策略

1. 先测端到端，再测 kernel；
2. 使用 GPU 事件计时，避免把 host 异步返回当成完成时间；
3. 预热，重复多次，报告中位数和波动；
4. 同时记录吞吐（GB/s、GFLOP/s、元素/s）；
5. 用 roofline 或“速度极限”判断是否还有合理空间；
6. 优化后检查输出误差、资源用量和不同输入规模。

Listing 7.3 CUDA event 计时骨架

```

1 cudaEvent_t start, stop;
2 cudaEventCreate(&start);
3 cudaEventCreate(&stop);
4 cudaEventRecord(start);
5 kernel<<<grid, block>>>(...);
6 cudaEventRecord(stop);
7 cudaEventSynchronize(stop);
8 float ms = 0.0f;
9 cudaEventElapsedTime(&ms, start, stop);

```

⚠ 易错点

一次出现 2% 的“加速”可能只是测量噪声。复杂优化还会增加维护成本。应报告重复次数、误差范围和输入规模，并优先保留稳定、可解释、可移植的收益。

🔧 动手练习

选择前面矩阵乘或转置 kernel，按“基线 - 合并访存 - shared tile - padding - 粗化 - 展开”的顺序建立实验表。每步只改变一个因素，记录时间、吞吐、寄存器、共享内存和 occupancy。

✅ 掌握检查

你应能根据 profiler 现象选择对应优化，而不是凭印象堆技巧；能解释向量加载、padding、粗化、展开和双缓冲各自解决什么问题，又可能引入什么新瓶颈。

7.11 由浅入深精讲：性能优化先做定量诊断

📖 从浅到深

第一层：先会用：正确计时并计算吞吐。**第二层：再理解：**用有效带宽、算术强度和资源指标判断瓶颈。**第三层：最后优化：**建立“假设-修改-测量-解释”的优化闭环。

7.12 第一步：定义你到底在测什么

对 GPU 程序至少有三种时间：

- **kernel latency**：设备执行某个 kernel 的时间；
- **GPU pipeline time**：多个 kernel 和设备间拷贝组成的时间；
- **end-to-end time**：包含主机准备、申请、传输、同步和验证的应用时间。

不同目标使用不同指标。优化在线服务时可能关注单请求延迟；科学计算批处理可能关注总吞吐；库 kernel 比较常关注稳态 kernel 性能。报告中必须写清测量边界。

7.12.1 GPU event 计时的完整注意事项

Listing 7.4 重复测量并报告平均 kernel 时间

```
1 // warm-up
2 for (int i = 0; i < 10; ++i) kernel<<<grid, block>>>(...);
```

```

3  CUDA_CHECK(cudaDeviceSynchronize());
4
5  cudaEvent_t start, stop;
6  CUDA_CHECK(cudaEventCreate(&start));
7  CUDA_CHECK(cudaEventCreate(&stop));
8  CUDA_CHECK(cudaEventRecord(start));
9  for (int i = 0; i < 100; ++i) kernel<<grid, block>>>(...);
10 CUDA_CHECK(cudaEventRecord(stop));
11 CUDA_CHECK(cudaEventSynchronize(stop));
12 float ms = 0.0f;
13 CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
14 ms /= 100.0f;

```

预热用于排除上下文初始化、首次模块加载和缓存冷启动等影响；重复测量减少计时分辨率与系统噪声。若 kernel 写相同内存，确认编译器不会消除工作，并在测量后检查结果。

7.13 有效带宽：内存型 kernel 的第一指标

若一个 kernel 理论上读取 R B、写入 W B，时间为 t 秒，则有效带宽

$$B_{eff} = \frac{R + W}{t}.$$

向量加法 $C = A + B$ 处理 n 个 float，通常按 $12n$ B 计算。若 $n = 2^{26}$ ，数据量约 805 MB，kernel 用时 2 ms，则

$$B_{eff} \approx 402.7 \text{ GB/s}.$$

它不是 DRAM 物理事务的精确计数，而是从算法必要字节量得到的可比较指标。若访问不合并，真实 DRAM 流量可能更大，因此有效带宽会显著低于硬件能力。

7.14 Roofline：把计算上限和带宽上限放在一起

设峰值计算吞吐 P_{max} 、可持续内存带宽 B_{max} 、算术强度 $I = F/Q$ ，则性能上界

$$P \leq \min(P_{max}, IB_{max}).$$

转折点为

$$I^* = \frac{P_{max}}{B_{max}}.$$

若 $I < I^*$ ，带宽屋顶更低，kernel 倾向内存受限；若 $I > I^*$ ，计算屋顶更低，倾向计算受限。

✂ 完整例题

某 GPU 的可持续 FP32 吞吐为 20 TFLOP/s、带宽为 1 TB/s，则转折点 $I^* = 20$ FLOP/B。向量加法 $I \approx 0.083$ ，带宽上界仅约 83 GFLOP/s，远低于 20 TFLOP/s；即使算术单元数量翻倍也几乎无帮助。若 tiled GEMM 达到 40 FLOP/B，其带宽上界为 40 TFLOP/s，高于计算峰值，更可能受计算吞吐限制。

⚠ 易错点

Roofline 是上界模型，不会自动解释所有低性能。低于屋顶还可能因为未合并访问、依赖链、低并行度、发散、bank conflict、同步、指令吞吐或尾部效应。它用于缩小方向，不替代 profiler。

7.15 合并访问的两个对照 kernel

Listing 7.5 连续访问：相邻线程读相邻元素

```
1 int i = blockIdx.x * blockDim.x + threadIdx.x;
2 out[i] = in[i];
```

Listing 7.6 跨步访问：相邻线程间隔 stride

```
1 int i = blockIdx.x * blockDim.x + threadIdx.x;
2 out[i] = in[i * stride];
```

当 stride 增大，同一 warp 的请求散布到更多内存段，事务有效字节比例下降。缓存可能暂时掩盖小数据问题，所以应使用大于缓存容量的数据，并查看 global load efficiency 或实际 DRAM bytes。

7.16 延迟受限、带宽受限和计算受限的区别

类型	常见现象	优化方向
延迟受限	带宽未饱和、eligible warp 少、依赖等待高	增加独立请求/warp、粗化、预取、减少串行依赖
带宽受限	DRAM 吞吐接近上限，算术单元利用率低	减少字节、提高复用、压缩/融合、改善事务利用率
计算受限	计算管线吞吐接近上限，算术强度高	减少 FLOP/指令、使用专用硬件、提高指令并行与数据类型效率
同步/竞争受限	barrier/atomic 等待高，吞吐随热点恶化	私有化、分层聚合、warp 原语、改变分解

7.17 线程粗化的定量权衡

一个线程处理 C 个输出，可能把每个输出的索引计算和循环控制从 C 份降为一部分共享，并增加 ILP；但线程总数约降为原来的 $1/C$ 。若原 grid 只有很少 block，粗化会使并行度不足；若原 grid 很大且每线程寄存器仍可接受，粗化可能降低开销。

连续粗化：

$$i = (\text{blockIdx}.x \text{blockDim}.x + \text{threadIdx}.x)C + j$$

能让单线程访问连续元素，但同一时刻相邻线程的第一个元素间隔 C ；交错粗化：

$$i = \text{blockIdx}.x \text{blockDim}.xC + \text{threadIdx}.x + j \text{blockDim}.x$$

让每一轮相邻线程访问连续地址。具体选择要看循环结构和 cache/合并访问。

7.18 循环展开和寄存器压力

展开减少循环分支，暴露独立乘加，但会让更多中间值同时存活。编译器报告若显示寄存器从 32 增到 70，可能减少驻留 block 或出现 spill。优化不是“展开越多越快”，而是找出指令开销、ILP 和资源占用的平衡点。

7.19 双缓冲的时序模型

单缓冲每阶段时间近似

$$T_{\text{single}} = T_{\text{load}} + T_{\text{compute}}.$$

若加载下一 tile 能与计算当前 tile 重叠，稳态阶段理想时间接近

$$T_{double} \approx \max(T_{load}, T_{compute}).$$

但需要额外缓冲、异步复制或足够的独立指令，并正确处理启动阶段、稳态阶段和收尾阶段。若加载和计算无法真正并行，双缓冲只会增加复杂度和资源使用。

7.20 优化实验记录模板

每次优化至少填写：

项目	内容
基线与输入	代码版本、数据规模、数据分布、精度
优化假设	预计减少哪类流量/同步/指令，理论变化多少
唯一改动	本次只改变的参数或代码结构
正确性	误差阈值、边界规模、随机测试
性能	中位时间、吞吐、波动、端到端与 kernel 边界
资源	registers/thread、shared/block、occupancy、block 数
解释	profiler 是否支持假设；是否出现新瓶颈

分层实验

以矩阵转置为主线，依次实现：直接转置、shared tile、shared tile 加 padding、向量化加载。每一步先预测 global 访问和 bank conflict，再测时间与 profiler 指标。禁止一次同时改 block 形状、布局和向量宽度，否则无法知道收益来源。

第三部分

掌握可迁移的并行计算模式

第 8 章 卷积与 stencil：从邻域计算学习 halo tiling

📖 **原书对应：**原书第 7-8 章：卷积、constant memory、halo cell、缓存辅助 tiling；stencil、shared/register tiling 与线程粗化。

🎯 本章学习目标

- 识别卷积和 stencil 的共同“邻域访问”结构；
- 区分输出 tile、输入 tile 与 halo；
- 使用 constant memory 保存小型只读卷积核；
- 比较 shared-memory halo、cache halo、线程粗化和寄存器 tiling。

8.1 模式本质：每个输出依赖一个局部邻域

一维离散卷积可写为

$$P[i] = \sum_{j=-R}^R N[i+j]M[j+R],$$

其中 N 是输入， M 是长度 $2R+1$ 的 mask。二维卷积为

$$P[y,x] = \sum_{dy=-R}^R \sum_{dx=-R}^R N[y+dy, x+dx]M[dy+R, dx+R].$$

不同输出可以并行，但相邻输出的输入邻域高度重叠。优化目标是让重叠数据只从 global memory 加载较少次数。

stencil 也根据邻域更新网格点，例如二维五点 stencil：

$$P_{y,x} = c_0 N_{y,x} + c_1 (N_{y-1,x} + N_{y+1,x} + N_{y,x-1} + N_{y,x+1}).$$

区别主要在于应用语义和邻域形状；优化思想高度相似。

8.2 基本卷积 kernel

Listing 8.1 一维卷积基本 kernel

```

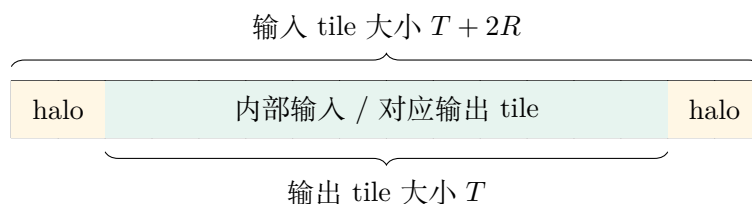
1  __constant__ float mask_c[2 * MAX_RADIUS + 1];
2
3  __global__ void conv1dBasic(const float* in, float* out,
4                             int n, int radius) {
5      int i = blockIdx.x * blockDim.x + threadIdx.x;
6      if (i >= n) return;
7      float acc = 0.0f;
8      for (int j = -radius; j <= radius; ++j) {
9          int k = i + j;
10         if (0 <= k && k < n) {
11             acc += in[k] * mask_c[j + radius];
12         }
13     }
14     out[i] = acc;
15 }

```

mask 很小、只读，且同一 warp 在同一循环步常读取同一系数，因此适合 constant memory。输入仍有大量重复 global load。

8.3 输出 tile、输入 tile 和 halo

若一个 block 产生 T 个连续输出，半径为 R ，则完整输入区域需要 $T + 2R$ 个元素。两侧额外的 R 个元素是 halo。



设计有两类：

显式加载 halo

shared memory 保存完整输入 tile。优点是计算时访问规则；缺点是加载元素多于输出线程，可能需要一个线程加载多个元素。

shared 只存内部，halo 依赖 cache

输入/输出 tile 同尺寸，越过 tile 的邻域直接读 global memory，期望 halo 已被相邻 block 或当前 block 的访问带入 cache。代码更简单，共享内存更少，但性能更依赖 cache 行为。

8.4 二维 tiled 卷积骨架

Listing 8.2 shared 内部 tile + cache halo 的二维卷积骨架

```

1  template<int TILE, int R>
2  __global__ void conv2dTiled(const float* in, float* out,
3                             int width, int height,
4                             const float* mask) {
5      __shared__ float tile[TILE][TILE];
6      int x = blockIdx.x * TILE + threadIdx.x;
7      int y = blockIdx.y * TILE + threadIdx.y;
8
9      tile[threadIdx.y][threadIdx.x] =
10         (x < width && y < height) ? in[y * width + x] : 0.0f;
11      __syncthreads();
12
13      if (x >= width || y >= height) return;
14      float acc = 0.0f;
15      for (int dy = -R; dy <= R; ++dy) {
16          for (int dx = -R; dx <= R; ++dx) {
17              int lx = static_cast<int>(threadIdx.x) + dx;
18              int ly = static_cast<int>(threadIdx.y) + dy;
19              float value;
20              if (0 <= lx && lx < TILE && 0 <= ly && ly < TILE) {
21                  value = tile[ly][lx];
22              } else {
23                  int gx = x + dx, gy = y + dy;
24                  value = (0 <= gx && gx < width &&
25                          0 <= gy && gy < height)
26                      ? in[gy * width + gx] : 0.0f;
27              }
28              acc += value * mask[(dy + R) * (2 * R + 1) + dx + R];
29          }
30      }
31      out[y * width + x] = acc;
32  }

```

实际代码若 mask 大小编译期固定, 可放 constant memory 并展开小循环。若 mask 可分离, 例如二维高斯核可写成横向一维卷积再纵向一维卷积, 计算复杂度从 $O(K^2)$ 降为 $O(2K)$, 这是**算法级优化**, 通常比微调 block 更重要。

8.5 stencil 的迭代与 ping-pong buffer

时间迭代 stencil 通常每轮根据旧网格生成新网格。不能在同一数组上无保护地读写，否则线程可能读到本轮已经更新的数据。常见做法是两个数组交替：

Listing 8.3 host 端 ping-pong 迭代骨架

```
1 for (int step = 0; step < steps; ++step) {
2     stencil<<<grid, block>>>(old_d, next_d, width, height);
3     std::swap(old_d, next_d);
4 }
```

每次 kernel 结束天然形成 grid 级阶段边界；若试图在一个普通 kernel 内跨 block 完成多轮，需要更强的同步机制或改变分解。

8.6 线程粗化与寄存器 tiling

三维 stencil 的 z 方向常适合让一个线程连续计算多个平面：

- x, y 邻域由 shared tile 提供；
- 前一层、当前层、后一层中心值保存在寄存器；
- 每前进一步，寄存器窗口滚动，减少重复加载。

这称为 register tiling。它提升数据复用和 ILP，但增加每线程工作与寄存器压力。

8.7 边界条件要先定义数学语义

常见边界策略包括：

- Dirichlet：边界固定为给定值；
- Neumann：固定法向导数；
- 周期边界：左边连接右边，上边连接下边；
- 图像处理中补零、复制、镜像。

不要把边界判断仅视为“防越界代码”，它是算法定义的一部分。

8.8 性能分析

卷积/模板常是内存受限。评价优化时关注：

- 每个输入元素从 global memory 被加载几次；
- 输出 tile 与输入 tile 比例，halo 开销 $\frac{(T+2R)^d}{T^d}$ ；
- shared memory 使用是否降低 occupancy；
- 边界和 mask 条件是否造成显著发散；
- 分离卷积、稀疏 mask 等算法变换是否可用。

动手练习

1. 对 $T = 16, R = 1$ 的二维卷积，计算完整输入 tile 元素数、输出元素数和 halo 比例；
2. 实现基本卷积、显式 halo tile、cache halo 三版并比较；
3. 对 3D 七点 stencil 设计一个线程沿 z 计算 4 个输出的 register tiling；
4. 将一个可分离 7×7 卷积改为两次一维卷积，比较 FLOP 数和中间数据流量。

掌握检查

看到邻域计算时，应立即识别输入重叠、halo、边界语义和迭代依赖，并能解释 shared tile、cache halo、粗化和寄存器窗口的取舍。

8.9 由浅入深精讲：邻域计算中的 tile 与 halo

从浅到深

第一层：先会用：能写出每个输出遍历邻域的基本 kernel。 **第二层：再理解：**能区分输出 tile、输入 tile 和 halo。 **第三层：最后优化：**能计算 halo 开销、数据复用率和边界策略对性能的影响。

8.10 先从一维卷积看清下标

输入 $x[0 \dots n-1]$ ，半径 r 的卷积核 $w[-r \dots r]$ ，零填充时

$$y_i = \sum_{j=-r}^r w_j x_{i+j},$$

越界的 x 视为 0。最直接实现一线程一输出：

Listing 8.4 一维基本卷积

```
1 __global__ void conv1d(const float* x, const float* w,
```

```

2         float* y, int n, int r) {
3     int i = blockIdx.x * blockDim.x + threadIdx.x;
4     if (i >= n) return;
5     float sum = 0.0f;
6     for (int j = -r; j <= r; ++j) {
7         int k = i + j;
8         if (k >= 0 && k < n) sum += w[j + r] * x[k];
9     }
10    y[i] = sum;
11 }

```

相邻输出的输入窗口高度重叠。例如 $r = 1$ 时, y_i 读取 x_{i-1}, x_i, x_{i+1} , y_{i+1} 读取 x_i, x_{i+1}, x_{i+2} , 两个窗口共享两个元素。

8.11 输出 tile 与输入 tile 的尺寸关系

一个 block 计算 T 个连续输出, 半径为 r , 则所需输入范围宽度为

$$T_{in} = T + 2r.$$

两侧额外的 r 个元素就是 halo。若 $T = 128, r = 3$, 输入 tile 为 134 个元素, halo 比例

$$\frac{2r}{T + 2r} = \frac{6}{134} \approx 4.48\%.$$

若输出 tile 只有 16, halo 比例为 $6/22 \approx 27.3\%$ 。tile 太小会让每个 block 重复加载大量 halo; tile 太大则增加线程、shared memory 和边界复杂度。

8.11.1 加载线程数不必等于输出线程数

输入 tile 可能比 block 线程数多。可让每个线程用循环加载多个元素:

Listing 8.5 协作加载含 halo 的输入 tile

```

1 extern __shared__ float tile[];
2 int out_start = blockIdx.x * blockDim.x;
3 int in_start = out_start - r;
4 for (int t = threadIdx.x; t < blockDim.x + 2 * r;
5     t += blockDim.x) {
6     int g = in_start + t;
7     tile[t] = (g >= 0 && g < n) ? x[g] : 0.0f;
8 }
9 __syncthreads();

```

所有线程必须经过 barrier; 随后只有对应有效输出的线程进行卷积。

8.12 二维卷积的输入 tile

block 输出大小为 $T_y \times T_x$ ，卷积半径为 (r_y, r_x) ，所需输入 tile 为

$$(T_y + 2r_y) \times (T_x + 2r_x).$$

以 16×16 输出和 3×3 核为例，输入 tile 为 $18 \times 18 = 324$ 个元素，而 block 有 256 个线程，因此部分线程要加载第二个元素。每个输入 tile 供 256 个输出使用，内部像素被多次复用。

? 为什么

为什么有的实现让 block 大小等于输入 tile，而另一些让 block 大小等于输出 tile？前者加载映射直观，但 halo 线程最终不写输出，存在空闲 lane；后者所有线程都产生输出，但需要循环协作加载额外输入。两者的优劣取决于核大小、tile 大小和硬件资源。

8.13 constant memory 适合小而只读的卷积核

卷积权重通常远小于输入，且同一 warp 在某一循环迭代读取相同的权重下标。将小卷积核放入 constant memory，可利用广播和缓存，减少参数从 global memory 反复读取。若每个线程读取不同的 constant 地址，广播优势会下降。

8.14 stencil 与卷积的相同和不同

两者都读取邻域，但 stencil 常用于迭代求解：

$$u_{i,j}^{(t+1)} = f(u_{i,j}^{(t)}, u_{i\pm 1,j}^{(t)}, u_{i,j\pm 1}^{(t)}).$$

同一迭代的输出不能覆盖仍被其他线程读取的输入，所以常用 ping-pong buffer：一次从 A 读写 B，下一次从 B 读写 A。不同迭代之间存在全局阶段依赖，最简单方法是每次迭代一个 kernel；kernel 边界提供全局同步。

8.14.1 为什么块内 barrier 不能完成所有迭代

`__syncthreads` 只同步一个 block，不同 block 可能处于不同迭代。如果下一迭代读取邻居 block 尚未完成的数据，会违反算法语义。除非采用受约束的 cooperative grid 同步或重新设计分区，否则应使用 kernel 边界。

8.15 线程粗化与寄存器滑窗

一线程计算多个相邻输出时, 相邻输出窗口也重叠。线程可以把部分输入保存在寄存器中形成滑动窗口: 计算 y_i 后, 计算 y_{i+1} 只需丢弃最左元素并加载一个新元素。收益是减少 shared/global load; 代价是寄存器增加和线程数减少。

分层实验

实验 1: 一维卷积。实现基本版、constant weight 版、shared halo 版, 比较半径 1、3、7。

实验 2: halo 定量分析。对多个 tile 尺寸计算 halo 比例, 并与实际时间对照。

实验 3: 二维 stencil。实现 5 点 stencil 的 ping-pong 迭代, 验证每步边界与 CPU 参考。

实验 4: 粗化。每线程计算 2 或 4 个相邻输出, 记录寄存器和 occupancy 变化。

第 9 章 直方图：原子操作、竞争与私有化

📖 **原书对应：**原书第 9 章：原子操作、基本直方图、原子延迟与吞吐、私有化、线程粗化与线程级私有化。

🎯 本章学习目标

- 解释读-改-写竞争为什么会丢失更新；
- 使用原子操作保证正确性，并理解作用域与内存顺序的概念；
- 通过 block/warp/thread 私有化降低竞争；
- 根据 bin 数量和数据分布判断私有化是否值得。

9.1 为什么普通自增不正确

直方图对输入值 x_i 所属 bin 执行

$$H[b(x_i)] += 1.$$

普通 $H[b]++$ 实际包含读取旧值、加一、写回三步。两个线程可能同时读到 7，各自写回 8，正确结果 9 被丢失。这是**数据竞争**。

9.2 基本原子版本

Listing 9.1 基本图像直方图 kernel

```
1 __global__ void histogramBasic(const unsigned char* data,
2                               unsigned int* bins,
3                               int n) {
4     int i = blockIdx.x * blockDim.x + threadIdx.x;
5     if (i < n) {
6         atomicAdd(&bins[data[i]], 1u);
7     }
8 }
```

原子操作保证对同一位置的更新不可分割，解决正确性，但对同一 bin 的大量更新仍需串行化。若数据集中到少数 bin，竞争严重；若均匀分散到许多 bin，吞吐更高。

9.3 作用域与内存顺序

现代 CUDA C++ 原子接口可表达：哪些线程需要看到该原子 (block、device、system 等作用域)，以及操作之间需要多强的顺序关系。直方图只需保证计数不丢失，通常不需要最强顺序。原则是选择满足正确性的最小作用域和最弱顺序，以避免不必要开销；具体 API 应按所用 CUDA 版本的文档实现。

9.4 私有化：先局部聚合，再全局合并

把一个高度竞争的全局直方图复制成多个局部副本：

1. 每个 block 在 shared memory 中初始化私有 bins；
2. block 内线程原子更新本 block 的私有 bins；
3. barrier，确保局部统计完成；
4. 线程协作把私有 bins 原子加到全局 bins。

Listing 9.2 block 私有化直方图骨架

```

1  __global__ void histogramPrivate(const unsigned char* data,
2                                  unsigned int* bins,
3                                  int n, int numBins) {
4      extern __shared__ unsigned int local[];
5      for (int b = threadIdx.x; b < numBins; b += blockDim.x)
6          local[b] = 0;
7      __syncthreads();
8
9      for (int i = blockIdx.x * blockDim.x + threadIdx.x;
10          i < n;
11          i += blockDim.x * gridDim.x) {
12          atomicAdd(&local[data[i]], 1u);
13      }
14      __syncthreads();
15
16      for (int b = threadIdx.x; b < numBins; b += blockDim.x) {
17          unsigned int v = local[b];
18          if (v) atomicAdd(&bins[b], v);
19      }

```

全局原子次数由近似 n 降为“非零局部 bin 数 $\times$ block 数”。block 内仍有竞争，但 shared memory 原子通常比全局竞争更局部，且不同 block 不再争同一私有副本。

9.5 私有化层次的权衡

层次	优点	代价
每 block 一份	容量相对可控，合并次数低	block 内仍竞争，共享内存限制 occupancy
每 warp 一份	竞争进一步减少	副本数增加，合并工作增加
每 thread 一份	更新无原子竞争	只适合 bin 很少；寄存器/local memory 消耗巨大
分段私有化	只私有化热点或部分 bins	逻辑复杂，需要多轮或分区

私有化不是免费复制。若有 B 个 bins、每项 4 B、每 block 一份，则共享内存为 $4B$ B。bin 太多时无法容纳，或会显著降低驻留 block 数。

9.6 线程粗化与 grid-stride loop

上面的 kernel 让每线程处理多个输入。这样减少 grid 规模和初始化/合并开销占比。grid-stride loop 是常见通用写法：

$$i_0 = \text{global thread id}, \quad i_{k+1} = i_k + \text{total threads in grid}.$$

它保持每轮相邻线程访问相邻输入，通常比让每线程连续处理大块更有利于合并访存。

9.7 热点分布与性能

直方图性能强烈依赖输入分布：

- 均匀分布：更新分散，原子吞吐较好；
- 单点热点：所有线程更新一个 bin，几乎完全串行；
- 局部热点：私有化收益通常明显；
- bin 数巨大且稀疏：初始化整个私有数组可能比有效工作还多。

实验必须同时覆盖均匀、偏斜和极端热点数据，不能只报告一种随机输入。

9.8 正确性与溢出

计数上限可能超过 32 位无符号整数。若 n 很大，应使用 64 位计数并确认硬件/API 支持对应原子操作。还要在 kernel 前把 bins 清零，并检查多次运行是否意外累加旧结果。

动手练习

1. 构造所有输入都属于同一 bin 的数据，对比基本原子和 block 私有化；
2. 改变 bin 数 16、256、4096，记录共享内存、occupancy 与时间；
3. 实现每 warp 一份私有直方图，设计合并步骤；
4. 分析一个稀疏巨大 bin 空间为何不适合完整初始化，并提出哈希或排序替代方案。

掌握检查

你应能把“竞争导致错误”和“竞争导致变慢”分开解释：原子解决前者，私有化主要缓解后者。还能根据 bins、分布和资源容量选择私有化层次。

9.9 由浅入深精讲：原子操作到底解决了什么

从浅到深

第一层：先会用：知道多个线程更新同一 bin 必须协调。 **第二层：再理解：**能写出丢失更新的具体交错。 **第三层：最后优化：**能根据 bin 数量与数据分布选择原子作用域和私有化层次。

9.10 普通自增不是一个不可分割动作

语句 `bins[b]++` 逻辑上包含：读旧值、加一、写新值。假设 bin 初始为 10：

时刻	线程 A	线程 B
1	读取 10	
2		读取 10
3	计算 11	计算 11
4	写入 11	
5		写入 11

正确结果应为 12，实际为 11，这叫丢失更新。原子加保证对同一位置的读改写操作按某种顺序完整执行，因此正确，但热点 bin 上的操作会串行化。

9.11 基本版本的成本模型

每个输入元素做一次原子更新。若输入均匀分布到很多 bin，竞争较低；若大量元素落入少数 bin，吞吐下降。性能不仅由输入长度决定，还由分布决定。评测应至少包含：均匀分布、单热点、偏斜分布和真实数据。

9.12 block 私有化为什么有效

让每个 block 在 shared memory 中维护私有直方图：

1. block 内线程并行清零 shared bins；
2. barrier；
3. 输入元素原子更新 shared bins；
4. barrier；
5. 把每个 block 的局部结果原子累加到 global bins。

全局原子次数从“每输入一次”降为“每 block 每非零 bin 一次”。shared 原子仍可能竞争，但通常延迟更低且作用域更小。

完整例题

输入 $n = 10^8$ ，bin 数 $B = 256$ ，grid 有 4096 个 block。基本版最多有 10^8 次 global atomic；block 私有化的最终合并最多约 $4096 \times 256 \approx 10^6$ 次 global atomic，减少约两数量级。代价是每 block 需要 $256 \times 4 = 1$ KiB shared memory，并多出清零、同步和合并。

9.13 私有化层次怎样选

- **每线程私有**：几乎无竞争，但寄存器/local memory 成本随 bin 数迅速增长；
- **每 warp 私有**：竞争比 block 私有低，副本数量适中；
- **每 block 私有**：实现直接，适合中小 bin 数；
- **分块 bin**：bin 很大时一次只处理一部分，代价是重复扫描输入或更复杂的路由。

不存在无条件最佳层次。关键变量是 B 、分布热点、每 block 输入量、shared memory 容量和合并成本。

9.14 作用域与内存顺序只使用所需强度

若原子对象只由同一 block 访问, block scope 足够; 最终 global histogram 需要 device scope。直方图自增通常只要求原子性, 不依赖复杂的先后可见性, 可使用较弱的 relaxed 顺序。更强的 scope/order 可能增加成本, 但必须以正确性语义为前提。

9.15 grid-stride 与线程粗化

每线程处理多个输入可减少启动和索引开销:

```
1 for (int i = blockIdx.x * blockDim.x + threadIdx.x;
2     i < n; i += blockDim.x * gridDim.x) {
3     int b = map_to_bin(data[i]);
4     // update private/shared histogram
5 }
```

粗化也让每个 block 处理更多元素, 使局部直方图的初始化和最终合并成本被更多工作摊薄。但过度粗化会减少并发和加剧同一局部副本的竞争。

分层实验

实现 global atomic、block-private 和 warp-private 三个版本。对 256、4096、65536 个 bin, 以及均匀/热点输入, 记录吞吐、global atomic 数量近似值、shared memory 和 occupancy。解释为什么某个版本只在部分区间最优。

第 10 章 归约：树形并行、同步与 warp 原语

📖 **原书对应：**原书第 10 章：归约树、基本 kernel、减少控制/内存发散、减少 global memory、warp-level primitives、任意长度与线程粗化。

🎯 本章学习目标

- 把求和、最大值等结合操作写成树形归约；
- 分析工作量、同步次数和控制发散；
- 使用 shared memory 和 warp shuffle 分层归约；
- 正确处理任意长度输入和多 block 结果。

10.1 归约模式

给定结合运算 $\oplus$ ，归约计算

$$y = x_0 \oplus x_1 \oplus \cdots \oplus x_{n-1}.$$

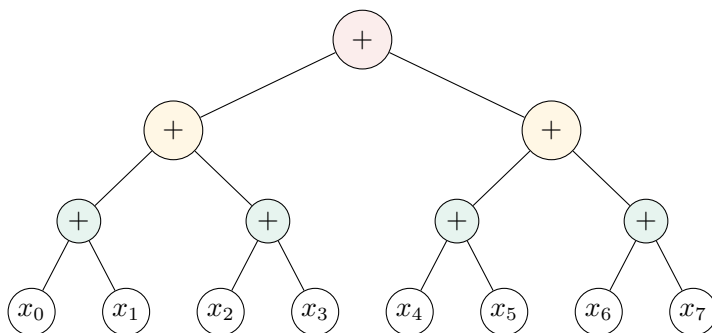
常见运算有加法、最大值、最小值、逻辑与/或。结合性允许改变括号顺序：

$$(a \oplus b) \oplus (c \oplus d).$$

但浮点加法在有限精度下不严格结合，所以并行结果可能与顺序结果有小差异。

10.2 树形归约

顺序求和深度是 $O(n)$ 。平衡二叉树每层把元素数减半，深度为 $O(\log n)$ ，总操作仍约 $n - 1$ 。



10.3 block 级 shared-memory 归约

一种常见结构让每线程先加载一个或多个元素到 shared memory，再逐步把后半部分加到前半部分。

Listing 10.1 block 级求和归约骨架

```

1  __global__ void reduceBlock(const float* in, float* partial, int n) {
2      extern __shared__ float s[];
3      unsigned int t = threadIdx.x;
4      unsigned int i = blockIdx.x * (2 * blockDim.x) + t;
5
6      float x = 0.0f;
7      if (i < n) x += in[i];
8      if (i + blockDim.x < n) x += in[i + blockDim.x];
9      s[t] = x;
10     __syncthreads();
11
12     for (unsigned int stride = blockDim.x / 2;
13         stride > 0; stride >>= 1) {
14         if (t < stride) s[t] += s[t + stride];
15         __syncthreads();
16     }
17     if (t == 0) partial[blockIdx.x] = s[0];
18 }

```

每 block 输出一个 partial sum。随后可再次启动归约 kernel，或在 partial 数量较小时复制到 CPU。不能指望普通 kernel 中不同 block 直接 barrier 后由 block 0 完成最终归约。

10.4 减少控制和内存发散

早期交错索引版本可能使用 `if (t % (2*stride)==0)`，同一 warp 中活跃线程分散，且访问模式不连续。更常用的顺序寻址版本让 `t < stride` 的前半连续线程工作，随着

stride 下降，活跃 warp 整体减少，发散更局部。

还可以在加载阶段让每线程先加两个元素，使输入规模减半并减少 block 数与 global memory 往返。

10.5 warp shuffle: 最后 32 个值不必反复走 shared memory

warp shuffle 允许线程直接读取同 warp 其他 lane 的寄存器值。求和示意：

Listing 10.2 warp 内归约函数

```
1 __device__ float warpReduceSum(float x) {
2     unsigned mask = 0xffffffffu;
3     for (int offset = warpSize / 2; offset > 0; offset >>= 1) {
4         x += __shfl_down_sync(mask, x, offset);
5     }
6     return x; // lane 0 持有总和
7 }
```

完整 block 归约可分两级：每个 warp 先归约，lane 0 把 warp 和写入 shared memory；第一个 warp 再归约这些 warp 和。这样减少 barrier 和 shared memory 访问。

⚠ 易错点

shuffle 的 mask 必须正确描述参与线程。若最后一个 warp 只有部分有效线程，不能无条件使用全 1 mask 并假定无效 lane 有意义。可用活动掩码并对越界输入赋单位元，同时保证参与集合一致。

10.6 任意长度与单位元

对于求和，越界线程加载 0；最大值归约应加载 $-\infty$ 或类型最小值；最小值应加载 $+\infty$ 。这个“不会改变结果的值”称为单位元。任意长度处理的核心是：补齐树结构，但补入单位元。

10.7 线程粗化

每线程可用 grid-stride loop 先累计多个输入到寄存器，再进行 block 归约：

Listing 10.3 每线程局部累加

```

1 float sum = 0.0f;
2 for (int i = blockIdx.x * blockDim.x + threadIdx.x;
3     i < n;
4     i += blockDim.x * gridDim.x) {
5     sum += in[i];
6 }
7 // then reduce 'sum' within the block

```

这降低 block 数和归约树开销，增加 ILP；但 grid 太小会损失并行度，且每线程连续依赖链可能限制流水线。可以用多个累加器打断依赖链。

10.8 数值稳定性

树形求和通常比单一长链有更小的误差增长，但结果仍受顺序影响。对高精度需求可考虑：

- 用 double 累加 float 输入；
- pairwise summation；
- Kahan 等补偿求和（代价更高）；
- 固定归约树以提高可复现性。

性能和数值稳定性必须根据应用要求权衡。

动手练习

1. 把求和 kernel 改为最大值归约，并选择正确单位元；
2. 对 $n = 1000$ 、block 256，计算第一阶段 partial 数量；
3. 实现 shared-only 与 warp-shuffle 两版，比较 barrier 次数；
4. 构造数量级差异很大的浮点输入，比较顺序、树形和 double 累加误差。

掌握检查

你能画出归约树，解释为什么需要多阶段 kernel，正确选择单位元，并说明 shared memory、warp shuffle 与线程粗化各减少了哪类开销。

10.9 由浅入深精讲：从顺序求和到树形归约

从浅到深

第一层：先会用：知道归约把多个输入合成一个值。 **第二层：再理解：**能手算树形阶段与 barrier。 **第三层：最后优化：**能处理任意长度、多 block、warp shuffle 和浮点误差。

10.10 为什么不能让所有线程直接写一个 sum

顺序求和

$$s = (((x_0 + x_1) + x_2) + \cdots) + x_{n-1}$$

有一条长度 n 的依赖链。让每个线程执行 `sum += x[i]` 会发生竞争；使用一个 `global atomic` 虽正确，但所有更新集中到同一地址，无法发挥并行吞吐。树形归约先在局部并行合并，再逐层减少活动元素。

10.11 8 个元素的完整树

初始 $[x_0, \dots, x_7]$:

阶段 1: $[x_0 + x_1, x_2 + x_3, x_4 + x_5, x_6 + x_7]$,

阶段 2: $[(x_0 + x_1) + (x_2 + x_3), (x_4 + x_5) + (x_6 + x_7)]$,

阶段 3: $[\sum_{i=0}^7 x_i]$.

跨度从 7 次串行加法降为 $\log_2 8 = 3$ 阶段，总加法仍为 7，具有工作效率。

10.12 block 级 shared 归约的稳定骨架

Listing 10.4 每 block 输出一个 partial sum

```
1 __global__ void reduce_block(const float* x, float* partial, int n) {
2     extern __shared__ float s[];
3     int t = threadIdx.x;
4     int i = 2 * blockIdx.x * blockDim.x + t;
5     float v = 0.0f;
6     if (i < n) v += x[i];
7     if (i + blockDim.x < n) v += x[i + blockDim.x];
8     s[t] = v;
```

```

9     __syncthreads();
10
11     for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
12         if (t < stride) s[t] += s[t + stride];
13         __syncthreads();
14     }
15     if (t == 0) partial[blockIdx.x] = s[0];
16 }

```

加载时每线程先合并两个元素，block 数减半，global load 连续。每个阶段 barrier 确保下一阶段读取到完整部分和。

10.12.1 为什么这种索引比交错索引好

早期写法可能让线程在 stride 为 1 时用 `if(t % 2 == 0)`，同一 warp 中交替活跃，产生发散；且 shared 访问模式不连续。让前半线程连续活跃并读取相距 stride 的位置，控制和内存行为更规则。

10.13 最后一个 warp：用 shuffle 交换寄存器值

当只剩 32 个值，可以用 `__shfl_down_sync` 让 lane 直接读取同 warp 其他 lane 的寄存器：

Listing 10.5 warp 内求和

```

1 __device__ float warp_reduce_sum(float v) {
2     unsigned mask = 0xffffffffu;
3     for (int offset = 16; offset > 0; offset >>= 1) {
4         v += __shfl_down_sync(mask, v, offset);
5     }
6     return v; // lane 0 contains the sum
7 }

```

这减少 shared memory 读写和 block barrier。但 mask 必须准确描述参与 lane；不满 warp 或条件执行时，不能盲目使用全 1 mask。

10.14 多 block 归约的两阶段

第一 kernel 为每个 block 输出一个 partial；第二 kernel 再归约 partial。若 partial 仍很大可递归多轮，直到只剩一个。kernel 边界提供全局同步。也可用最后一个 block、原

子或 cooperative groups 设计单 kernel 方案，但正确性和可移植性更复杂，初学应先掌握多阶段方法。

✂ 完整例题

$n = 10^7$ ，每 block 256 线程，每线程加载两个元素。第一阶段 block 数约

$$\left\lceil \frac{10^7}{512} \right\rceil = 19532.$$

第二阶段只需归约 19532 个 partial，规模已缩小约 512 倍。第一阶段通常占主要时间。

10.15 单位元与任意长度

求和的单位元是 0，最大值的单位元可取负无穷，乘积的单位元是 1。越界线程应加载单位元，使它不改变结果。这样同一树形结构可以处理非 2 的幂长度。

10.16 浮点加法顺序改变结果

浮点加法不满足严格结合律：

$$(a + b) + c \neq a + (b + c)$$

可能在末位不同。树形归约与 CPU 顺序求和顺序不同，因此验证应使用容差，而非位级相等。对误差敏感时可使用更高精度累加、成对求和、补偿求和或按量级重排，但会增加成本。

10.17 归约的性能账本

理想求和至少读取 $4n$ B，最终写少量 partial，FLOP 约 $n - 1$ ，算术强度约 0.25 FLOP/B，通常内存受限。优化重点：

- 合并 global load，并让每线程加载多个元素；
- 减少 barrier 和 shared 访问；
- 保持足够 block 数隐藏延迟；
- 避免多个阶段产生过多启动与中间流量。

分层实验

按顺序实现：global atomic、基础 shared tree、连续活跃线程、每线程加载两个元素、warp shuffle 收尾。对随机数、全 1、极端量级混合数据验证。记录每版 kernel 时间、有效带宽、barrier 数和误差。

第 11 章 扫描与过滤：把顺序依赖改写为并行前缀

📖 **原书对应：**原书第 11–12 章：Kogge–Stone 与 Brent–Kung 扫描、同步/工作效率/多 block 扫描；不稳定与稳定过滤、原子聚合、私有化和 scan 应用。

🎯 本章学习目标

- 区分 inclusive scan 与 exclusive scan；
- 理解 Kogge–Stone 扫描的 $O(\log n)$ 深度和 $O(n \log n)$ 工作；
- 构造跨 block 的分层扫描；
- 用原子或 scan 实现不稳定/稳定过滤。

11.1 扫描是什么

给定输入 $x_0, \dots, x_{n-1}$ 和结合运算 $\oplus$ ：

$$\text{inclusive: } y_i = x_0 \oplus x_1 \oplus \dots \oplus x_i,$$

$$\text{exclusive: } y_i = e \oplus x_0 \oplus \dots \oplus x_{i-1},$$

其中 e 是单位元。求和输入 $[3, 1, 7, 0, 4]$ ：

$$\text{inclusive} = [3, 4, 11, 11, 15], \quad \text{exclusive} = [0, 3, 4, 11, 11].$$

扫描不仅给总和，还给每个位置之前的聚合结果，因此可用于地址分配、过滤、radix sort、流压缩和动态工作队列。

11.2 为什么顺序写法不能直接线程化

```
1 for (int i = 1; i < n; ++i) x[i] += x[i - 1];
```

第 i 项依赖刚更新的 $i-1$ 项。若所有线程同时执行，会读到不同阶段的数据，产生竞争。并行扫描通过增加每层的“跳距”来构造对数深度的数据依赖图。

11.3 Kogge–Stone 扫描

第 d 轮的步长为 2^d 。线程 i 若 $i \geq 2^d$ ，把前方 2^d 位置的值加到自己。对 8 个元素需要 3 轮。

Listing 11.1 单 block inclusive scan 的教学版

```

1  __global__ void scanKoggeStone(const float* in, float* out, int n) {
2      extern __shared__ float s[];
3      int t = threadIdx.x;
4      s[t] = (t < n) ? in[t] : 0.0f;
5      __syncthreads();
6
7      for (int offset = 1; offset < blockDim.x; offset <= 1) {
8          float add = (t >= offset) ? s[t - offset] : 0.0f;
9          __syncthreads();
10         if (t >= offset) s[t] += add;
11         __syncthreads();
12     }
13     if (t < n) out[t] = s[t];
14 }
```

两次 barrier 的目的分别是：所有线程先完成上一轮读取；再完成本轮写入后进入下一轮。教学代码清楚但同步和 shared memory 流量较多。

11.4 双缓冲与 warp 原语

双缓冲让每轮从旧数组读、向新数组写，避免读写同一 shared 数组的相位问题；代价是共享内存加倍。warp 内扫描可用 shuffle 减少 shared memory 和 barrier：

Listing 11.2 warp 内 inclusive sum scan 示意

```

1  __device__ float warpScan(float x) {
2      unsigned mask = 0xffffffffu;
3      int lane = threadIdx.x % warpSize;
4      for (int offset = 1; offset < warpSize; offset <= 1) {
5          float y = __shfl_up_sync(mask, x, offset);
6          if (lane >= offset) x += y;
7      }
8      return x;
9  }
```

block 扫描可先做每 warp 扫描，再扫描各 warp 总和，最后给每个 warp 加上前面 warp

的前缀。

11.5 工作效率

顺序扫描做 $n - 1$ 次运算。Kogge–Stone 每层大量线程工作，总工作约 $O(n \log n)$ ，并非 work-efficient。Brent–Kung 等树形方法可接近 $O(n)$ 工作，但深度和索引模式不同。

work efficiency

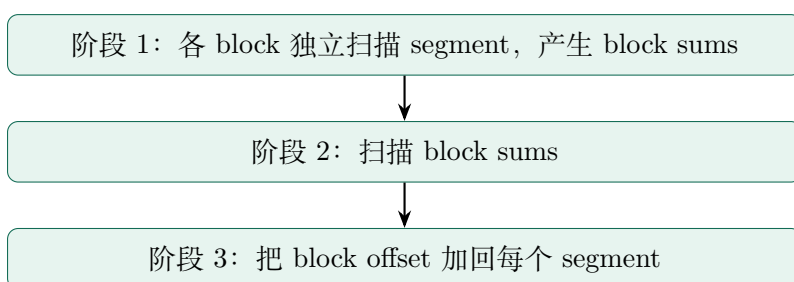
并行算法的总操作数量与最佳顺序算法是否同阶。并行时间短但总工作过多，规模增大后可能浪费算力和能量，甚至变慢。

线程粗化可让每线程先顺序扫描一小段，再扫描各段总和，最后把段前缀加回。这既减少全局并行扫描的元素数，也提高工作效率和访存利用率。

11.6 跨 block 的全局扫描

普通 block barrier 不能跨 block 同步。典型三阶段分解：

1. 每个 block 扫描自己的 segment，并输出 block sum；
2. 扫描 block sums，得到每个 block 之前的总和；
3. 每个 block 把对应 offset 加到自己的局部扫描结果。



更高级的 lookback 方法允许 block 通过状态和原子语义获取前驱前缀，减少独立 kernel 阶段，但实现与正确性推理更复杂。初学应先掌握三阶段方案。

11.7 过滤：保留满足谓词的元素

给定谓词 $p(x)$ ，过滤输出所有 $p(x) = \text{true}$ 的元素。

11.7.1 不稳定过滤：原子分配位置

Listing 11.3 不稳定过滤的基本思想

```

1  if (pred(x[i])) {
2      unsigned int pos = atomicAdd(count, 1u);
3      out[pos] = x[i];
4  }
```

输出顺序取决于线程到达原子的顺序，因此不稳定。可用 warp vote/shuffle 将一个 warp 的多个保留元素聚合为一次原子申请，再由 lane 前缀确定 warp 内位置，减少原子次数。

11.7.2 稳定过滤：predicate + exclusive scan + scatter

先生成标志

$$f_i = \begin{cases} 1, & p(x_i) \\ 0, & \text{otherwise} \end{cases}$$

对 f 做 exclusive scan 得到 s_i 。若 $f_i = 1$ ，元素写到 $\text{out}[s_i]$ 。因为 s_i 等于前面满足谓词的元素数，原顺序被保留。

例：

$$x = [8, 3, 6, 1, 4], \quad f = [1, 0, 1, 0, 1], \quad s = [0, 1, 1, 2, 2],$$

输出为 $[8, 6, 4]$ 。

11.8 in-place 稳定过滤为何困难

若输入和输出同一数组，前部元素写入可能覆盖后面尚未读取的输入。需要更复杂的移动调度、分块缓冲或双向策略。除非内存约束强烈，先实现 out-of-place 版本更稳妥。

11.9 过滤优化选择

需求	推荐基本方案	主要瓶颈
不要求顺序、保留率低	原子计数	原子竞争可能较小
不要求顺序、保留率高	warp 聚合/私有化	减少原子次数
必须稳定	标志 + scan + scatter	多阶段和中间数组流量
超大数据	分块 scan/filter	block sums 和全局协调
内存极紧	in-place 稳定方案	实现复杂、同步困难

动手练习

1. 手算 8 个元素的 Kogge-Stone 每一轮结果；
2. 把 inclusive scan 转为 exclusive scan；
3. 实现稳定过滤，验证输出顺序；
4. 对保留率 1%、50%、99% 比较基本原子、warp 聚合和 scan 过滤。

掌握检查

你应能把扫描理解成“为每个元素计算前面有多少工作”，并用它解决稳定过滤位置分配。还应能解释 Kogge-Stone 的深度优势与工作效率代价，以及跨 block 扫描为何需要额外协调。

11.10 由浅入深精讲：扫描怎样重构顺序依赖

从浅到深

第一层：先会用：区分 inclusive scan 和 exclusive scan。 **第二层：再理解：**能手算 Kogge-Stone 每个阶段。 **第三层：最后优化：**能理解跨 block 扫描以及稳定过滤为什么依赖 scan。

11.11 inclusive 与 exclusive 的具体区别

输入 [3, 1, 4, 2]:

inclusive = [3, 4, 8, 10], exclusive = [0, 3, 4, 8].

exclusive scan 的第 i 项表示“在我之前所有元素的和”，因此可作为每个元素的无冲突写入位置。很多并行算法需要的不是最终总和，而是每个前缀的定位信息。

11.12 Kogge–Stone: 8 个元素逐阶段

令初始数组 $x^{(0)}$ 。阶段 s 的距离 $d = 2^s$ ：

$$x_i^{(s+1)} = \begin{cases} x_i^{(s)}, & i < d, \\ x_i^{(s)} + x_{i-d}^{(s)}, & i \geq d. \end{cases}$$

对 8 个元素需要距离 1、2、4 三个阶段。每个阶段所有线程必须先读取旧值，再统一写新值；若原地更新且没有正确同步/临时值，后面的线程可能读取到同阶段刚写的数据，破坏算法。

Listing 11.4 block 内 Kogge–Stone 的概念版本

```

1 extern __shared__ int s[];
2 int t = threadIdx.x;
3 s[t] = input[...];
4 __syncthreads();
5 for (int offset = 1; offset < blockDim.x; offset <= 1) {
6     int add = (t >= offset) ? s[t - offset] : 0;
7     __syncthreads();
8     s[t] += add;
9     __syncthreads();
10 }
```

第一个 barrier 确保所有线程都已读取旧阶段值；第二个确保写完新阶段后再进入下一轮。可用双缓冲、warp shuffle 和更精细的实现减少同步。

11.13 工作效率：为什么 Kogge–Stone 不是唯一方案

长度 n 时，Kogge–Stone 约做 $n \log_2 n$ 次加法，而顺序 scan 只做 $n - 1$ 次。它跨度短、并行度高，但总工作更多。Brent–Kung 等树型方法减少总工作，却有不同阶段、活跃线程和同步特征。GPU 上的选择还受 warp 原语、内存和 block 大小影响，不能只看大 O。

11.14 跨 block 扫描的三步分解

对长数组：

1. 每个 block 独立扫描自己的 segment，并输出 block 总和；
2. 扫描 block 总和，得到每个 block 之前的累计 offset；
3. 每个 block 把自己的 offset 加到局部扫描结果。

若是 exclusive scan，局部和全局的 inclusive/exclusive 关系必须统一。错误常出现在“block 0 的 offset 是多少”“最后一个元素的总和在哪里”。

✂ 完整例题

输入分成三个 block，局部总和分别为 [10, 7, 20]。block offset 的 exclusive scan 是 [0, 10, 17]。block 0 不加；block 1 的所有局部结果加 10；block 2 加 17。这样得到全局前缀。

11.15 稳定过滤: scan 把布尔谓词变成写位置

对每个元素计算谓词 $p_i \in \{0, 1\}$ 。exclusive scan

$$pos_i = \sum_{j < i} p_j$$

给出元素 i 之前有多少个保留元素。若 $p_i = 1$ ，把输入写到 `out[pos_i]`。因为原数组中更早的保留元素获得更小位置，顺序保持稳定。

✂ 完整例题

输入 [7, 2, 9, 4, 5]，保留偶数：

$$p = [0, 1, 0, 1, 0], \quad pos = [0, 0, 1, 1, 2].$$

元素 2 写 `out[0]`，元素 4 写 `out[1]`，输出 [2, 4]。最后输出长度为 $pos_{n-1} + p_{n-1}$ 。

11.16 不稳定过滤为什么可以更简单

每个保留元素执行 atomic increment 取得位置：

```
1 if (keep(x[i])) {
2     int pos = atomicAdd(counter, 1);
3     out[pos] = x[i];
4 }
```

正确但位置分配顺序取决于线程执行，不能保证输入顺序。可以用 warp 聚合：先用 ballot 得到 warp 中保留 lane，warp 只做一次 atomic 申请连续区间，再由 lane 内前缀确定偏移，减少原子次数。

11.17 scan 的典型用途

除了过滤，scan 还用于：

- radix sort 中计算每个 digit 的目标位置；
- 稀疏数据压缩与 CSR 行指针；
- 图 frontier 构建；
- 变长输出的空间分配；
- stream compaction 和分段数据布局。

它是把“每个任务产生多少输出”转换为“每个任务从哪里写”的通用桥梁。

分层实验

实验 1：手算。对 8 个数写出 Kogge–Stone 每阶段数组。

实验 2：block scan。同时实现 inclusive 和 exclusive，测试非 2 的幂长度。

实验 3：全局 scan。实现 scan–scan–add 三阶段，并验证 block 边界。

实验 4：过滤。比较 atomic 不稳定过滤与 scan 稳定过滤，分别测试保留率 1%、50%、99%。

第 12 章 合并与排序：数据相关分区和规则化重排

📖 **原书对应：**原书第 13–14 章：co-rank、并行 merge、tiled/circular-buffer merge、线程粗化；odd-even、merge sort、radix sort 与合并访存。

🎯 本章学习目标

- 用 co-rank 把两个有序数组的合并结果分段；
- 理解并行 merge 的负载均衡与有序边界；
- 区分比较排序和 radix sort；
- 说明 filter/scan 如何组成并行 radix sort。

12.1 顺序 merge

输入有序数组 A 、 B ，用两个指针选择较小者写入 C 。工作量 $O(|A| + |B|)$ ，但一个指针链具有顺序依赖。

并行化的难点不是“让很多线程各自 merge”，而是为每个线程找到不重叠且边界正确的输入子区间。

12.2 co-rank：输出前缀对应多少 A 和 B

设输出前 k 个元素由 A 前 i 个和 B 前 j 个组成，则

$$i + j = k.$$

正确分割还需满足边界有序条件，例如

$$A[i - 1] \leq B[j], \quad B[j - 1] < A[i]$$

(具体严格/非严格关系决定相等键时的稳定性)。通过在 i 的可行区间二分搜索，可在 $O(\log \min(|A|, |B|))$ 时间找到 co-rank。

💡 直觉

把 A 和 B 看成两条已排序队伍。输出前 k 人一定从两队各取一个前缀。co-rank 的任务是找出这两个前缀的切点，而不是实际搬运所有人。

12.3 并行 merge 分区

若有 P 个线程或 tile，可令输出边界

$$k_p = \left\lfloor \frac{p(|A| + |B|)}{P} \right\rfloor, \quad p = 0, \dots, P.$$

对每个 k_p 计算 co-rank (i_p, j_p) ，线程 p 只需 merge

$$A[i_p : i_{p+1}) \quad \text{和} \quad B[j_p : j_{p+1})$$

到 $C[k_p : k_{p+1})$ 。输出工作量近似均分，即使两个输入元素分布不同也能保持负载平衡。

Listing 12.1 co-rank 的伪代码骨架

```

1  int coRank(int k, const Key* A, int m, const Key* B, int n) {
2      int iLow = max(0, k - n);
3      int iHigh = min(k, m);
4      while (iLow <= iHigh) {
5          int i = (iLow + iHigh) / 2;
6          int j = k - i;
7          if (i > 0 && j < n && A[i - 1] > B[j]) {
8              iHigh = i - 1;
9          } else if (j > 0 && i < m && B[j - 1] >= A[i]) {
10             iLow = i + 1;
11          } else {
12              return i;
13          }
14      }
15      return iLow;
16 }

```

12.4 tiled merge 与 circular buffer

直接 merge global memory 可能访问不规则。tiled 版本把输入小段加载到 shared memory，再局部合并，提高合并访问和缓存复用。输入消耗速度由键分布决定，两个 tile 可能不同步耗尽；circular buffer 可在固定共享内存中滚动装入新数据，避免频繁搬移剩余元素。

线程粗化让每线程输出多个连续元素，减少 co-rank 和分区开销，但需要平衡每线程顺序工作与总并行度。

12.5 排序方法概览

12.5.1 odd-even sort

通过奇偶相邻比较阶段逐步排序，概念简单、同步多、工作量较高，适合作为教学或小规模 block 内排序，不适合大规模通用排序。

12.5.2 merge sort

递归地排序子段并合并。GPU 上关键是底层小段排序、并行 merge、阶段间 kernel 调度和全局内存流量。

12.5.3 radix sort

不直接比较完整键，而按若干位的 digit 分多轮稳定分配。若每轮处理 r 位，有 2^r 个 bucket。每轮需要：

1. 提取 digit；
2. 统计各 bucket 数量；
3. 扫描得到 bucket 起始 offset；
4. 稳定地把元素 scatter 到下一数组。

因此 radix sort 依赖 histogram、scan 和 stable filter 三类模式。

12.6 radix 大小的权衡

每轮位数 r 越大，轮数减少，但 bucket 数 2^r 指数增加：

- 私有 histogram 更大；
- 扫描与 offset 管理更复杂；
- shared memory/寄存器压力增大；
- 数据分布与键宽度影响最佳选择。

12.7 稳定性与键值对

排序常处理 $(key, value)$ 。稳定排序保证相等 key 的原始相对顺序不变，对多关键字排序和 radix 多轮处理至关重要。只移动 key 而忘记同步移动 $value$ 会破坏语义。

12.8 内存流量是排序的主要成本

大规模排序通常多轮读写整个数组。即使每元素比较或位运算很少，数据往返会占主导。优化重点包括：

- 合并读写；
- 每轮融合多个小操作，减少中间数组；
- 在 shared memory 中处理局部 tile；
- 双缓冲输入/输出数组，避免不安全 in-place；
- 合理选择 radix，减少轮数而不过度增加局部资源。

动手练习

1. 给定 $A = [1, 4, 7, 9]$ 、 $B = [2, 3, 8]$ ，为 $k = 0 \dots 7$ 手算 co-rank；
2. 用 4 个线程均分 13 个输出，写出每个线程的输出区间；
3. 设计 32 位无符号整数、每轮 4 bit 的 radix sort 流程，计算轮数和 bucket 数；
4. 对几乎有序、逆序、重复键很多三类输入比较 merge sort 和 radix sort 的数据依赖。

掌握检查

你能说明 co-rank 如何把数据相关的 merge 变成独立区间，并能把 radix sort 拆成 histogram、scan、stable scatter。还应理解排序优化常由内存流量而非比较次数主导。

12.9 由浅入深精讲：co-rank 如何把一个 merge 切成独立任务

从浅到深

第一层：先会用：知道两个有序数组可线性合并。 **第二层：再理解：**知道输出前缀长度决定两个输入前缀长度。 **第三层：最后优化：**能用 co-rank 为线程或 block 生成无重叠分区。

12.10 输出前缀对应输入前缀

有序数组 A 长 m 、 B 长 n ，合并输出 C 。取输出前 k 个元素，它们必然由 A 的前 i 个和 B 的前 j 个组成：

$$i + j = k.$$

正确分界还需满足（忽略边界）：

$$A_{i-1} \leq B_j, \quad B_{j-1} < A_i.$$

这保证左侧选出的元素都不大于右侧未选元素，并用一个严格不等式处理稳定性。通过二分搜索 i 即可求 co-rank， $j = k - i$ 。

完整例题

$A = [1, 4, 7, 10]$ ， $B = [2, 3, 8, 9]$ ，求输出前 $k = 5$ 的 co-rank。尝试 $i = 3, j = 2$ ：已选 A 前 3 个 $[1, 4, 7]$ 和 B 前 2 个 $[2, 3]$ 。但 $A_2 = 7 > B_2 = 8$ 为假？实际 $7 \leq 8$ 成立，且 $B_1 = 3 < A_3 = 10$ 成立，所以 $(i, j) = (3, 2)$ 。前五个输出是 $[1, 2, 3, 4, 7]$ 。

12.11 并行分区

若希望 P 个任务，每个任务负责输出区间

$$[k_p, k_{p+1}), \quad k_p = \left\lfloor \frac{p(m+n)}{P} \right\rfloor,$$

分别求 k_p 和 k_{p+1} 的 co-rank，就得到该任务在 A 、 B 中的输入区间。每个任务内部顺序 merge，任务之间互不重叠且覆盖全部输出。

12.12 为什么需要 tiled merge

直接让每个任务从 global memory 合并一段，输入读取模式可能不够连续，且任务粒度小会有二分与启动开销。tiled merge 把输入片段加载到 shared memory，线程在片上

合作合并，再连续写回。circular buffer 可在 tile 滑动时复用未消费输入，减少重复搬运。

12.13 radix sort 是多个模式的组合

对某一位/某个 radix digit：

1. 计算每个元素的 digit；
2. histogram 统计各 bucket 数量；
3. scan 得到 bucket 起始位置；
4. 对每个元素计算 bucket 内局部排名；
5. scatter 到新位置；
6. 处理下一组 bit。

因此 radix sort 依赖 histogram、scan、稳定 filter/scatter。radix 越大，pass 数越少，但 bucket 数、私有直方图和 scan 成本越高。

12.14 稳定排序和键值对

只排序 key 时，稳定性决定相同 key 的原顺序是否保留；排序 key-value 时，必须让 value 随 key 一起移动。radix sort 每一 pass 必须稳定，否则低位已经建立的顺序会被高位处理破坏。

分层实验

手算两个小数组在 $k = 0, 3, 5, 8$ 的 co-rank。实现“每线程一段”的并行 merge，验证所有分区边界。随后实现 4-bit 与 8-bit radix pass，比较 pass 数、共享内存和时间。

第 13 章 高级矩阵乘优化：从 shared tile 到寄存器分块

📖 **原书对应：**原书第 15 章，并回顾第 3、5、6 章：数据复用分析、较大 tile、线程粗化、寄存器 tiling、合并写回、bank conflict、occupancy、软件流水与专用硬件。

🎯 本章学习目标

- 解释矩阵乘为何适合多级分块；
- 设计“每线程计算多个 C 元素”的寄存器 tile；
- 处理向量加载、转置布局、合并写回和 bank conflict；
- 理解低 occupancy 下用 ILP 和软件流水维持吞吐；
- 知道何时应直接使用成熟 GEMM 库或专用矩阵硬件。

13.1 优化阶梯

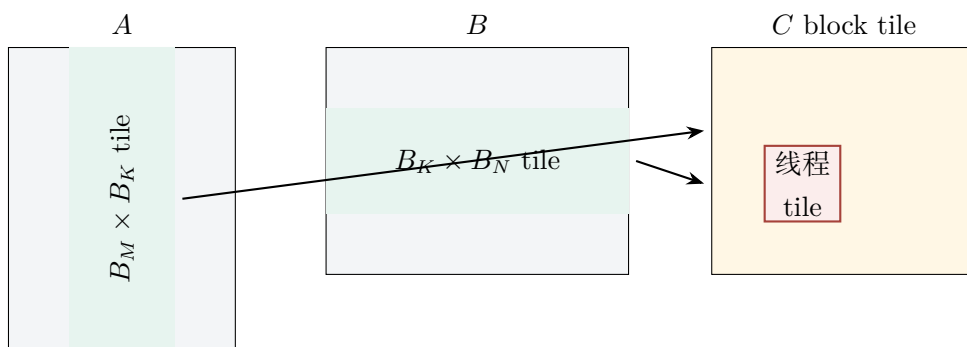
矩阵乘 $C_{M \times N} = A_{M \times K} B_{K \times N}$ 是数据复用最丰富的计算之一。典型优化层次：

1. 一线程一输出的朴素 kernel；
2. block 级 shared-memory tiling；
3. 一个 block 产生更大的输出 tile；
4. 每线程计算一个寄存器子 tile；
5. 向量化、合并加载和合并写回；
6. shared memory 布局消除 bank conflict；
7. 双缓冲/异步复制形成软件流水；
8. 使用 tensor core 等专用单元。

13.2 多级数据复用

假设 block 计算 $B_M \times B_N$ 输出 tile，并沿 K 方向每次处理 B_K ：

- A tile 大小 $B_M \times B_K$ ，被输出 tile 的 B_N 列复用；
- B tile 大小 $B_K \times B_N$ ，被输出 tile 的 B_M 行复用；
- shared memory 在 block 内复用；
- 每线程寄存器 tile 在单线程内进一步复用加载到寄存器的 A/B 片段。



13.3 每线程计算多个输出

若每线程计算 $T_M \times T_N$ 个 C 元素，需要 $T_M T_N$ 个累加器。每个 k 步加载 T_M 个 A 值和 T_N 个 B 值，然后做外积：

$$\text{acc}_{i,j} += a_i b_j, \quad i = 0 \dots T_M - 1, j = 0 \dots T_N - 1.$$

加载数量是 $T_M + T_N$ ，乘加数量是 $T_M T_N$ ，寄存器内复用随 tile 增大而增强。

Listing 13.1 寄存器外积的核心伪代码

```

1 float acc[TM][TN] = {0};
2 for (int k = 0; k < BK; ++k) {
3     float a[TM];
4     float b[TN];
5     // load a[] and b[] from shared memory
6     #pragma unroll
7     for (int i = 0; i < TM; ++i)
8         #pragma unroll
9         for (int j = 0; j < TN; ++j)
10             acc[i][j] += a[i] * b[j];
11 }
```

代价是寄存器数量快速增长，可能降低 occupancy。高性能 GEMM 往往接受较低 occupancy，用高 ILP、强复用和流水线弥补。

13.4 较大 block tile 不等于更多线程

block 最大线程数有限。要产生更大的输出 tile，不能简单让每输出一个线程，而应让每线程计算多个元素。这样增加计算粒度，降低 block 数，但必须确保 grid 仍有足够 block 填满所有 SM，并避免严重 tail effect。

13.5 global load 的布局与向量化

加载 A/B tile 时，希望 warp 的线程读取连续地址。常见做法：

- 按线性线程 id 分配 tile 元素；
- 每线程用 float4 加载连续 4 个对齐元素；
- 必要时把 B tile 以转置或 swizzle 形式写入 shared memory，使后续读取无冲突；
- 非整块边界使用掩码或补零。

向量加载减少指令数，但要求地址和 leading dimension 满足对齐；边界路径需要标量或掩码处理。

13.6 合并写回

寄存器 tile 中同一线程的多个结果不一定在全局内存连续。若线程映射不当，写回会跨步。可通过选择线程 tile 的列方向、在 shared memory 中暂存并重排输出、或使用向量存储，使相邻 lane 写连续地址。

13.7 bank conflict 与 shared layout

矩阵乘中 shared tile 的读取模式比简单转置复杂。可采用 padding、交错布局或 swizzle，让同一 warp 的各 lane 访问不同 bank。布局优化必须与向量加载、线程 tile 和硬件 bank 组织一起设计，不能只机械加一列。

13.8 软件流水与双缓冲

阶段 p 计算时，提前准备阶段 $p + 1$ ：

1. global memory $\rightarrow$ 下一 shared buffer；
2. 当前 shared buffer $\rightarrow$ 寄存器并计算；

3. 等待下一 buffer 就绪；
4. 交换 buffer。

还可在 shared $\rightarrow$ register 层进一步预取下一 k 的片段，形成多级流水。流水增加状态、寄存器和边界处理复杂度，必须用 profiler 确认加载与计算确实重叠。

13.9 专用矩阵硬件

现代 GPU 提供面向小矩阵乘累加的专用单元以及配套的数据搬运能力。程序通常把大 GEMM 分解为硬件支持的 fragment，使用低精度输入和更高精度累加。关键问题包括：

- 支持的数据类型与 tile 形状；
- 矩阵布局、对齐和 leading dimension；
- 精度、溢出和累加类型；
- 数据搬运是否能跟上计算；
- 边界和非规则尺寸处理。

💡 直觉

高性能 GEMM 是“数据搬运系统”，不是只有乘法循环。真正的目标是让每一级内存都以合适的 tile 和节奏供数，使矩阵单元持续工作。

13.10 何时自己写，何时用库

生产环境通常优先使用经过长期调优的 BLAS/GEMM 库。自己写 kernel 适合：

- 学习性能机制；
- 特殊形状、特殊融合或非标准数据类型；
- 研究新的布局/硬件特性；
- 库调用前后有大量可融合操作。

比较时必须与库的相同精度、相同数学语义、相同布局和相同计时范围对齐。

13.11 性能建模检查

- 计算量约 $2MNK$ FLOP；

- 理想最小 global memory 流量接近读 A/B 一次、写 C 一次，但实际由 tile 复用和 cache 决定；
- 报告 GFLOP/s 或 TFLOP/s，并与该数据类型的合理硬件上限比较；
- 检查数值误差，尤其低精度与不同累加顺序。

动手练习

1. 设计 128×128 block tile、 8×8 thread tile，需要多少线程？每线程累加器多少个？
2. 比较 $T_M = T_N = 1, 4, 8$ 的寄存器复用和寄存器压力；
3. 给 shared tile 添加 padding 或转置布局，测量 bank conflict；
4. 与成熟 GEMM 库对比，并分别报告 kernel-only 和端到端时间。

掌握检查

你应能从朴素矩阵乘逐级解释 shared tile、线程粗化、寄存器 tile、向量加载、合并写回和软件流水，而不是只背一个最终 kernel。还应知道高性能库通常是实际首选。

13.12 由浅入深精讲：高性能 GEMM 的多级复用

从浅到深

第一层：先会用：shared tile 复用 global 数据。 **第二层：再理解：**每线程用寄存器计算多个输出，复用 shared 数据。 **第三层：最后优化：**同时协调 global 布局、shared bank、寄存器、occupancy 和软件流水。

13.13 GEMM 为什么值得复杂优化

矩阵乘运算量为约 $2MNK$ FLOP，输入输出数据量仅 $MK + KN + MN$ 个元素。若能充分复用，算术强度随矩阵尺寸增长，适合 GPU 的高计算吞吐。难点是把理论复用真正落到寄存器和 shared memory，而不是反复从 DRAM 读取。

13.14 三级 tile 的概念

常见分层：

- **block tile**：一个 thread block 负责 $B_M \times B_N$ 输出区域；

- **K tile**: 每次从 global 加载 $B_M \times B_K$ 的 A 和 $B_K \times B_N$ 的 B 到 shared;
- **thread tile**: 每线程在寄存器中维护 $T_M \times T_N$ 个输出累加器。

block 所需线程数约为

$$\frac{B_M}{T_M} \times \frac{B_N}{T_N},$$

所以可以让输出 tile 大于线程二维尺寸。

✂ 完整例题

取 $B_M = B_N = 128$ ，每线程计算 $T_M = T_N = 8$ ，则线程网格为 $16 \times 16 = 256$ 。每个 block 产生 $128 \times 128 = 16384$ 个输出，但只用 256 线程，每线程维护 64 个累加器。这样同一 shared A 值可被多个列输出复用，同一 shared B 值可被多个行输出复用。

13.15 寄存器外积更新

每个 K 步，从 shared 读取 T_M 个 A 值和 T_N 个 B 值，做一个小外积：

$$C_{reg}[i, j] += A_{reg}[i] B_{reg}[j], \quad 0 \leq i < T_M, 0 \leq j < T_N.$$

一次读取 $T_M + T_N$ 个值，产生 $T_M T_N$ 个乘加，寄存器复用显著。但累加器数量 $T_M T_N$ 直接增加寄存器压力。

13.16 global load 的向量化与布局

加载 tile 时希望 warp 访问连续、对齐地址。可用 float4 表达 16 B 向量加载，但必须满足：

- 基地址和行起点满足对齐；
- tile 边界和尾部安全；
- 向量化后 lane 映射仍覆盖完整 tile；
- 寄存器/指令变化确实有收益。

有时为了让计算阶段 shared 读取无冲突，会以转置或 swizzle 形式存入 shared。global 最优布局和 shared 最优布局可能不同，加载阶段承担重排。

13.17 合并写回

thread tile 的输出在寄存器中可能按计算友好布局排列，但最终 global store 要让 warp 写连续地址。若每线程的输出位置分散，可在 shared 中重新排列或改变线程到输出

的映射。写回只发生一次，但大型 GEMM 的输出量仍可观，且不合并会浪费事务。

13.18 bank conflict 的消除不是只加一列

高级 GEMM 可能对 shared tile 使用 padding、转置、交错或 swizzle。目标是让同一 warp 在每条 load 指令中访问不同 bank，或者利用广播。应从“lane l 访问的 shared 地址”推导 bank 映射，而不是机械套用 $TILE+1$ 。

13.19 软件流水：加载下一块时计算当前块

双缓冲为 A、B 各准备两个 shared tile：

1. 预加载 tile 0；
2. 计算 tile 0，同时发起 tile 1 的加载；
3. 等待 tile 1 可用并交换缓冲；
4. 重复直到 K 维结束。

有效时可把加载延迟隐藏在计算中，但 shared memory 翻倍，可能减少驻留 block。若单 block 已有足够计算隐藏加载，或 kernel 受计算峰值限制，收益可能有限。

13.20 一个简化资源预算

假设 block 256 线程，每线程 80 寄存器，shared 双缓冲总 64 KiB。即使线程槽允许多个 block，寄存器和 shared memory 也可能只允许每 SM 驻留 1 个 block。低 occupancy 不一定差：一个 block 内可能有 8 warp，并具有大量 ILP 和异步流水，足以保持计算管线忙。高级 GEMM 常主动用较低 occupancy 换更高数据复用。

13.21 与专用硬件的关系

Tensor Core 等专用单元按小矩阵片段执行混合精度矩阵乘累加。高性能实现仍需要同样的层次：global tile、shared staging、寄存器 fragment、流水和布局，只是计算微内核由专用指令完成。理解普通 CUDA GEMM 是理解专用矩阵硬件的软件基础。

13.22 逐级优化的实验顺序

1. 朴素一线程一输出；

2. shared tile;
3. 每线程多个输出;
4. register tiling 外积;
5. 向量化 global load;
6. shared padding/swizzle;
7. 合并写回;
8. 双缓冲/异步复制;
9. 与高性能库比较。

每一步同时记录 GFLOP/s、带宽、寄存器、shared、occupancy 和数值误差。若某一步没有收益，应解释当前瓶颈，而不是继续堆叠技巧。



分层实验

选择至少三组形状：大方阵、 M 很小的矩阵、 K 很小的矩阵。相同 kernel 在不同形状下复用和并行度不同。绘制性能随形状和 tile 参数变化的表，并说明为什么“方阵最优参数”不能直接迁移到所有 GEMM。

第四部分

高级模式、AI 与多 GPU

第 14 章 动态规划与 wavefront：沿依赖前沿提取并行性

■ 原书对应：原书第 16 章：动态规划、实现方法、wavefront、Floyd-Warshall、Smith-Waterman、block tiling、超平面变换与进一步优化。

◎ 本章学习目标

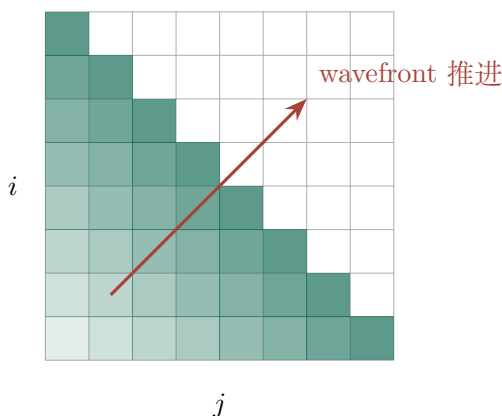
- 从动态规划递推式画出依赖图；
- 识别反对角线或 wavefront 上可并行的单元；
- 用 block tile 减少 kernel 启动和全局同步；
- 理解超平面/倾斜变换如何改善线程利用率和局部性。

14.1 动态规划为何“不完全并行”

动态规划把问题拆成重叠子问题并保存中间结果。以二维递推为例：

$$D[i, j] = F(D[i - 1, j], D[i, j - 1], D[i - 1, j - 1], x_i, y_j).$$

同一行相邻元素存在依赖，不能让所有 (i, j) 同时执行；但满足 $i + j = c$ 的反对角线元素只依赖更小的 $i + j$ ，因此同一 wavefront 可并行。



14.2 最直接方法：一条反对角线一个 kernel

第 d 条反对角线包含满足 $i + j = d$ 的单元。host 依次启动 kernel，kernel 内线程并行计算该对角线。kernel 边界形成全局同步，正确且易懂，但小 wavefront 上线程少、启动次数多，利用率低。

14.3 Floyd–Warshall 的阶段依赖

全源最短路递推：

$$D_{i,j}^{(k)} = \min \left(D_{i,j}^{(k-1)}, D_{i,k}^{(k-1)} + D_{k,j}^{(k-1)} \right).$$

固定 k 时，大量 (i, j) 可并行；不同 k 顺序依赖。高效 tiled Floyd–Warshall 常分三类 block：

1. 更新包含 pivot (k, k) 的对角 tile；
2. 更新 pivot 所在行和列的 tiles；
3. 使用已更新行/列 tiles 更新其余 tiles。

每个阶段内部可并行，阶段之间需全局顺序。

14.4 Smith–Waterman 序列比对

局部序列比对的一个简化递推是

$$H_{i,j} = \max \begin{cases} 0, \\ H_{i-1,j-1} + s(x_i, y_j), \\ H_{i-1,j} - g, \\ H_{i,j-1} - g. \end{cases}$$

依赖来自左、上、左上，天然形成反对角 wavefront。实际仿射 gap 还会增加辅助矩阵，但并行结构类似。

14.5 block-level tiling

把 DP 表切成 $T \times T$ tiles。一个 block 在 shared memory 中计算一个 tile，tile 之间按 wavefront 调度。这样：

- 全局同步从每个元素对角线降为每个 tile 对角线；

- tile 内中间数据可驻留 shared memory；
- 每个 block 有更多连续工作，摊薄启动开销。

但方形 tile 内部的有效 wavefront 长度先增长后缩短，早期和末期 warp 利用率低；边界数据的布局也可能不利于连续访问。

14.6 超平面/倾斜变换

可以改变迭代坐标，让依赖方向与线程执行方向更匹配。例如定义

$$u = i + j, \quad v = j,$$

原来反对角线 $i + j = u$ 变成固定 u 的“直线”。在 tile 级别做倾斜 (skewing) 可把菱形/平行四边形工作域映射成更规则的线程区域，使更多线程长期活跃，并改善边界数据的连续性。

💡 直觉

并行化不一定改变数学递推，但可以改变“用什么坐标遍历状态空间”。优秀的问题变换会让依赖保持正确，同时让并行前沿更规则。

14.7 同步层次设计

wavefront 算法通常包含三种同步：

- warp 内：相邻 lane 交换边界，可用 shuffle/warp barrier；
- block 内：tile 内多个局部 wavefront，用 shared memory 和 barrier；
- grid 间：tile 对角线之间用多个 kernel，或在满足驻留限制时使用 cooperative launch/-grid barrier。

应优先使用最小必要范围。全局同步越少越好，但不能牺牲依赖正确性。

14.8 负载与边界

方形 $N \times N$ 表的 wavefront 长度从 1 增至 N 再降至 1。小 wavefront 难以占满 GPU。常见改进：

- 批处理多个独立 DP 问题；
- 每 block 同时处理多个短序列；

- tile 化增加单 block 工作；
- 将长度相近的任务分组，减少负载不均；
- 对极长序列使用多 GPU 分区，但边界通信复杂。

14.9 正确性验证

DP 结果不仅检查最终最优值，还应：

- 对很小输入打印整个 DP 表，与 CPU 逐格比较；
- 测试空序列、长度 1、全匹配、全不匹配；
- 验证 tile 边界和最后非整 tile；
- 若需要回溯路径，确认方向信息与分数同步更新。

动手练习

1. 对 5×7 DP 表列出每条 wavefront 的坐标与长度；
2. 设计 2×2 tiled Floyd–Warshall 的三阶段执行顺序；
3. 为 Smith–Waterman 写出一个小规模 CPU 表，再映射到反对角线程；
4. 比较“每元素对角线 kernel”和“每 tile 对角线 kernel”的启动次数。

掌握检查

你应能从递推式画依赖箭头，找出无依赖前沿，选择元素级或 tile 级 wavefront，并解释坐标变换如何改善并行形状，而不是把动态规划简单判为“不能并行”。

14.10 由浅入深精讲：依赖图怎样变成 wavefront

从浅到深

第一层：先用：能画出动态规划单元格依赖。 **第二层：再理解**：能找出同一反对角线上可并行的单元格。 **第三层：最后优化**：能把单元格级 wavefront 提升为 tile 级 wavefront，并分析负载和同步。

14.11 先从二维依赖图开始

许多动态规划表满足

$$D_{i,j} = f(D_{i-1,j}, D_{i,j-1}, D_{i-1,j-1}).$$

单元格 (i, j) 依赖上、左、左上，因此不能按整行或整列全部并行。但满足 $i + j = k$ 的同一反对角线单元格互不依赖：它们只读取 $k - 1$ 或 $k - 2$ 的反对角线。

✂ 完整例题

对 4×4 表，wavefront 长度依次为 1, 2, 3, 4, 3, 2, 1。中间并行度高，开头和结尾并行度低。即使每条对角线一个 kernel，短 wavefront 的启动开销和低利用率也可能占明显比例。

14.12 单元格级 kernel 的局限

每条反对角线启动一个 kernel，正确性直观，因为 kernel 边界提供全局同步。但问题包括：

- kernel 数量随表尺寸增长；
- 前后 wavefront 太短，GPU 利用率低；
- 每个单元格反复访问 global memory；
- 对复杂递推，单线程工作过小。

因此实际常把表分成 tile，每个 block 负责一个 tile，在 tile 内使用 shared memory 和 block 同步。

14.13 tile 级 wavefront

若表被分成 $P \times Q$ 个 tile，tile (p, q) 依赖 $(p - 1, q)$ 、 $(p, q - 1)$ 和 $(p - 1, q - 1)$ ，仍可按 $p + q$ 的 tile 反对角线执行。一个 tile 内部又有局部 wavefront。这样：

- global 同步次数从单元格维度降到 tile 维度；
- tile 内边界数据可放 shared/register；
- 每个 block 工作量增加，启动开销被摊薄。

但 tile 太大时，单条 tile wavefront 上 block 数变少，外层并行度降低。

14.14 Floyd-Warshall 的阶段依赖

Floyd-Warshall 第 k 阶段更新

$$D_{i,j}^{(k)} = \min \left(D_{i,j}^{(k-1)}, D_{i,k}^{(k-1)} + D_{k,j}^{(k-1)} \right).$$

所有 (i, j) 在固定 k 下可并行，但阶段 $k+1$ 依赖阶段 k 。分块实现通常把含 pivot 的 tile、同一行/列 tile、其余 tile 分成不同阶段，以复用 pivot 数据并保证依赖。

14.15 Smith-Waterman 的边界数据

序列比对中每个 tile 只需从上方 tile 接收底边、从左方 tile 接收右边，并接收左上角值；tile 内部再沿反对角线推进。通信量主要与 tile 边长有关，计算量与 tile 面积有关，增大 tile 提高计算/边界比，但会降低并行 tile 数和增加片上存储。

14.16 超平面变换的直觉

原坐标中 wavefront 长度变化，使线程活跃数随阶段变化。通过坐标倾斜，把依赖方向转换为更规则的执行区域，可以让线程在更多阶段保持活跃并改善连续访问。理解重点不是记变换矩阵，而是：改变迭代空间的坐标表示，保持依赖方向为“过去”，同时让并行维度更规则。

分层实验

实验 1：画出 6×8 DP 表每条反对角线的长度。

实验 2：实现一对角线一 kernel 的简单版本，验证依赖。

实验 3：改为 16×16 tile，记录 kernel 数、global load 和时间。

实验 4：改变长宽比，观察 wavefront 最大并行度和尾部效应。

第 15 章 稀疏矩阵与图遍历：数据结构决定并行效率

■ 原书对应：原书第 17–18 章：COO、CSR、ELL、HYB、JDS、CSC 与 SpMV；BFS 的 vertex/edge-centric、frontier、私有化、cooperative groups 与其他优化。

◎ 本章学习目标

- 解释稀疏格式如何改变访存、负载均衡和 padding；
- 为 SpMV 在 COO、CSR、ELL 等格式间做选择；
- 理解 BFS 的 vertex-centric、edge-centric 与 frontier 方案；
- 处理不规则图中的负载不均、原子竞争和动态工作量。

15.1 为什么稀疏问题难

稠密矩阵的每行长度相同，索引规则；稀疏矩阵只存非零元素，节省计算与内存，却引入：

- 每行非零数不同，线程工作量不均；
- 列索引间接访问向量 x ，地址不规则；
- 格式元数据增加流量；
- 数据分布决定最佳映射，单一 kernel 很难适配全部矩阵。

SpMV 计算

$$y_i = \sum_{j:A_{i,j} \neq 0} A_{i,j} x_j.$$

每个非零仅做一次乘加，算术强度低，通常内存受限。

15.2 常见稀疏格式

格式	存储方式	并行特征	适用情况
----	------	------	------

COO	每个非零存 row、col、value	非零级并行简单；同一行输出需归约/原子	构建、转换、极不规则矩阵
CSR	rowPtr + colIdx + value	每行一线程/warp；行长不均导致负载差	通用行遍历、SpMV 常用
ELL	每行补到相同长度，二维规则数组	合并访问好、线程工作规则；padding 浪费	行长相近
HYB	主体用 ELL，长尾用 COO	规则主体高效，异常项单独处理	大多数行相近、少数很长
JDS	按行长排序后沿“锯齿对角”存储	减少发散，改善活跃线程	允许重排行并管理置换
CSC	按列压缩	便于列访问；SpMV scatter 可能竞争	列导向算法、转置相关操作

15.3 CSR SpMV 的映射选择

15.3.1 一线程一行

实现简单。短行有效，长行造成单线程长循环，行长差异导致 warp 内发散。

Listing 15.1 CSR 一线程一行 SpMV

```

1  __global__ void spmvCSR(const int* rowPtr, const int* col,
2                          const float* val, const float* x,
3                          float* y, int rows) {
4      int r = blockIdx.x * blockDim.x + threadIdx.x;
5      if (r >= rows) return;
6      float sum = 0.0f;
7      for (int p = rowPtr[r]; p < rowPtr[r + 1]; ++p)
8          sum += val[p] * x[col[p]];
9      y[r] = sum;
10 }
```

15.3.2 一 warp 一行

warp 各 lane 分担长行非零，再做 warp 归约。长行吞吐更好，短行会浪费 lane。可按行长分桶，为不同区间使用不同 kernel。

15.4 格式选择是算法的一部分

不能只比较“压缩率”。需要同时看：

- 行长均值、方差、最大值；
- x 访问的局部性；
- padding 与索引字节；
- 是否反复使用矩阵，能否摊销格式转换；
- 后续操作需要按行还是按列遍历；
- 是否允许行重排以及如何恢复结果顺序。

15.5 图与稀疏矩阵的联系

图的邻接表本质类似 CSR：每个顶点对应一行，邻居是非零列。图遍历的不规则性来自度分布和动态 frontier。

15.6 BFS 的层次语义

BFS 从源点开始，按距离层扩展。若 `level[v]` 未访问，发现它时设置为当前层 +1。多个父节点可能同时发现同一顶点，需要原子操作或等价机制确保只加入一次 frontier。

15.6.1 vertex-centric push

对当前 frontier 中每个顶点遍历出边，尝试发现邻居。frontier 小时工作高效；高出度顶点造成负载不均。

15.6.2 edge-centric

每轮扫描所有边，只处理源顶点在 frontier 的边。映射规则、负载均衡较好，但 frontier 小时做大量无用扫描。

15.6.3 pull

对未访问顶点检查其入邻居是否在 frontier。frontier 很大时可能更高效，因为找到一个父节点即可停止；需要适合的入边结构。

15.7 frontier 压缩

用显式 frontier 数组只启动当前活跃顶点。下一 frontier 的构建可以：

- 对每个新顶点原子追加；
- block/warp 私有缓冲后批量追加；
- 生成标志后用 scan 压缩；
- 使用 bitset 以节省空间并支持 pull 检查。

这再次复用了 histogram、filter 和 scan 模式。

Listing 15.2 frontier push BFS 的核心伪代码

```

1 int u = frontier[i];
2 for (int e = rowPtr[u]; e < rowPtr[u + 1]; ++e) {
3     int v = col[e];
4     if (atomicCAS(&level[v], UNVISITED, level[u] + 1) == UNVISITED) {
5         int pos = atomicAdd(nextCount, 1);
6         nextFrontier[pos] = v;
7     }
8 }

```

15.8 降低竞争与负载不均

- 高度顶点可由一个 warp、一个 block 或多个协作单元处理；
- 小度顶点可一个线程处理，避免浪费；
- block 私有 frontier 缓冲减少全局原子；
- direction-optimizing BFS 在 push/pull 间切换；
- 将多个 BFS 层放入一个 cooperative kernel 可减少启动开销，但要求 grid 能协作驻留并正确做全局 barrier。

15.9 图结构决定性能

道路网度低且规则，社交网络可能有少数超高出度顶点。必须用不同图族测试：

- 顶点数、边数和平均度；
- 度分布与最大度；

- 直径和每层 frontier 大小；
- 连通性与重复边；
- 邻接排序是否带来局部性。

只用一个随机图得出的结论不能泛化。

动手练习

1. 给定每行非零数 [1, 1, 2, 2, 20]，比较一线程一行和一 warp 一行；
2. 把一个小矩阵手工转换为 COO、CSR、ELL，计算各格式存储量；
3. 对链图、星形图、完全图手算 BFS frontier 大小；
4. 设计按顶点度分桶的 BFS：小度一线程，中度一 warp，高度一 block。

掌握检查

你能说明稀疏格式不是单纯压缩，而是在空间、规则性、访存和负载间权衡；也能根据 frontier 大小和度分布选择 BFS 的 push、pull 或 edge-centric 方案。

15.10 由浅入深精讲：不规则数据为什么先要选表示

从浅到深

第一层：先会用：能读懂 COO、CSR、ELL 的数组。 **第二层：再理解：**能根据每行非零数分析负载与访存。 **第三层：最后优化：**能把格式选择、线程粒度和图遍历方向作为同一个算法决策。

15.11 用一个小矩阵理解 CSR

矩阵

$$A = \begin{bmatrix} 5 & 0 & 2 & 0 \\ 0 & 3 & 0 & 0 \\ 4 & 0 & 0 & 7 \end{bmatrix}$$

CSR 可表示为

$$values = [5, 2, 3, 4, 7], \quad col = [0, 2, 1, 0, 3], \quad rowPtr = [0, 2, 3, 5].$$

第 r 行的非零元素位于区间

$$[rowPtr[r], rowPtr[r + 1]).$$

SpMV $y = Ax$ 中，一线程一行可写为遍历该区间并累加 `values[k]*x[col[k]]`。

15.12 一线程一行的负载不均

若每行非零数差异很大，warp 中短行线程很快结束，长行线程继续工作，产生发散和低利用率。若平均每行只有少量非零，一线程一行简单且有效；若存在长行，可让一个 warp 或 block 合作处理一行，并用归约合并部分和。

✂ 完整例题

一个 warp 处理 32 行，其中 31 行各 2 个非零，1 行有 1000 个非零。一线程一行时，31 个 lane 很快空闲，只剩 1 个 lane 串行处理大行。改为一 warp 一行可让 32 lane 分摊长行，但对短行会浪费 lane。因此可按行长度分组，采用混合策略。

15.13 格式是性能算法的一部分

- **COO**: 每个非零显式存行列，简单但输出行冲突可能需要原子；
- **CSR**: 行遍历紧凑，适合按行 SpMV，但行长度不规则；
- **ELL**: 每行 padding 到固定长度，访问规则、易合并，但长尾导致空间浪费；
- **HYB**: 主体用 ELL，少量异常长元素用 COO，平衡规则性和 padding；
- **JDS**: 按行长度重排，减少 warp 内不均，但增加重排和索引复杂度；
- **CSC**: 按列访问方便，适合列向遍历或转置相关操作。

选择格式时同时考虑矩阵结构、操作类型、重复使用次数和转换成本。

15.14 BFS 与稀疏矩阵的联系

图的邻接表类似 CSR: `rowPtr[v]` 到 `rowPtr[v+1]` 是顶点 v 的邻边。BFS 每层处理 frontier，并产生下一层 frontier。不同顶点度数就是不同“行长度”，因此遇到与 SpMV 相同的负载不均和不规则访问。

15.15 push、pull 与 edge-centric

15.15.1 push

从当前 frontier 顶点遍历出边，尝试发现邻居。frontier 小时只访问少量边，高效；但多个父节点可能竞争更新同一邻居，需要原子或幂等标记。

15.15.2 pull

对每个未访问顶点检查入邻居，只要发现一个在 frontier 即停止。frontier 很大时可减少重复写竞争，但会扫描大量未访问顶点和边。

15.15.3 edge-centric

并行扫描所有边，映射规则、负载均匀，但在 frontier 小时做大量无效工作。

方向优化 BFS 会根据 frontier 大小、未访问顶点数和边数在 push/pull 间切换。这说明高性能图算法不能只固定一种映射。

15.16 frontier 压缩是 filter/scan 的应用

线程发现新顶点后，需要构建紧凑的下一 frontier。直接 global atomic append 简单但竞争；可先在 warp/block 私有缓冲中聚合，再一次申请连续区间；或者生成布尔标记，通过 scan/filter 压缩。这复用了前面直方图、过滤和扫描的模式。

15.17 局部性与编号重排

图邻居编号随机时，读取顶点属性呈散乱访问。重新编号可让经常相邻访问的顶点靠近，提高 cache 和事务利用率；但重排有预处理成本，并可能改变其他操作的局部性。对于重复运行的图分析，预处理更容易摊薄。

分层实验

实验 1：CSR 手算。把一个 5×6 稀疏矩阵转换为 CSR，并手算一行 SpMV。

实验 2：负载分布。生成均匀行长和幂律行长矩阵，比较一线程一行与一 warp 一行。

实验 3：BFS。对网格图和幂律图比较 push、edge-centric；记录每层 frontier 和访问边数。

实验 4：frontier 构建。比较 atomic append 与 block 聚合。

第 16 章 深度学习与大语言模型：卷积、注意力、KV cache 与 FlashAttention

📖 **原书对应：**原书第 19–20 章及附录 B：CNN 基础、CUDA 卷积层、卷积转 GEMM、cuDNN；Transformer、多头注意力、CUDA attention、KV cache、FlashAttention、内存需求与批处理。

🎯 本章学习目标

- 把神经网络层还原为矩阵乘、卷积、归约和逐元素操作；
- 理解卷积层直接实现与 GEMM 表述的关系；
- 推导 scaled dot-product attention 的形状和计算步骤；
- 解释 KV cache 节省了什么计算、增加了什么内存；
- 理解 FlashAttention 为何通过 IO-aware tiling 减少中间矩阵流量。

16.1 深度学习层本质上是并行模式组合

常见层可拆成：

- 全连接：矩阵乘 + bias + activation；
- 卷积：邻域加权和，可直接卷积或转换为 GEMM；
- normalization：归约（均值/方差）+ 逐元素变换；
- softmax：最大值归约、指数、求和归约、归一化；
- attention：多个 GEMM + softmax + 数据重排；
- embedding：索引 gather；训练时梯度可能需要 scatter/atomic。

因此前面学习的 tiling、归约、scan、合并访存和批处理会在 AI 工作负载中重复出现。

16.2 CNN 卷积层

输入可记为 $X[N, C_{in}, H, W]$ ，卷积核 $K[C_{out}, C_{in}, R, S]$ ，输出

$$Y[n, c_o, h, w] = \sum_{c_i, r, s} X[n, c_i, h + r, w + s] K[c_o, c_i, r, s],$$

这里省略 stride、padding 和 dilation 的详细下标。并行维度包括 batch、输出通道和输出空间位置；归约维度是输入通道与核空间。

16.2.1 直接卷积

一个线程或线程组负责一个/多个输出元素，使用 shared memory 复用输入 tile 和 filter。优点是避免显式展开中间矩阵；难点是布局、边界、通道归约和不同卷积形状的适配。

16.2.2 卷积转 GEMM

把每个输入窗口展平为矩阵一行/列，把 filter 展平为另一矩阵，即可用 GEMM：

$$Y_{\text{matrix}} = X_{\text{patch}} K_{\text{matrix}}.$$

显式 im2col 会复制重叠输入，增加内存；隐式 GEMM 在 kernel 内按需生成逻辑 patch 索引，复用高性能矩阵乘框架。

16.3 为什么使用深度学习库

成熟库会根据形状、数据类型、硬件和 workspace 选择算法。自己写卷积主要用于学习、特殊融合或研究。生产比较时要注意：

- 数据布局 NCHW/NHWC；
- 是否包含 bias、activation、padding；
- 精度与累加类型；
- workspace 上限；
- warm-up 和自动调优是否完成。

16.4 Transformer 的核心数据流

设序列长度 L 、模型维度 d 、头数 h 、每头维度 $d_h = d/h$ 。输入 $X \in \mathbb{R}^{L \times d}$ 通过线性投影得到

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

每个 attention head 计算

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V,$$

其中 M 可表示 causal mask 或其他屏蔽。

16.4.1 形状检查

对单头：

$$Q, K, V \in \mathbb{R}^{L \times d_h}, \quad S = QK^\top \in \mathbb{R}^{L \times L}, \quad P = \text{softmax}(S) \in \mathbb{R}^{L \times L}, \quad O = PV \in \mathbb{R}^{L \times d_h}.$$

$L \times L$ 的 score/probability 矩阵是标准实现的主要内存压力之一。

16.5 朴素 attention 的多 kernel 实现

一个直观实现分为：

1. GEMM 计算 $S = QK^\top$;
2. scale、mask;
3. 每行 softmax: max reduction、指数、sum reduction、归一化;
4. GEMM 计算 $O = PV$ 。

每一步单独写回 global memory，会多次读写 L^2 中间矩阵。即使 GEMM 很快，IO 和 kernel 启动也可能成为瓶颈。

16.6 稳定 softmax

直接计算 e^{s_i} 可能溢出。使用

$$p_i = \frac{e^{s_i - m}}{\sum_j e^{s_j - m}}, \quad m = \max_j s_j.$$

softmax kernel 需要两次归约或融合的在线算法。每行可映射到一个 block/warp，具体取决于行长。

16.7 自回归解码与 KV cache

生成第 t 个 token 时，新的 query 只对应当前 token，但需要与所有历史 key/value 做 attention。若每步都重新计算历史 token 的 K, V ，会重复大量线性投影。KV cache 保存历史层、历史 token 的 key/value，只为新 token 追加一项。

对每层每 token，每个 batch 元素的 KV 数据量近似为

$$\text{bytes/token/layer} = 2 \times h \times d_h \times b = 2db,$$

其中 2 对应 K 与 V， b 是每元素字节数。总 KV cache 近似

$$M_{KV} = 2 B L N_{\text{layer}} d b,$$

若采用 grouped-query/multi-query attention，K/V 头数小于 query 头数，公式相应替换为 KV 的实际头数。

💡 直觉

KV cache 是“用内存换计算”。它避免重复生成历史 K/V，却让解码越来越依赖读取不断增长的 cache。prefill 往往计算密集，单 token decode 更容易受内存带宽和调度开销限制。

16.8 KV cache 布局

布局影响相邻线程是否读取连续地址。需要考虑维度顺序：layer、batch、KV head、token、head dimension。连续访问通常希望 lane 沿 head dimension 或适合向量化的维度展开。动态请求长度还带来：

- 预分配浪费；
- 内存碎片；
- 请求结束后的回收；
- batch 中长度不同；
- beam search 或共享前缀的复制/共享。

分页或块化 KV 管理是系统层对这些问题的解决思路，本教程只要求理解其动机。

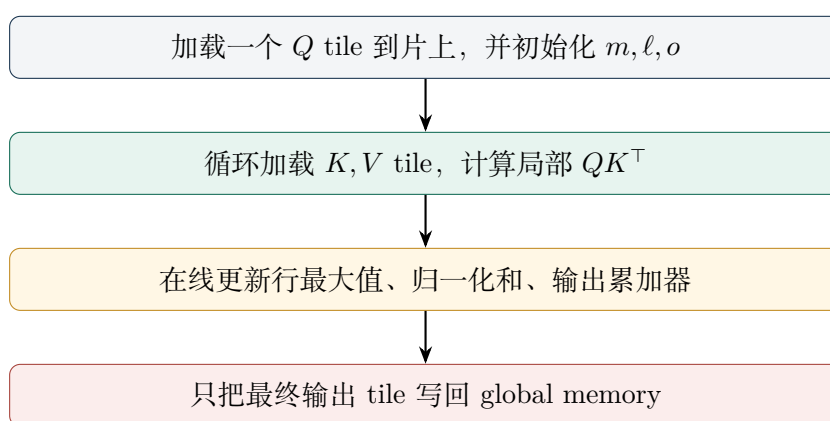
16.9 FlashAttention 的核心：不物化完整 L^2 矩阵

FlashAttention 将 Q, K, V 分成块，在片上内存中计算 score tile、在线更新 softmax 统计量和输出累加器，不把完整 S 或 P 写入 global memory。

关键难点是 softmax 跨多个 K tile。对某一 query 行，维护当前最大值 m 、归一化和 ℓ 、未归一化输出累加 o 。处理新 score block s ：

$$\begin{aligned} m' &= \max(m, \max s), \\ \ell' &= e^{m-m'} \ell + \sum_j e^{s_j-m'}, \\ o' &= e^{m-m'} o + \sum_j e^{s_j-m'} v_j. \end{aligned}$$

所有 tile 处理完后输出 o/ℓ 。缩放旧统计量保证数值等价于一次性稳定 softmax。



它的关键收益不是减少理论 FLOP，而是大幅减少对 L^2 中间矩阵的 global memory IO，并融合多个 kernel。

16.10 causal mask 与不规则 tile

自回归 attention 中，第 i 个 query 只能访问 $j \leq i$ 的 key。对完全位于对角线右上的 tile 可直接跳过；与对角线相交的 tile 需要逐元素 mask，可能出现分支或谓词开销。良好的 tile 调度会避免无效计算并保持连续访存。

16.11 attention 的算术强度与序列长度

标准全序列 attention 的 QK^T 和 PV 计算随 $L^2 d_h$ 增长；中间数据也随 L^2 增长。FlashAttention 减少中间 IO，使计算单元更有机会成为瓶颈。解码阶段只有一个或少数 query，读取历史 KV 的字节随上下文长度线性增长，而每个 KV 元素参与的计算较少，算术强度偏低。

16.12 batching: 吞吐与延迟的交换

批处理可以增大 GEMM、提高硬件利用率和吞吐，但请求需要等待组 batch，单请求延迟可能增加。动态请求中还要处理不同序列长度：

- padding 造成无效计算；
- 按长度分组降低 padding，但调度更复杂；
- continuous batching 在有请求结束/到达时动态替换；
- prefill 和 decode 的资源特征不同，混合调度需要权衡。

16.13 融合与数值精度

AI kernel 常融合 bias、activation、residual、normalization，以减少中间写回和启动。低精度提高吞吐、降低内存，但要关注：

- 输入/权重类型与累加类型；
- softmax、归约等敏感操作是否提升精度；
- 溢出、下溢和量化尺度；
- 融合后是否改变舍入顺序与结果容差。

动手练习

1. 对 $L = 1024, d = 4096, h = 32$ 写出每头 Q, K, V, S, O 形状；
2. 根据给定层数、batch、上下文和数据类型计算 KV cache 大小；
3. 手算在线 softmax 合并两个 score block，验证与一次性 softmax 一致；
4. 比较朴素 attention 与 tiled fused attention 需要写回哪些中间数组；
5. 对 prefill 和单 token decode 分别判断更可能计算受限还是内存受限。

掌握检查

你应能从公式和张量形状解释 attention 的两个 GEMM 与 softmax；说明 KV cache 是用内存换去历史投影重算；说明 FlashAttention 的本质是在线 softmax + tiling + 避免 L^2 中间 IO，而不是“近似 attention”。

16.14 由浅入深精讲：把 Transformer 拆回熟悉的并行模式

从浅到深

第一层：先会用：会检查张量形状并写出 attention 公式。 **第二层：再理解：**能把 attention 拆成 GEMM、归约、逐元素和布局变换。 **第三层：最后优化：**能定量分析 KV cache、FlashAttention、prefill 与 decode 的计算和内存特征。

16.15 从一个具体形状走完整个 attention

设 batch $B = 2$ 、序列长度 $L = 128$ 、模型维度 $d = 512$ 、头数 $h = 8$ ，则每头维度 $d_h = 64$ 。输入

$$X \in \mathbb{R}^{2 \times 128 \times 512}.$$

线性投影后，重排为

$$Q, K, V \in \mathbb{R}^{2 \times 8 \times 128 \times 64}.$$

对每个 batch 和 head：

$$\begin{aligned} S &= QK^T \in \mathbb{R}^{128 \times 128}, \\ P &= \text{softmax}(S/8 + M), \\ O &= PV \in \mathbb{R}^{128 \times 64}. \end{aligned}$$

所有 head 拼接回 $2 \times 128 \times 512$ ，再经过输出投影。

16.15.1 计算量的数量级

单 head 的 QK^T 约 $2L^2d_h$ FLOP， PV 也约 $2L^2d_h$ ，全部 head 与 batch 约

$$F_{\text{attn}} \approx 4BhL^2d_h = 4BL^2d.$$

当 L 翻倍，attention 主体计算约增 4 倍；这也是长上下文成本快速增长的根源。

16.16 softmax 为什么必须先减最大值

对一行 score s_i ：

$$p_i = \frac{e^{s_i}}{\sum_j e^{s_j}}.$$

若 s_i 很大，指数可能溢出。减去任意常数不改变结果：

$$\frac{e^{s_i-c}}{\sum_j e^{s_j-c}} = \frac{e^{-c} e^{s_i}}{e^{-c} \sum_j e^{s_j}}.$$

取 $c = m = \max s$ 后，所有指数输入不大于 0，提高数值稳定性。并行 softmax 的步骤是行最大值归约、指数与求和归约、归一化；这直接复用 reduction 模式。

16.17 朴素实现的 IO 账本

若显式物化 S 和 P ，对每个 head 要写、读 L^2 规模的中间矩阵多次：

1. GEMM 写 S ;
2. scale/mask 读写 S ;
3. softmax 多次读写行数据并写 P ;
4. 第二个 GEMM 读 P 。

当 L 很大，中间 IO 和 kernel 启动可能显著。FlashAttention 的核心不是改变 attention 数学定义，而是重排计算顺序，让 score tile 在片上生成、归一化并立即作用到 V ，不把完整 S, P 写回显存。

16.18 在线 softmax 合并的详细推导

已处理一部分 score，维护最大值 m 、指数和 ℓ 、加权值和 o ：

$$\ell = \sum_{j \in \text{old}} e^{s_j-m}, \quad o = \sum_{j \in \text{old}} e^{s_j-m} v_j.$$

新块最大值为 m_b ，新全局最大值 $m' = \max(m, m_b)$ 。旧项要从基准 m 改到 m' ：

$$e^{s_j-m'} = e^{m-m'} e^{s_j-m}.$$

因此

$$\begin{aligned} \ell' &= e^{m-m'} \ell + \sum_{j \in \text{block}} e^{s_j-m'}, \\ o' &= e^{m-m'} o + \sum_{j \in \text{block}} e^{s_j-m'} v_j. \end{aligned}$$

最终 o/ℓ 即 softmax 加权输出。这个递推允许按 K/V tile 流式处理，同时保持与稳定 softmax 等价。

完整例题

旧块 score $[1, 2]$, 新块 $[3]$ 。旧 $m = 2$, $\ell = e^{-1} + 1$ 。新 $m' = 3$, 旧和缩放 $e^{-1}(e^{-1} + 1) = e^{-2} + e^{-1}$, 再加新项 1, 得到 $e^{-2} + e^{-1} + 1$, 正是所有 score 相对最大值 3 的指数和。

16.19 KV cache: 按实际 KV 头数计算

自回归解码中, 每层缓存历史 K、V。设 batch B 、上下文长度 L 、层数 N_l 、KV 头数 h_{kv} 、每头维度 d_h 、每元素 b 字节:

$$M_{KV} = 2BLN_l h_{kv} d_h b.$$

注意不是总 query 头数一定等于 KV 头数。MHA 中 $h_{kv} = h$; GQA/MQA 中更少。

完整例题

$B = 1, L = 8192, N_l = 32, h_{kv} = 8, d_h = 128$, FP16/BF16 每元素 2 B:

$$M_{KV} = 2 \times 1 \times 8192 \times 32 \times 8 \times 128 \times 2 = 1,073,741,824 \text{ B} \approx 1 \text{ GiB}.$$

若 $h_{kv} = 32$, 则约 4 GiB。长上下文和并发 batch 会线性放大显存需求。

16.20 prefill 与 decode 是两类不同 kernel 形态

16.20.1 prefill

一次处理整个提示, Q 数量为 L , attention 具有大矩阵乘, 计算量高, 较容易利用 Tensor Core, 通常更偏计算密集。

16.20.2 decode

每步只有一个或少数新 query, 却要读取所有历史 K/V。对每层每步, 读取 KV 随上下文长度线性增长, 而输出 token 数很少, 矩阵形状瘦长, 算术强度和批处理效率较低, 容易受显存带宽、KV 布局 and 调度开销限制。

16.21 KV cache 布局的地址问题

常见维度包括 batch、KV head、token、head dimension。若 warp lane 沿 head dimension 读取, 可获得连续向量; 若请求长度不同、cache 分页, 逻辑连续 token 可能分布在不同物理块, 需要页表/块表间接寻址。布局设计要在:

- 连续读取和向量化；
- 动态增长与内存碎片；
- 多请求批处理；
- beam/shared prefix；
- 写入新 token 与读取历史 token

之间权衡。

16.22 causal mask 与 tile 跳过

对 query 位置 i ，只允许 key 位置 $j \leq i$ 。FlashAttention 处理 score tile 时：

- 完全在对角线左下的 tile 全部有效；
- 完全在右上的 tile 可直接跳过；
- 穿过对角线的 tile 需要逐元素 mask。

避免右上无效 tile 可减少计算；对角 tile 的条件处理则可能降低规则性。

16.23 batching 的定量视角

增大 batch 能把多个小 GEMM 合并为更大工作，提高吞吐，但请求可能等待凑批。对在线服务，常区分：

- **首 token 延迟**：主要受 prefill 和排队影响；
- **每 token 延迟**：主要受 decode、KV 读取和调度影响；
- **tokens/s**：系统总吞吐；
- **并发容量**：受权重、KV cache 和临时 workspace 限制。

原书强调批处理的延迟-吞吐交换，理解这一点比只看单 kernel TFLOP/s 更接近实际 LLM 推理。

16.24 与前面并行模式的对应

LLM 操作	已学习的模式与优化
QKV 与输出投影	GEMM、寄存器分块、专用矩阵硬件
逐行 softmax	最大值归约、求和归约与逐元素运算
LayerNorm / RMSNorm	归约与逐元素融合
FlashAttention	分块、在线归约、双缓冲与算子融合
KV cache 访问	合并访问、向量化加载、数据布局与分页
MoE token 路由	直方图、扫描、过滤与散射
多 GPU 张量并行	集合通信与计算-通信重叠

分层实验

实验 1：形状。对任意 B, L, d, h, h_{kv} 写出每个中间张量形状。

实验 2：softmax。实现一行一个 block 的稳定 softmax，分步比较最大值和求和。

实验 3：在线合并。用 CPU 小数组验证分块在线 softmax 与一次性结果。

实验 4：KV 估算。对多组模型和上下文计算 cache，说明 GQA 的影响。

实验 5：IO。列出朴素 attention 与 tiled fused attention 的中间读写数组，定性比较显存流量。

第 17 章 问题表述与算法选择：从静电势到通用计算思维

■ 原书对应：原书第 21–22 章：静电势图中的 scatter/gather、线程粗化、合并访存、cutoff binning；算法选择、问题分解、Amdahl 定律、问题表述与 batching。

◎ 本章学习目标

- 区分 input-centric scatter 与 output-centric gather；
- 理解算法复杂度、精度和并行效率的三方权衡；
- 使用“表述 - 选择 - 分解 - 优化”流程设计 GPU 算法；
- 把 batching 看成延迟与吞吐之间的系统决策。

17.1 案例：静电势图

给定空间中多个带电粒子，在网格点 r 计算势能可抽象为

$$V(r) = \sum_{a=1}^N \frac{q_a}{\|r - r_a\|}.$$

有两个自然并行表述。

17.1.1 input-centric scatter

一个线程处理一个粒子，把贡献散射到许多网格点。优点是粒子数据局部；缺点是多个粒子同时更新同一网格点，需要原子操作，写入分散且竞争重。

17.1.2 output-centric gather

一个线程处理一个网格点，遍历所有粒子并累加。每个线程独占输出，不需要原子，控制较规则；但粒子数据被所有网格线程重复读取，可依赖 constant/cache、tiling 和线程粗化复用。

表述	优势	风险
scatter (输入中心)	输入工作自然分配, 适合每输入影响少量输出	输出竞争、随机写、原子
gather (输出中心)	每线程独占输出, 易合并写、无竞争	重复读取输入, 可能做无效贡献

选择不是固定的。若每个输入只影响少量输出且冲突可控, scatter 可能好; 若输出规则且可高效筛选输入, gather 常更适合 GPU。

17.2 cutoff: 用可控误差换复杂度

直接计算所有粒子与所有网格点是 $O(NG)$ 。若物理贡献随距离衰减, 可只显式计算半径 R_c 内粒子, 远场用近似或忽略。需要空间 binning:

1. 将粒子按空间单元分箱;
2. 每个网格点只遍历附近 bins;
3. 处理 bin 边界和实际距离检查;
4. 评估 cutoff 引入的误差。

这降低渐近工作量, 却增加预处理、索引和负载不均。优化不能只看 kernel 时间, 还要包括 bin 构建与数据重排。

17.3 线程粗化与合并访存

一个线程可计算一系列或多个相邻网格点, 复用加载的粒子坐标与电荷。若线程沿内存连续的 x 方向负责相邻输出, 写回合并; 但每线程累加器增加, 可能降低 occupancy。选择粗化方向要与行主序布局一致。

17.4 通用问题求解流程

17.4.1 第一步: 问题表述

明确输入、输出、精度、边界、依赖和性能目标。问: 同一数学任务是否可以换一个输出定义、坐标、数据结构或计算顺序?

17.4.2 第二步：算法选择

比较复杂度、常数、精度、稳定性和并行形状。例如精确 $O(NG)$ 与 cutoff 近似；比较排序与哈希；直接卷积与 GEMM。

17.4.3 第三步：问题分解

选择任务粒度和线程到数据映射：

- input-centric / output-centric；
- 一线程一元素 / 一 warp 一行 / 一 block 一 tile；
- 静态均分 / 动态队列 / 按特征分桶；
- 单 GPU / 域分解多 GPU。

17.4.4 第四步：优化

先优化主瓶颈：减少工作、减少 IO、提高局部性、降低同步、平衡负载，再考虑微观指令优化。

17.5 将 Amdahl 定律放回应用层

假设 GPU kernel 已经极快，但输入解析、数据传输或 CPU 后处理占比高，整体仍受限制。应用级优化应考虑：

- 是否把更多流水线移到 GPU；
- 是否让数据长期驻留设备；
- 是否重叠 I/O、传输与计算；
- 是否通过 batching 摊薄固定开销；
- 是否需要改变接口以避免格式转换。

17.6 batching：不只是把维度前面加一个 B

批处理把多个独立任务一起执行，增大并行度并摊薄启动/传输开销。但代价是等待组批、内存增加和长短任务互相拖累。

吞吐与延迟

吞吐是单位时间完成多少任务；**延迟**是单个任务从到达完成经历多久。增大 batch 通常改善吞吐，但可能增加排队延迟。系统目标必须明确优先级和服务约束。

17.7 性能目标要可验收

一个完整设计文档应写：

- 正确性：参考实现、误差容限、边界案例；
- 性能：基线、输入族、计时范围、目标吞吐/延迟；
- 资源：显存上限、临时 workspace、可用 GPU 数；
- 可扩展性：输入增大、硬件变化、batch 变化；
- 失败模式：OOM、极端数据分布、数值异常、死锁。

17.8 决策示例：图像模糊

同一任务可以有多种表述：

- 直接二维卷积：简单， K^2 计算；
- 可分离卷积：若核可分离，变为两次 K 计算；
- 频域卷积：大核时可能有利，但包含 FFT 和复数乘；
- 多尺度近似：允许误差时可降低成本。

最佳方案取决于核大小、图像批量、精度、是否重复使用同一核和端到端流水线。

动手练习

选择一个你熟悉的问题，写一页设计：输入输出、顺序基线、可选算法、线程映射、数据布局、同步、预期瓶颈、验证方法。要求至少比较 input-centric 与 output-centric 两种表述。

掌握检查

你应能把“写 kernel”推迟到问题表述和算法选择之后。面对新任务，先问是否能减少总工作、改变数据结构或换分解，再使用底层优化清单。

17.9 由浅入深精讲：先改变问题，再优化 kernel

从浅到深

第一层：先会用：从数学定义写出输入和输出。 **第二层：再理解：**比较 input-centric 与 output-centric 分解。 **第三层：最后优化：**把算法复杂度、内存流量、同步和误差容忍共同纳入选择。

17.10 scatter 与 gather 的正确性和性能差异

假设每个输入粒子影响多个网格点。

- **scatter/input-centric:** 一个线程负责输入粒子，把贡献写到多个输出；并行输入容易，但不同粒子可能更新同一输出，需要 atomic。
- **gather/output-centric:** 一个线程负责一个输出网格点，遍历可能影响它的输入；写入无冲突，但可能扫描大量无关输入。

二者数学上可等价，性能却由输入/输出规模、影响范围、空间索引和冲突程度决定。

完整例题

有 N 个粒子、 G 个网格点，每粒子平均影响 k 点。scatter 工作约 Nk ，但有竞争；朴素 gather 工作约 GN ，通常不可接受。若通过空间 binning，让每个输出只检查附近平均 q 个粒子，gather 可降到 Gq 并避免 atomic。算法级索引结构可能比任何 kernel 微优化更重要。

17.11 cutoff 与 binning 的推导思路

若物理贡献随距离衰减，可设置 cutoff 半径 r_c 忽略远处贡献，把全对全计算变为局部邻域。再把空间划分为边长与 r_c 相关的 bin，每个输出只遍历相邻 bin。需要明确：

- 截断误差如何验证；
- bin 中元素数量是否均衡；
- 建 bin 的预处理成本能否被重复计算摊薄；
- 邻域访问是否连续，数据布局是否需要重排。

17.12 通用决策清单

对一个新问题依次回答：

1. 数学输出是什么，允许多大误差？
2. 是否存在更低复杂度算法或可利用的稀疏性/局部性？
3. 并行单位选输入、输出、边、tile 还是任务队列？
4. 每个单位的工作量分布如何，是否需要分组或动态调度？
5. 主要字节流量、FLOP、atomic 和 barrier 各是多少？
6. 数据是否能长时间驻留 GPU，多个阶段能否融合？
7. 规模变化后瓶颈是否会改变？

只有完成这一步，才进入 block 大小、unroll 等局部参数调优。

分层实验

选一个你熟悉的 CPU 算法，分别写出 input-centric 和 output-centric 映射，列出每种方案的读写位置、冲突、工作量和可能的数据结构。不要写 CUDA 代码，先完成一页设计表。

第 18 章 多 GPU 编程：域分解、halo exchange 与通信重叠

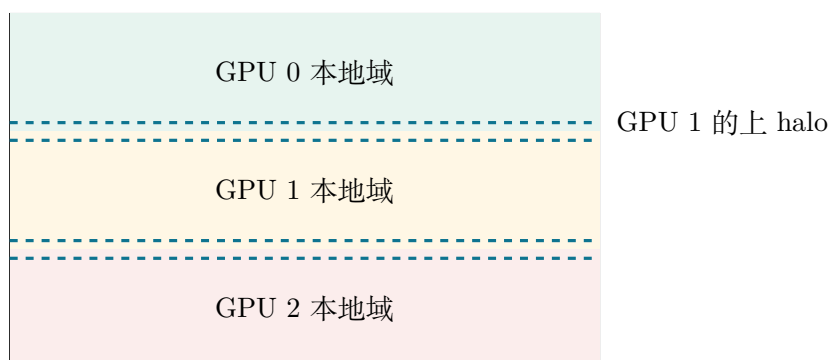
■ **原书对应：**原书第 23 章：以 stencil 为例的 MPI、计算通信重叠、NCCL 与 NVSHMEM 多 GPU 编程。

◎ 本章学习目标

- 将全局数据按 GPU 域分解，并识别 halo 数据；
- 区分 MPI、NCCL、NVSHMEM 的编程模型；
- 设计“内部计算 - halo 通信 - 边界计算”的重叠流程；
- 评估强扩展、弱扩展、通信量和负载均衡。

18.1 从单 GPU 到多 GPU 的第一步：分数据

以二维 stencil 为例，矩阵按行主序存储，最自然的是沿 y 方向把连续行分给不同 GPU。每个 GPU 拥有本地内部行，但更新边界行需要相邻 GPU 的一行旧值，这些额外数据是 halo/ghost cells。



域分解应兼顾：

- 每 GPU 计算量均衡；
- 本地数据尽量连续；
- 分区表面积小，halo 通信少；
- 设备间拓扑和带宽；

- 最小分区仍有足够工作填满 GPU。

18.2 通信模型一：CUDA-aware MPI

MPI 面向分布式进程和显式消息传递。CUDA-aware 实现可直接接受设备指针，避免用户手工 staging 到 host（实际路径取决于系统）。典型每进程控制一个 GPU：

Listing 18.1 MPI halo exchange 的结构伪代码

```

1 // post receives into device halo buffers
2 MPI_Irecv(topHalo_d, count, ..., up, tag, comm, &req[0]);
3 MPI_Irecv(botHalo_d, count, ..., down, tag, comm, &req[1]);
4 // send boundary rows from device memory
5 MPI_Isend(firstRow_d, count, ..., up, tag, comm, &req[2]);
6 MPI_Isend(lastRow_d, count, ..., down, tag, comm, &req[3]);
7
8 computeInterior<<<...>>>(...); // does not need halo
9 MPI_Waitall(4, req, status);
10 computeBoundary<<<...>>>(...); // consumes received halo

```

MPI 优势是跨节点通用、点到点和集体操作丰富；代价是进程、通信配对、设备选择和同步管理较显式。

18.3 通信模型二：NCCL

NCCL 面向 GPU 集体通信和点到点通信，常用于 all-reduce、all-gather、reduce-scatter、broadcast 等。它了解 GPU/互连拓扑并对大数据传输优化。

stencil halo 可使用 NCCL send/recv 组合；训练中更典型的是梯度 all-reduce 或参数/激活的集体通信。调用常在 CUDA stream 上排队，因此需要用 stream 依赖保证“发送前边界已算好”“使用前接收已完成”。

18.4 通信模型三：NVSHMEM

NVSHMEM 提供对称内存和 one-sided put/get。GPU kernel 可直接发起远端访问或通信，适合更细粒度、设备驱动的模式。编程者需要理解：

- 对称对象在各 processing element 上的分配；
- 远端地址如何对应；
- put/get 完成语义与 barrier/fence/quiet；

- 远端访问粒度与拓扑性能。

它减少 host 参与，但正确性和性能调试也更接近分布式共享内存思维。

18.5 计算与通信重叠

把本地输出分成内部和边界：

1. 上一轮结果就绪后，异步发送边界并接收 halo；
2. 在独立 stream/执行路径计算不依赖 halo 的内部；
3. 通信完成后计算边界；
4. 交换 old/new buffer，进入下一轮。

理想单步时间从

$$T_{\text{comm}} + T_{\text{interior}} + T_{\text{boundary}}$$

变为近似

$$\max(T_{\text{comm}}, T_{\text{interior}}) + T_{\text{boundary}}.$$

前提是通信和计算资源可真正并行，内部计算足够长，并且没有多余全局同步。

⚠ 易错点

“使用非阻塞 API” 不等于已经重叠。若紧接着立即 wait，或默认 stream 的依赖把所有工作串行化，时间线仍不会重叠。必须用 profiler 查看 copy/communication 与 kernel 是否在时间轴上并行。

18.6 多 GPU 正确性

常见错误包括：

- 上下邻居 rank/GPU 写反；
- 发送新数组、接收后却读取旧数组；
- halo 宽度不足以支持 stencil 半径；
- 边界 kernel 在通信完成前启动；
- 最后一分区尺寸不同但仍使用固定 count；
- 多进程错误地选择同一 GPU；
- collective 调用顺序在不同 rank 不一致导致挂起。

验证时先用 2 GPU、小网格、少数迭代，打印每个分区和 halo，与单 GPU 结果逐点比较。

18.7 扩展性指标

18.7.1 强扩展

固定总问题规模，增加 GPU。理想时间按 $1/P$ 降低，但每 GPU 计算变少、通信占比增大。效率

$$E_{\text{strong}}(P) = \frac{T_1}{PT_P}.$$

18.7.2 弱扩展

每 GPU 保持固定本地问题规模，总规模随 P 增长。理想时间近似不变。适合评估系统能否处理更大问题。

18.7.3 表面积/体积比

d 维规则域中，计算量随体积增长，halo 随表面积增长。分区越小，通信/计算比通常越高。选择分解方向应尽量减少跨 GPU 边界。

18.8 拓扑与通信路径

设备可能通过 PCIe、NVLink、交换结构或跨节点网络连接。相同代码在不同拓扑上通信成本差异很大。多 GPU 程序应查询拓扑、绑定进程/NUMA、选择合适 collective 算法，并分别报告单节点和跨节点结果。

18.9 MPI、NCCL、NVSHMEM 如何选

模型	强项	典型使用
MPI	跨节点通用消息传递、成熟生态	科学计算域分解、复杂点到点通信
NCCL	GPU 集体通信和拓扑优化	分布式训练、规则 collective、GPU P2P
NVSHMEM	GPU 发起 one-sided 远端访问	细粒度设备驱动通信、不规则共享数据

实际应用可组合使用，例如 MPI 管理进程与节点，NCCL 做 GPU collective。

动手练习

1. 将 1000×800 网格沿行分到 3 个 GPU，给出每个分区行范围和 halo；
2. 画出一轮 stencil 的通信/内部/边界时间线；
3. 分析 stencil 半径从 1 增到 3 时 halo 流量如何变化；
4. 设计强扩展实验，要求报告计算时间、通信时间、重叠比例和效率。

掌握检查

你应能从单 GPU kernel 上升到“分区 + halo + 通信 + 重叠 + 扩展性”的完整模型，并能说明 MPI、NCCL 和 NVSHMEM 各自提供的抽象，而不是把多 GPU 理解为简单启动多个 kernel。

18.10 由浅入深精讲：多 GPU 的核心是数据所有权与通信

从浅到深

第一层：先会用：能把全局域分给多个 GPU，并识别 halo。 **第二层：再理解：**能写出每步计算与通信顺序。 **第三层：最后优化：**能分析强/弱扩展、拓扑和通信重叠的上限。

18.11 先定义 ownership

以二维 $H \times W$ stencil 为例，按行分给 P 个 GPU。GPU p 拥有全局行区间 $[s_p, e_p)$ ，本地存储除 owned rows 外，还需上下 halo。每次迭代：

1. 用本地边界行打包/直接发送给相邻 GPU；
2. 接收邻居 halo；
3. 计算不依赖新 halo 的 interior；
4. halo 到达后计算 boundary；
5. 交换输入输出缓冲进入下一步。

首先要保证全局边界、内部 GPU 边界和索引偏移正确，再讨论通信重叠。

18.12 通信量与计算量的表面积/体积比

每 GPU 子域高约 H/P 、宽 W 。每步计算量与面积 HW/P 成正比，halo 通信与两条边长度 $2W$ 成正比。通信/计算比例约

$$\frac{2W}{HW/P} = \frac{2P}{H}.$$

强扩展时全局 H, W 固定，增加 P 会使比例线性增大，最终通信主导；弱扩展保持每 GPU 子域大小不变，比例近似稳定。

✂ 完整例题

$H = 16384$ ，从 1 GPU 扩到 128 GPU，按行分解时每 GPU 只有 128 行。若 stencil 半径较大或每点计算很少，halo 与 kernel 启动可能成为主要成本。仅增加 GPU 不能无限缩短时间。

18.13 CUDA-aware MPI 的语义

CUDA-aware MPI 允许发送/接收缓冲区直接是 device pointer，避免用户显式把数据复制到 host staging buffer。典型过程：

Listing 18.2 概念性 halo exchange 顺序

```
1 MPI_Irecv(d_top_halo, count, MPI_FLOAT, up, tag, comm, &req[0]);
2 MPI_Irecv(d_bot_halo, count, MPI_FLOAT, down, tag, comm, &req[1]);
3 MPI_Isend(d_first_owned, count, MPI_FLOAT, up, tag, comm, &req[2]);
4 MPI_Isend(d_last_owned, count, MPI_FLOAT, down, tag, comm, &req[3]);
5 // launch interior kernel while transfers progress
6 MPI_Waitall(4, req, MPI_STATUSES_IGNORE);
7 // launch boundary kernel
```

实际是否直接 GPU-GPU、是否经 host、如何与 CUDA stream 同步取决于 MPI 实现、网络和拓扑，必须用实际系统验证。

18.14 NCCL 更适合 collective

NCCL 提供 all-reduce、all-gather、reduce-scatter、broadcast 等 GPU collective，常用于数据并行梯度同步、张量并行通信。它擅长根据 NVLink/PCIe/网络拓扑组织 ring/tree 等算法。点对点 halo 可用 send/recv，但规则 stencil 的邻居通信与 MPI/NVSHMEM 也很自然。

18.15 NVSHMEM 的 one-sided 视角

NVSHMEM 提供对称内存和 put/get, GPU kernel 可主动访问远端 PE 的内存, 并使用相应同步。它能减少 host 驱动的通信步骤, 并支持更细粒度 GPU-initiated 通信; 代价是程序员必须更谨慎地管理远端可见性、完成语义和同步。

18.16 计算通信重叠的必要条件

理想每步时间

$$T_{step} \approx T_{interior} + T_{boundary} + T_{comm}$$

若 interior 与通信完全重叠, 可接近

$$T_{step} \approx \max(T_{interior}, T_{comm}) + T_{boundary}.$$

要实现重叠, 需要:

- 非阻塞通信或异步 collective;
- interior 与 halo 数据无依赖;
- CUDA stream、通信库和事件正确衔接;
- GPU copy engine、网络与计算资源可并发;
- interior 足够大, 能覆盖通信时间。

把一个大 kernel 简单放在另一个 stream 并不保证重叠; 资源占满、隐式同步或库语义都可能阻止并发。

18.17 强扩展与弱扩展公式

强扩展固定总问题规模:

$$S_P = \frac{T_1}{T_P}, \quad E_P = \frac{S_P}{P}.$$

弱扩展固定每 GPU 问题规模, 理想情况下 T_P 接近 T_1 ; 可报告弱扩展效率 $E_P^{weak} = T_1/T_P$ 。所有时间应明确是否包含初始化、数据分发和最终汇总。

18.18 多 GPU 正确性检查

常见错误包括:

- 全局行号与本地含 halo 行号混淆；
- 发送错误边界或 count；
- 复用发送缓冲时通信尚未完成；
- 下一代提前读取未到达 halo；
- 不同 rank 使用不匹配 tag/collective 顺序；
- 只在单 GPU 验证，未测试非整除分区。

建议用小网格打印每 GPU owned/halo 区间，并与单 GPU CPU/GPU 结果逐步比较。

分层实验

实验 1：手算 $H = 10, P = 3$ 的不均匀行分区和 halo 索引。

实验 2：实现同步式 halo exchange，先保证正确。

实验 3：拆 interior/boundary kernel，引入非阻塞通信，使用时间线验证是否重叠。

实验 4：做 1、2、4、8 GPU 强扩展，报告效率和通信比例，而不只报告加速比。

第 19 章 综合项目：把知识变成可复现的 GPU 优化能力

📖 **原书对应：**对应原书前言推荐的两阶段学习方式与最终项目思想，并综合第 2–23 章。

🎯 本章学习目标

- 独立完成“基线 - 正确性 - 分析 - 优化 - 报告”的闭环；
- 避免只展示最快结果而缺少过程和证据；
- 建立可重复、可解释、可比较的性能实验；
- 用一份项目证明已掌握本书的计算思维。

19.1 项目不是“写一个很长的 kernel”

优秀项目应回答一个明确问题：

在给定输入族和硬件条件下，哪种问题分解、数据布局与优化组合最有效，为什么？

项目价值来自假设、设计、测量和解释，而不是代码行数。

19.2 推荐项目一：二维卷积优化阶梯

基线：CPU 参考 + GPU 直接卷积。

优化：constant mask、shared tile、显式 halo、cache halo、可分离卷积、向量加载、融合。

变量：图像尺寸、核大小、batch、边界模式。

报告：有效带宽、输出像素/s、算术强度、边界开销、shared memory 与 occupancy。

19.3 推荐项目二：归约/扫描原语库

实现 sum/max reduction、inclusive/exclusive scan 和 stable filter。比较 shared、warp primitive、粗化、多阶段全局扫描。要求支持任意长度并提供自动测试。

19.4 推荐项目三：小型 attention kernel

实现单头或多头 attention 的教学版本：

1. 朴素 score kernel；
2. 稳定 softmax；
3. $\text{probability} \times V$ ；
4. tiled/融合版本，避免完整 probability 写回；
5. causal mask 与不规则序列长度。

与高层框架结果比较，不要求超越成熟库。重点是量化中间 IO 和数值误差。

19.5 推荐项目四：稀疏矩阵格式自适应

实现 CSR 一线程一行、CSR 一 warp 一行、ELL/HYB。根据行长统计选择 kernel，测试规则 PDE 矩阵、幂律图矩阵和人工极端矩阵。

19.6 推荐项目五：多 GPU stencil

从单 GPU 2D stencil 扩展为多 GPU：域分解、halo exchange、内部/边界分离、通信重叠。报告强/弱扩展和不同消息大小。

19.7 项目设计文档模板

19.7.1 1. 问题与动机

说明输入、输出、应用意义、规模与性能要求。不要用“为了学习 CUDA”代替具体目标。

19.7.2 2. 数学与顺序基线

给出公式、边界和 CPU 参考实现复杂度。明确精度与容差。

19.7.3 3. 并行分解

画线程/warp/block 到数据的映射，说明：

- 每线程输出；
- 输入复用与共享数据；
- 同步和竞争；
- 非整块边界；
- 预计瓶颈。

19.7.4 4. 优化假设

每项优化写成可验证假设，例如：

“shared tile 将每个输入从 global memory 的平均加载次数从约 9 次降低到接近 1-2 次，因此大图像上有效带宽和时间应改善；小图像可能因同步和启动开销收益有限。”

19.7.5 5. 验证计划

至少包含：随机输入、结构化输入、边界规模、极小规模、非 block 整数倍、重复运行、CPU 对照、误差统计。

19.7.6 6. 性能计划

说明：

- 计时范围（kernel-only / transfer / end-to-end）；
- warm-up 和重复次数；
- 输入矩阵；
- 基线与库；
- 硬件和编译参数；
- 需要采集的 profiler 指标。

19.8 实验表格模板

版本	时间/ms	吞吐	寄存器/线程	shared/block	结论
CPU 参考			—	—	正确性基准
GPU 基线					初始瓶颈
优化 1					单变量变化
优化 2					是否符合假设
库实现			—	—	现实上限参考

19.9 常见无效实验

⚠ 易错点

- 只运行一次并报告最小值；
- GPU 只计 kernel、CPU 却计内存申请和初始化；
- 优化前后精度或算法语义不同；
- 使用太小输入，固定开销支配结果；
- 同时改 block、数据布局、算法和编译选项，无法归因；
- 只测均匀随机数据，忽略热点或不规则分布；
- 只报告加速比，不报告绝对时间和硬件上限；
- 错误结果“更快”，却没有逐元素验证。

19.10 最终报告结构

1. 摘要：问题、方法、主要结果；
2. 背景与公式；
3. 顺序基线和正确性；
4. GPU 分解与数据布局；
5. 优化阶梯；
6. 实验设置；
7. 结果、profiler 证据和误差；

8. 失败尝试与原因；
9. 局限和下一步；
10. 可复现命令。

19.11 毕业检查：能否回答这些问题

掌握检查

1. 为什么这个问题有并行性？依赖在哪里？
2. 每个线程负责什么？为什么这种映射有利于访存？
3. 哪些数据从 global memory 读取，复用多少次？
4. 需要哪些 barrier/atomic？能否缩小作用域或减少次数？
5. warp 是否发散？边界发散占多少比例？
6. kernel 是计算、带宽、延迟、同步还是负载受限？证据是什么？
7. occupancy 由什么资源限制？提高它会不会伤害寄存器复用？
8. 优化后的理论上限和实际差距是多少？
9. 换 GPU、换输入分布或换规模后，结论是否仍成立？
10. 结果如何自动验证和复现？

如果能对自己的项目给出清晰证据，就已经掌握了本书真正要训练的能力。

19.12 由浅入深的项目执行计划

从浅到深

第一层：先会用：完成正确的 CPU 与 CUDA 基线。 **第二层：再理解：**用模型和 profiler 找出一个主要瓶颈。 **第三层：最后优化：**形成可复现、可解释、可迁移的优化报告。

19.13 六个里程碑

1. **第 1 周：问题定义。** 写清输入、输出、边界、精度和规模；准备 CPU 参考与小测试。

2. **第 2 周：GPU 基线。**使用最直接映射，保证所有边界规模正确，记录端到端与 kernel 时间。
3. **第 3 周：定量模型。**计算 FLOP、必要字节、算术强度、理论并行度和同步/原子次数。
4. **第 4 周：第一瓶颈。**使用 profiler 验证是访存、计算、发散、同步还是负载不均。
5. **第 5–6 周：优化阶梯。**一次一个改动，保留每个版本和实验记录。
6. **第 7 周：泛化与报告。**测不同规模、形状和数据分布，说明方法在哪些条件下有效或失效。

19.14 每次提交必须回答的五句话

1. 这一版与上一版唯一差异是什么？
2. 它理论上减少了多少 global bytes、barrier、atomic 或指令？
3. 正确性如何验证，误差是否变化？
4. 实际性能变化多少，测量边界和波动是什么？
5. profiler 是否支持原假设，新的瓶颈是什么？

若不能回答，说明优化过程仍是“试参数”，尚未形成工程能力。

19.15 建议的最小最终成果

- 一份可独立构建的源码和运行脚本；
- CPU 参考、至少 4 类边界测试、随机测试；
- 至少 4 个逐级 CUDA 版本；
- 一张理论模型表、一张性能表、一张 profiler 对照表；
- 对失败优化的分析，而不是只保留最好结果；
- 说明硬件、驱动、CUDA、编译选项和 GPU 时钟/功耗条件。

✂ 完整例题

二维卷积项目的合格优化链：

1. naive global-memory kernel；

2. weight 放 constant memory;
3. shared input tile + halo;
4. 调整输出/input tile 映射;
5. 每线程多个输出或寄存器滑窗;
6. 双缓冲/融合 (仅在 profiler 支持时)。

每一步都应给出输入流量估算和实测。若 shared 版本在小图更慢，应解释固定开销和低复用，而不是隐藏结果。

完成本教程的真正标准

面对一个未见过的并行问题，你能先写数学定义和依赖图，选择线程粒度与数据布局，建立正确基线；随后用算术强度、访存事务、资源和同步分析瓶颈；最后通过受控实验验证优化，而不是只会复述 CUDA 名词。

第五部分

附录

附录 A 数值计算基础：浮点表示、误差与低精度

■ **原书对应：**原书附录 A：浮点表示、可表示数、IEEE 特殊值、舍入、低精度格式、算法考虑与线性求解稳定性。

A.1 浮点数不是实数

常见二进制浮点数近似表示为

$$x = (-1)^s \times m \times 2^e,$$

其中符号、有效数和指数位数有限。多数十进制小数不能被精确表示，因此运算会舍入。

A.2 机器精度与误差模型

对正常范围内的一次基本运算，常用模型

$$\text{fl}(a \circ b) = (a \circ b)(1 + \delta), \quad |\delta| \lesssim u,$$

u 是 unit roundoff 的量级。长计算链会累积误差，运算顺序改变也会改变舍入。

A.3 特殊值

IEEE 浮点包括 $+0, -0, +\infty, -\infty$, NaN 和 subnormal。GPU kernel 中应警惕：

- 除零或溢出产生 infinity；
- 非法操作产生 NaN，并在后续传播；
- NaN == NaN 为假；
- 过小值下溢，某些模式可能 flush-to-zero；
- fast math 可能改变精度和特殊值行为。

A.4 并行归约的非确定末位

浮点加法不满足严格结合律：

$$(a + b) + c \neq a + (b + c)$$

在有限精度下可能成立。并行线程调度或归约树改变求和顺序，因此结果可能在末位波动。验证应使用容差并报告最大绝对/相对误差，而非盲目逐位相等。

A.5 低精度的收益与风险

更低位宽可提高吞吐、降低显存和带宽需求，但动态范围、有效精度和特殊值支持不同。实践中常采用混合精度：低精度保存/乘法，更高精度累加或维护关键统计量。

选择低精度前检查：

- 数据动态范围；
- 累加项数量；
- 梯度/概率是否接近零；
- 是否需要 loss scaling 或量化尺度；
- 误差是否影响最终任务指标，而不仅是元素误差。

A.6 稳定算法比更高精度更重要

稳定 softmax 使用减最大值；线性方程避免显式求逆；归一化采用稳定方差计算；求和可使用 pairwise 或补偿策略。先选择数值稳定的公式，再考虑提高数据类型。

A.7 误差报告模板

给参考 r_i 和结果 x_i ：

$$\begin{aligned} E_{\infty} &= \max_i |x_i - r_i|, \\ E_{\text{rel}} &= \max_i \frac{|x_i - r_i|}{\max(|r_i|, \epsilon)}, \\ E_{\text{RMSE}} &= \sqrt{\frac{1}{n} \sum_i (x_i - r_i)^2}. \end{aligned}$$

同时检查 NaN/Inf 数量和任务级指标。

动手练习

构造 $[10^8, 1, -10^8]$ 的 float 求和，比较不同顺序。再比较 float 累加与 double 累加一百万个小数的误差。

附录 B CUDA 快速参考与性能检查表

B.1 最常用索引公式

$$\begin{aligned}i &= \text{blockIdx.x} \text{blockDim.x} + \text{threadIdx.x}, \\x &= \text{blockIdx.x} \text{blockDim.x} + \text{threadIdx.x}, \\y &= \text{blockIdx.y} \text{blockDim.y} + \text{threadIdx.y}, \\idx_{2D} &= yW + x, \\idx_{3D} &= zHW + yW + x, \\\text{blocks} &= \frac{n + T - 1}{T}.\end{aligned}$$

B.2 核心 API

API/语法	作用
<code>cudaMalloc(&p, bytes)</code>	申请设备 global memory
<code>cudaFree(p)</code>	释放设备内存
<code>cudaMemcpy(dst, src, n, kind)</code>	主机/设备之间或设备内复制
<code>kernel<<<grid, block, smem, stream>>>(...)</code>	启动 kernel; 后两个配置可选
<code>cudaGetLastError()</code>	获取最近的启动错误
<code>cudaDeviceSynchronize()</code>	等待设备并暴露异步错误
<code>__syncthreads()</code>	block 级 barrier; 整个 block 必须一致到达
<code>__syncwarp(mask)</code>	warp 级同步
<code>atomicAdd(...)</code>	原子读-改-写
<code>cudaEventRecord/ElapsedTime</code>	GPU 时间线计时

B.3 kernel 正确性检查

- ✔ 所有数组字节数正确, 指针地址空间正确;

- ✔ grid 足够覆盖数据，边界判断无越界；
- ✔ 二维 x/y 和 row/col 未颠倒；
- ✔ 有 barrier 时所有 block 线程一致到达；
- ✔ 共享数据读前已写并同步，重写前已用完并同步；
- ✔ 并发写同一位置时使用正确原子/归约/所有权；
- ✔ kernel 启动和同步错误均检查；
- ✔ 与 CPU 参考和极端输入比较。

B.4 性能检查

- ✔ 先确定端到端瓶颈，不只盯 kernel；
- ✔ 相邻 lane 访问相邻且对齐的 global 地址；
- ✔ 重复数据是否进入 shared/register/cache；
- ✔ 算术强度是否足以使用计算单元；
- ✔ warp 控制流是否一致；
- ✔ 原子热点是否可私有化/聚合；
- ✔ barrier 数量和作用域是否最小；
- ✔ shared memory 是否 bank conflict；
- ✔ 寄存器/shared 是否造成 occupancy 悬崖；
- ✔ block 数是否足够、最后 wave 是否过小；
- ✔ 是否可用粗化、展开、预取、双缓冲；
- ✔ 优化后是否仍正确，是否跨规模稳定。

B.5 模式选择速查

问题特征	典型模式	关键技术
局部邻域	卷积/stencil	halo tile、constant、register window
多线程更新少数桶	histogram	atomic、私有化、粗化
全体变一个值	reduction	树、shared、shuffle、多阶段
每项需要前缀	scan	Kogge-Stone、分层 scan、look-back
压缩满足条件项	filter	原子或 predicate+scan+scatter
两个有序序列	merge	co-rank、tiled merge
整数键多轮分桶	radix sort	histogram+scan+stable scatter
强局部复用矩阵	GEMM	多级 tiling、寄存器、流水
二维递推依赖	wavefront	对角线、tile、坐标变换
不规则行/邻接	sparse/graph	格式选择、frontier、动态负载
跨设备邻域	multi-GPU stencil	域分解、halo、重叠

附录 C 部分练习答案与推导

本附录提供关键题的解题路径，不覆盖所有开放实验。建议先独立完成，再核对。

C.1 第 1 章：线性化

500×400 行主序矩阵中 $(20, 10)$:

$$20 \times 400 + 10 = 8010.$$

$300 \times 500 \times 400$ 张量中 $(z, y, x) = (5, 20, 10)$:

$$5 \times 500 \times 400 + 20 \times 400 + 10 = 1,008,010.$$

C.2 第 2 章：Amdahl 定律

$p = 0.9, s = 20$:

$$S = \frac{1}{0.1 + 0.9/20} = \frac{1}{0.145} \approx 6.90.$$

即并行部分加速 20 倍，整体只有约 6.9 倍。若要整体至少 10 倍，需

$$(1 - p) + p/s \leq 0.1.$$

若 $s \rightarrow \infty$ ，必须 $p \geq 0.9$ ；有限 s 时需要更高 p 。

C.3 第 3 章：线程数量

$n = 1000, T = 256$:

$$B = (1000 + 255)/256 = 4,$$

总线程 1024，多余 24。全局索引由 $i = bT + t$ 给出。

C.4 第 4 章：二维图像覆盖

$H = 62, W = 76$, block 16×16 :

$$G_y = \lceil 62/16 \rceil = 4, \quad G_x = \lceil 76/16 \rceil = 5.$$

总线程 $4 \times 5 \times 256 = 5120$ ，有效像素 $62 \times 76 = 4712$ ，多余线程 408。

C.5 第 5 章: warp 数

block 48 线程需要 $\lceil 48/32 \rceil = 2$ warps, 第二 warp 仅 16 个有效线程。128、256、768 分别为 4、8、24 warps, 均是 32 的整数倍。

C.6 第 6 章: tiled GEMM 边界

$$M = 5, N = 7, K = 6, T = 4:$$

$$G_y = \lceil 5/4 \rceil = 2, \quad G_x = \lceil 7/4 \rceil = 2, \quad \text{phases} = \lceil 6/4 \rceil = 2.$$

第二个 K phase 只有 $k = 4, 5$ 有效, tile 中对应 $k = 6, 7$ 的加载补 0。输出右下 block 中只有 row 4、col 4-6 有效。

C.7 第 8 章: halo 比例

二维 $T = 16, R = 1$, 完整输入 tile 为 $18 \times 18 = 324$, 输出为 $16 \times 16 = 256$ 。额外元素比例相对输出为

$$\frac{324 - 256}{256} \approx 26.6\%.$$

三维同参数为 $18^3/16^3 \approx 1.424$, halo 相对开销更明显。

C.8 第 10 章: 归约 partial 数量

若每 block 有 256 线程, 每线程加载两个元素, 则每 block 最多处理 512 个输入。 $n = 1000$ 需要

$$\lceil 1000/512 \rceil = 2$$

个 block, 第一阶段输出 2 个 partial sums。第二阶段一个 block 即可完成。

C.9 第 11 章: 稳定过滤

输入 $x = [8, 3, 6, 1, 4]$, 保留偶数:

$$f = [1, 0, 1, 0, 1].$$

exclusive scan:

$$s = [0, 1, 1, 2, 2].$$

对 $f_i = 1$ 的位置写 $\text{out}[s[i]] = x[i]$, 得到 $[8, 6, 4]$ 。

C.10 第 12 章: co-rank 小例子

$A = [1, 4, 7, 9]$, $B = [2, 3, 8]$, 合并结果

$$C = [1, 2, 3, 4, 7, 8, 9].$$

输出前缀长度 $k = 0 \dots 7$ 对应从 A 取的数量 i 可列为

$$[0, 1, 1, 1, 2, 3, 3, 4],$$

从 B 取 $j = k - i$ 。例如 $k = 5$, 前五项 $[1, 2, 3, 4, 7]$ 来自 A 前 3 项和 B 前 2 项, 所以 co-rank 为 $(3, 2)$ 。

C.11 第 13 章: 寄存器 tile

block 输出 128×128 , 若每线程输出 8×8 , 所需线程数

$$(128/8) \times (128/8) = 16 \times 16 = 256.$$

每线程至少 64 个累加器, 另需 A/B 片段、地址和流水寄存器, 寄存器压力很高。实际实现可能改变 tile、数据类型或分配策略。

C.12 第 16 章: attention 形状

$L = 1024, d = 4096, h = 32$, 每头 $d_h = 128$ 。单头

$$Q, K, V : 1024 \times 128, \quad S, P : 1024 \times 1024, \quad O : 1024 \times 128.$$

所有头输出拼接回 1024×4096 。

C.13 第 16 章: KV cache 估算

若 batch $B = 8$ 、上下文 $L = 4096$ 、层数 32、模型维度 $d = 4096$ 、每元素 2 B, 且 K/V 头总维度等于 d :

$$M = 2 \times 8 \times 4096 \times 32 \times 4096 \times 2 = 17,179,869,184 \text{ B} \approx 16 \text{ GiB}.$$

若 K/V 头数减少, 应使用实际 KV 维度重新计算。

C.14 第 18 章：三 GPU 行分区

总高度 1000，宽 800。可近似均分为 334、333、333 行。例如：

- GPU 0：全局行 0–333；需要下 halo 行 334；
- GPU 1：全局行 334–666；需要上 halo 333、下 halo 667；
- GPU 2：全局行 667–999；需要上 halo 666。

物理外边界按问题定义处理，不从不存在的邻居接收。

附录 D 术语表

术语	含义
Arithmetic intensity	算术强度，单位字节数据搬运对应的计算量，常用 FLOP/B。
Atomic operation	不可被并发线程打断的读-改-写操作，用于避免丢失更新。
Bank conflict	同一 warp 对 shared memory 同一 bank 的不同地址并发访问导致串行化。
Barrier	一组线程必须全部到达后才能继续的同步点。
Block	CUDA 线程块；同 block 线程被放在同一 SM，可共享 shared memory 并做 block barrier。
Coalescing	合并访存；将相邻线程的内存访问组合为较少事务。
Co-rank	并行 merge 中，输出前缀对应的两个输入前缀切点。
Control divergence	同一 warp 线程走不同控制路径，硬件分多次执行路径。
Device	CUDA 语境中的 GPU 执行设备。
Global memory	容量大的设备内存，所有线程可访问，延迟高、带宽重要。
Grid	一次 kernel 启动产生的全部线程块集合。
Halo / ghost cells	分块邻域计算中，为计算本块边界而需要的相邻块数据。
Host	CUDA 语境中的 CPU 及其主机代码/内存。
ILP	指令级并行，单线程/warp 中多个独立指令可重叠。
Kernel	在 GPU 上由大量线程执行的函数。
Latency hiding	一个 warp 等待时调度其他 ready warp，以隐藏长延迟。
MLP	内存级并行，同时存在多个独立内存请求。
Occupancy	实际驻留 warp 数相对硬件最大 warp 数的比例。
Privatization	为 block/warp/thread 创建局部副本，降低共享更新竞争，最后合并。
Reduction	用结合运算把多个元素汇聚成一个或少量结果。

Register tiling	把多个输出或滑动窗口保存在寄存器中以提高线程内复用。
Scan	前缀运算，为每个位置计算此前元素的聚合。
Shared memory	位于 SM 的软件管理片上存储，同 block 线程共享。
SIMD	单指令多数据硬件执行方式。
SIMT	CUDA 的单指令多线程编程/执行抽象。
SPMD	单程序多数据；线程执行同一程序但处理不同数据。
Tail effect	最后一 wave block 数不足，导致部分硬件空闲。
Thread coarsening	每线程处理多个输出或输入，减少线程/同步开销并增加复用。
Tiling	把数据/计算划分为可在片上内存中复用的小块。
Warp	常见 32 线程的硬件调度与执行组。
Wave	同时驻留执行的一批 blocks；整个 grid 可能需要多个 waves。
Wavefront	按依赖前沿推进的并行模式，常见于动态规划。
Work efficiency	并行算法总工作量相对最佳顺序算法是否保持同阶。

结语与参考教材

这份教程的目标不是替代原书的 673 页内容，而是提供一条可执行的主线：先掌握线程到数据的映射，再理解硬件与内存，随后通过并行模式积累可迁移的设计经验，最后进入 AI、图计算和多 GPU。真正的掌握标准是能够对一个新问题独立完成正确实现、性能分析、优化选择和可复现实验。

主要依据

参考文献

- [1] Wen-mei W. Hwu, David B. Kirk, Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*, Fifth Edition, Morgan Kaufmann / Elsevier, 2027.

版权与使用说明

本教程基于用户提供的教材进行学习性总结、重组和原创讲解。未复制原书版式与插图；代码为教学目的重新编写的示例。涉及 CUDA 具体 API、硬件限制和工具行为时，应以所使用版本的官方文档与实际设备查询结果为准。

从“一个线程负责什么”开始，最终学会回答“整个系统为什么这样设计”。